

MA 551: Computational Statistics

JMR Chapters 1, 2, and 3.1-3.4

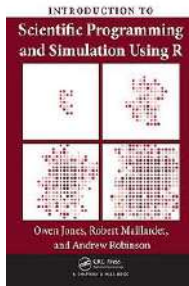
Fangfang Wang

Spring 2026

Texts

(JMR) Jones, Mailardet and Robinson, 2009, “Introduction to Scientific Programming and Simulation Using R”, 2nd Edition, ISBN-978-1-4665-6999-7

Rizzo, 2007, “Statistical Computing with R”, ISBN-10: 1-58488-545-9



Additional references

- *An Introduction to Statistical Learning: with Applications in R*. 2014, by G. James, D. Witten, T. Hastie, and R. Tibshirani. Springer-Verlag New York. ISBN 978-1-4614-7137-0.
- *Pattern Recognition and Machine Learning*, by Chris Bishop. Springer-Verlag New York. ISBN: 978-1-4939-3843-8.
- *Computational Statistics, 2nd Edition*, by Geof H. Givens and Jennifer A. Hoeting. Wiley. ISBN: 978-0-470-53331-4.

Software

This course will make extensive (and exclusive) use of **R**

- Language in which almost all statistical research is conducted.
- Free! Available for Windows, Linux, Mac.
- See www.r-project.org for installation details.
- Includes a basic text editor for code.
- More sophisticated editors available (Tinn-R, Emacs, RStudio, others...), use as you wish, let me know if there's one not here that you particularly like.
- Refresh material: An Introduction to R, by William N. Venables, David M. Smith and the R Development Core Team



Communication & Homework

- All course announcements and materials will be posted on Canvas.
- Slides are pedagogical tools, not substitutes for notes.
- `R Markdown` would be used extensively in homework assignments. Here are some useful links:
 - R Markdown cheat-sheet
`https://www.rstudio.com/wp-content/uploads/2015/02/rmarkdown-cheatsheet.pdf`
 - `Knitr`
`http://kbroman.org/knitr\_knutshell/`

Learning Outcomes – Or What's The Point?

I aim to teach you enough programming and enough statistics that you will be able to

- Program your own statistical methods “from scratch”.
- Design ways to assess whether a procedure (t-test, linear regression) provides reliable answers.
- Develop computational procedures to calculate confidence intervals and conduct tests when “standard” methods are not valid.
- Understand, detect, and predict where and why valid statistical procedures may not be computable.

This will require some programming, some numerical analysis, some probability and statistics and some math.

Topics

- Linear models, central limit theorems, and simulation.
- Permutation tests, bootstrap, and Jackknife.
- Generating random variables: inversion method, rejection sampling
- Numerical integration and Monte Carlo Integration.
- Variance Reduction: importance sampling
- Maximizing, minimizing, and finding roots.
- Expectation–Maximization (EM) algorithm.
- Bayesian Inference and Computation.
- Statistical Text Mining.

Assumed Math

This course will make considerable (practical) use of the following:

- Matrix and vector operations – addition, multiplication, inverses.
- Differentiation and root finding.
- Probability: expectations (as integrals), probability laws, Bayes theorem.
- Basic statistical concepts: sampling distributions, p -values, test statistics.

Some Notes on What to Expect

Programming

- Programming $\neq$ following a recipe, programming = *creating* the recipe. Homework cannot be copied from lecture material - code will need modification.
- Coding takes much more time than you think – **leave plenty of time for homework.**
- Not just writing, but debugging – code must be *exactly* correct. Finding errors can be long and painful.
- Efficiency improves with practice (and only with practice).

Basics of R

- R is a calculating environment that is controlled by text commands.
- When you start R, you will get a prompt:
>
- This is where you type in commands.
- Commands can be single line calculations (like a calculator) or instructions to carry out a statistical analysis, or computer programs.
- Additionally, you can give objects (numbers, vectors, text strings) names to be stored in memory that can be recalled.
- Statistical analysis: like a GUI (JMP, Minitab, SPSS,...) but instructions are written rather than in menus.
- One advantage: you can keep a record of exactly what you have done and copy and paste it into the prompt to repeat.

Calculation

Using R like a calculator:

```
> 5+3  
[1] 8
```

Standard operations: +, -, *, /. Also ^ for power.

Can also use brackets – these matter!

```
> 2+3^1.2  
[1] 5.737193  
> (2+3)^1.2  
[1] 6.898648
```

Unlike many calculators, R will not remember the last value computed.

Logical Operations And Other Data Types

R can test the truth of mathematical statements with inequalities `<`, `<=`, `>`, `>=`

```
> 2*3 > 6
```

```
[1] FALSE
```

```
> 2*3 >= 6
```

```
[1] TRUE
```

For equality use `==` (see below for `=`)

```
> 2*3 == 6
```

```
[1] TRUE
```

We can also look at 'and' and 'or' statements with `&` and `|`

```
> TRUE & FALSE
```

```
[1] FALSE
```

```
> TRUE | FALSE
```

```
[1] TRUE
```

```
> (5.2*3 > 14) | (pi < 3)
```

```
[1] TRUE
```

Functions

- More operations are coded in functions.
- These called by a name with inputs (arguments) in brackets

```
> sqrt(2)
```

```
[1] 1.414214
```

and separated by commas

```
> choose(5,2)
```

```
[1] 10
```

- You can get help for these by typing

```
> help(choose)
```

- Note that arguments have names, but also a default order:

```
> choose(n=5,k=2)
```

```
[1] 10
```

```
> choose(k=2,n=5)
```

```
[1] 10
```

```
> choose(2,5)
```

```
[1] 0
```

Recording Operations – Code and Commenting

- You can keep track of your operations by writing them in a text editor.
- In R's editor, `ctrl-R` sends highlighted text to the prompt.
- You need to record one input per line.
- Very important character: `#`, R ignores everything following this character on the line.
- Use this to document what your code does (important, esp. for assignments)

Example:

```
# Here we demonstrate using argument names
```

```
choose(n=5, k=2)  # is the same thing as  
choose(k=2, n=5)  # but not the same as  
choose(2, 5)
```

Storing Objects, Vectors and Matrices

- R will also allow you to name objects and recall their value

```
> x=5  
> x  
[1] 5  
> x+3  
[1] 8
```

- There are two *assignment* commands: “=” and “<-”

```
y <- x+2  
y  
[1] 7
```

- These are the same; “=” is used by many other languages, “<-” is harder to confuse with “==”.
- Different people have different preferences.
- Note above that `y` stores the value 7, if we change `x`, `y` doesn't change.

Vectors

You can create and store strings of numbers (vectors)

```
> x = c(1, 5.2, 3.4, 2.1)
> x
[1] 1.0 5.2 3.4 2.1
```

also

```
> (y = 1:4)
[1] 1 2 3 4
> (z = rep(1, 4))
[1] 1 1 1 1
```

Elements can be referenced by square brackets

```
> x[3]
[1] 3.4
> y[2:4]
[1] 2 3 4
```

You can also assign things to elements (and add vectors)

```
> z[2:3] = x[2:3] + y[2:3]
> z
[1] 1.0 7.2 6.4 1.0
```


Matrices

Sometimes it's useful to have more dimensions.

```
> A = matrix(1:6,nrow=2,ncol=3,byrow=TRUE)
```

```
> A
```

	[,1]	[,2]	[,3]
[1,]	1	2	3
[2,]	4	5	6

I can obtain elements of `A` by separating row and column by commas

```
> A[2,1]
```

```
[1] 4
```

gives the element in the 2nd row and 1st column

```
> A[1:2,c(1,3)]
```

	[,1]	[,2]
[1,]	1	3
[2,]	4	6

produces a submatrix from the first two rows and 1st and 3rd columns.

More Matrix Indices

Two other useful index conventions:

- 1 Empty indices means all rows (or columns)

```
> A
```

```
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

```
> A[,2:3]
```

```
      [,1] [,2]
[1,]    2    3
[2,]    5    6
```

- 2 Negative indices mean remove that element

```
> A[1,-2]
```

```
[1] 1 3
```

Repeating Operations – Loops and Ifs

- Often, you need to repeat the same sequence of operations many times.
- You might also need to make decisions automatically.
- Three helpful programming structures:
 - `for loops` take you through a fixed set of operations.
 - `while loops` repeat until a condition is true.
 - `if statements` only execute if a condition is true.

for Loops

Suppose we wanted element-wise minimum of `x` and `y`:

```
> x
[1] 1.0 5.2 3.4 2.1
> y
[1] 1 2 3 4
```

that is, we want the minimum of `x[1]` and `y[1]`, then the minimum of `x[2]` and `y[2]`, ...

We'll store this in `m`

```
> m = rep(0, 4)
```

now loop over the 4 elements of `x` and `y`

```
> for(i in 1:4){
+ m[i] = min(x[i], y[i])
+ }
> m
[1] 1.0 2.0 3.0 2.1
```

Anatomy of a `for` loop

```
> for(i in 1:4){ m[i] = min(x[i],y[i]) }
```

- `(i in 1:4)`: counter `i` in a set of objects `1:4`
- Do whatever is within '`{ }`' with the counter equal to each element in the listed set.
- Can have multiple lines of code within '`{ }`'.
- Set could be vector you like, can name vector anything as well

```
> for(g in x){ print(g) }  
[1] 1  
[1] 5.2  
[1] 3.4  
[1] 2.1
```

(the `print` command is very useful for seeing what is going on inside a loop)

A More Complex Example

I want the sum of $\sqrt{k}$ recorded for k from 1 to 50:

```
# First set up a vector to record the sums
```

```
res = rep(0,50)
```

```
# Now we'll start with the sum = 0
```

```
ss = 0
```

```
# And loop
```

```
for(k in 1:50){
```

```
  ss = ss + sqrt(k)
```

```
  res[k] = ss
```

```
}
```

Note indenting lines within '{ }' makes code much more readable.

while loops

How would we find the smallest k such that

$$\sqrt{1} + \sqrt{2} + \dots + \sqrt{k} > 100?$$

Keep track of which number we're up to:

```
k = 0; ss = 0
```

```
while(ss <= 100){  
    k = k + 1  
    ss = ss + sqrt(k)  
    print(c(k,ss))  
}
```

R keeps repeating the commands inside '{ }' until the condition `ss <= 100` is `FALSE`.

Note that `k` updated to 28, *before* `ss` is updated, so we do run once with `ss > 100`.

if statements

Are used to decide on an action

```
> if( res[k] > 100 ){  
+   print('Hello World')  
+ }  
[1] "Hello World"
```

if we set `k` to 27, `R` will return nothing.

We can also define the action if the condition is `FALSE`

```
if( res[k] > 100 ){  
  print('Hello World')  
} else{  
  print('Stuff You')  
}
```


These Can All Be Nested

```
for(k in 1:50){  
  if( res[k] > 100 ){  
    print('Hello World')  
  } else{  
    print('Stuff You')  
  }  
}
```

Plotting

- Graphical displays are one of the most important parts of applied statistics.
- R provides very flexible (if tedious to use) graphics, plus many additional libraries for specialized graphs.
- Basic tool is `plot`
`plot(x=1:50,y=res)`
- To which you can add a title
`title('Plot of sum sqrt(k)')`
- And straight lines
`abline(h=100)`
- And lines and points and many other features.

Summary

Basic R functions:

- Storing numbers, vectors, matrices
- Calculations; numeric + logical
- Loops, conditional statements
- Plotting

Many functions have multiple options (especially `plot`); learning to use help files effectively is very important.

Reading and Writing Data Complex Data Structures Writing Functions

JMR Chapters 4, 5, and 6

Fangfang Wang
MA 551

Outline

- 1 Reading/Writing data
 - `read.table` plus data frames, `load`, `save` and others.
- 2 Complex Objects
 - data frames, factors, lists
- 3 Functions
 - structure of a function
 - good coding practice
 - environments

Reading Data

- Statistics requires data – we need a way to get things in and out of R.
- Generally we'll expect data in ASCII text files (.txt, .csv, .dat)
- Function `read.table` will read these values in

```
ships = read.table('ships.csv', sep=',', head=TRUE)
```
- `ships.csv` is a file in my working directory
- `sep=','` tells me that entries are separated by commas.
- `head=TRUE` says the first line are column names.
- Many other options (see help files).
- MUST be rectangular – same number of entries on each line.

Reading Data and Data Frames

- Result of command is an object `ships` that looks like a matrix.
- But, we observe some differences
 - 1 Headings on each of the columns.
 - 2 Some columns have character entries and some are numerics.
- If we look at

```
> class(ships)
[1] "data.frame"
```

this structure allows some columns to be some data types and other columns to be others.

More on Data Frames

```
> names(ships)
[1] "Type" "Construction" "Operation" "Months" "Damage"
```

You can also refer to columns in data frames by their name

```
ships$Months
ships[, 'Months']
```

both retrieve the fourth column.

```
names(ships)[2] = 'construction'
```

retrieves and changes the column names (this also works with vectors and matrices).

```
SmallShips = data.frame(ships$Type, ships$Months, ships$Dam)
```

creates a new data frame. (R will also complete names if it can work out how).

You can put any matrices or vectors into a data frame so long as the number of rows match up.

Other Input/Output Functionality

- `scan` for when your files are not as nicely structured; but you need to do a lot of manipulation in `R` afterwards.
- `load` if you have objects saved in a `.Rdata` file.
- `save` to save objects in a `.Rdata` file.
- `write.table` to write a data frame or matrix to an ASCII file.

```
write.table(SmallShips, file='smallships.csv',  
            sep=" ", row.names = FALSE)
```

- `source` will read a text file as though you had copied and pasted the contents into the `R` prompt. Especially useful if you are loading lots of functions.

See help files for details.

Factors

```
> ships[1:5,1]
[1] A A A A A
Levels: A B C D E
```

R has decided that `Type` really lists five categorical values; this will be useful in linear regression.

You can change these to be character values by

```
> as.character(ships$Type)[1:5]
[1] "A" "A" "A" "A" "A"
```

Can also decide to make something categorical by calling it a *factor*:

```
> ships$Operation = as.factor(ships$Operation)
> ships$Operation[1:5]
[1] 60 75 60 75 60
Levels: 60 75
```

Lists

- Lists collect any set of R objects.
- Often output of functions, can also be useful as input.
- Indexed by name and by position

Linear regression to predict number of damage incidents from covariates:

```
> model = lm(log(Damage+0.5) ~ ., data=ships)
```

```
> names(model)
[1] "coefficients" "residuals"    "effects"      "rank"
[5] "fitted.values" "assign"        "qr"           "df.residual"
[9] "contrasts"    "xlevels"      "call"         "terms"
[13] "model"
```

Indexing Lists

R thinks of a list as a *vector* of objects; index accordingly

```
> small.model = model[c(1,2,5,8)]  
> names(small.model)  
[1] "coefficients" "residuals" "fitted.values" "df.residual"
```

Note that `model[8]` is a list with one element

```
> model[8]  
$df.residual  
[1] 32
```

If you want to access an *element* of a list, use '`[[ ]]`'

```
> model[[8]]  
[1] 32
```

or

```
> model$df  
[1] 32
```

Lists

Looking at the objects here (and reading the [lm](#) help files)

```
> model$coef
      (Intercept)           TypeB           TypeC           TypeD           TypeE
-3.4976448864    0.2280579299   -0.5661857786   -0.7458956086   -0.1065095790
      Construction   Operation75           Months
0.0581366336    0.8678229798    0.0001080624
```

Note that R has used dummy coding to expand `Type` into indicators of each category.

We could also plot residuals

```
plot(model$fit,model$resid)
```

Creating Lists

Command `list` allows you to set out a list of objects

```
> ShipsAndMod = list(ships,model$coef,model$fit,model$res)  
> names(ShipsAndMod)  
NULL
```

Add names

```
> names(ShipsAndMod) = c('ships','coefs','fit','resid')  
> names(ShipsAndMod)  
[1] "ships" "coefs" "fit"   "resid"
```

or specify them in the `list` function

```
> ShipsAndMod2 = list(ships=ships,coef=model$coef,  
+ fit=model$fit,resid=model$resid)  
> names(ShipsAndMod2)  
[1] "ships" "coef"  "fit"   "resid"
```

You can always index by position.

Functions

- Functions allow you to short-cut copying the same code to multiple places.
- Mean that errors only need to be fixed in one place.
- If used well, make code much more readable.

Notes

- Functions are objects in the [R](#) workspace.
- Must have defined inputs.
- Returns a single output; but that can be a list.

Example

A function to compute the row-wise minimum of two vectors:

```
row.min = function(x,y){  
  # Number of entries + result storage  
  m = length(x)  
  result = rep(0,m)  
  
  # loop over entries  
  for(i in 1:m){  
    result[i] = min(x[i],y[i])  
  }  
  return(result)  
}  
  
> x = c(1,5.2,3.4,2.1)  
> y = 1:4  
> row.min(x,y)  
[1] 1.0 2.0 3.0 2.1
```


Anatomy of a Function

- `function(x,y)` - list the *arguments* (inputs) into the function.
- You can add defaults, eg if y is left replace it with zeros
`row.min = function(x,y=rep(0,length(x))){...}`
- Set of commands to be executed are given inside '{ }'
- `return(..)` statement is the function output.
- Without a return statement, function returns result of last command.
- If you want to return multiple objects, put them in a list.

An Expanded Example

Record which object the minimum came from, too

```
row.min = function(x,y=rep(0,length(x))){  
  m = length(x)  
  row.min = y          # Answer defaults to y  
  min.obj = rep(2,m)    # and is changed only if  
                        # x[i] <= y[i]  
  
  # loop over entries  
  for(i in 1:m){  
    if( x[i] <= y[i] ){  # check -- do we need to  
      row.min[i] = x[i]  # do anything?  
      min.obj[i] = 1  
    }  
  }  
  return( list(row.min=row.min, min.obj=min.obj) )  
}
```

Running The Example

```
> a = x
> b = y
> c = row.min(a,b)
> c
$row.min
[1] 1.0 2.0 3.0 2.1

$min.obj
[1] 1 2 2 1
```

Environments (and a Caution)

- When a function is called, the operations take place in a new *environment*.
- Essentially: a workspace with only the function inputs.
- When the function returns, only the value of the output is returned.
- No objects in the main workspace are changed.
- E.g. an object `min.obj` in your workspace will not be changed.

The Caution

If you try to use an object not in your workspace, R complains

```
> x+d
```

```
Error: object 'd' not found
```

If a function does not find an object in its environment, it will look in the parent environment for it.

```
add.d = function(x) {  
  return(x+d)  
}
```

```
> d = 5
```

```
> add.d(y)
```

```
[1] 6 7 8
```

```
> rm(d)
```

```
> add.d(y)
```

```
Error in add.d(y) : object 'd' not found
```

Note that the value of `d` will not be changed in your workspace even if it is changed in the function.

3 Shortcuts and Coding Practice

- 1 `attach` to add columns of a data frame to the workspace
- 2 Functions will return result of last calculation (rather than using `return`)
- 3 Functions will look in the workspace for objects not part of input.

All of these can be convenient, but are poor coding practice

- They make code harder to read for someone else (and you in two months time).
- It's much harder to work out why code might not be working.
- Unless you record *every* command, code may not do the same thing next time you run it.

Guidelines for Good Coding Practice

Well-written code

- 1 Efficiently does what it was intended to do.
- 2 Is readily understandable by others.

The second is often as important as the first!

Keys to good coding

- Choose meaningful variable names.
- Indent within loops, other statements.
- Group commands into meaningful batches (eg processing inputs, assigning arrays for outputs)
- Comment what your code is doing, anyway.

A Comparison

```
func1 = function(x,y) {  
  m = length(x)  
  z = y  
  I = rep(2,m)  
  for(i in 1:m) {  
    if( x[i] <= y[i] ){  
      z[i] = x[i]  
      I[i]=1  
    }  
  }  
  list(z,I)  
}
```

- Same function as `row.min`.
- No indenting = harder to read.
- `z` and `I` not meaningful.
- No comment to explain default structure.
- Output not named in list, function name doesn't say what it does.
- Longer, more complex functions can be impossible to interpret.

Argument Checking

Code should also be robust to idiots who use it.

- `row.min` only works if `x` and `y` have the same number of entries.

- What happens when they don't?

```
> result = row.min(x,y[1:3])
Error in if (x[i] <= y[i]) { : missing value where TRUE/FALSE needed
```

Not particularly informative.

Instead, let's define a checking function:

```
CheckLength = function(x,y) {
  m = length(x)
  if( length(y) != m ){
    stop('x and y must have the same length.') }
  return(m)
}
```

A New row.min

You can use one function inside another:

```
row.min = function(x,y=rep(0,length(x))) {
  m = CheckLength(x,y)
  row.min = y          # Answer defaults to y
  min.obj = rep(2,m)    # and is changed only if
                        # x[i] <= y[i]

  # loop over entries
  for(i in 1:m){
    if( x[i] <= y[i] ){  # check -- do we need to
      row.min[i] = x[i]  # do anything?
      min.obj[i] = 1
    }
  }
  return( list(row.min=row.min, min.obj=min.obj) )
}
```

Packages

- Besides the basic functionality in `R`, users can contribute packages, making their own functions available to the public.
- These must be well documented (ie, have good helpfiles) and often have examples and data built in.
- You can access a package with `library` or `require`.
- `http://www.r-project.org/search.html` lists some tools to search for functionality.
- Packages must be installed, use `install.packages` or the Packages menu.
- You can “unload” a library by `detach('package:...')`.

We will make limited use of libraries in this course: the point is to code things for yourself!

Some Comments About R

- Very flexible; usually more than one way to get things done.
- Lots of functionality (available lots of ways); reasonable documentation is usually available.
- You should be able to use help files – we will not explain (formally) every function we use.
- Using `class` some patience and trial and errors will get you through a lot of things.

```
> class(A)  
[1] "matrix"
```

```
> class(A[1,])  
[1] "integer"
```

Summary

- 1 Reading in, writing and saving data utilities.
- 2 `data.frames` allow you to have data of multiple types.
- 3 lists allow you to bundle different sorts of objects together
- 4 functions provide a quick way to use the same code in many places.
- 5 It's very important to work through the functions you create carefully.
- 6 R has multiple ways of doing everything, don't be afraid of help files and trial-and-error.

Random Number Functions Simulation

JMR Chapters 15, 16, 17, and 18

Fangfang Wang
MA 551

Random Number Functions

`R` has a number of built-in functions associated with specific distributions.

These provide the density, distribution, and quantile functions of each distribution as well as generating random numbers from them.

Example: the normal family

`dnorm` Normal density.

`pnorm` Cumulative normal density.

`qnorm` Normal quantiles ($q(p) = \{u : P(x \leq u) = p\}$)

`rnorm` Generates normal random numbers.

Standard pattern

`d...`: density, `p...`: distribution, `q...`: quantiles `r...`: random numbers.

Examples

```
x = seq(-3,3,0.1)  # sequence of numbers

par(mfrow=c(2,2))  # Plot window with 2-by-2 panes
                    # filled by row.

plot(x,dnorm(x),type='l')
plot(x,pnorm(x),type='l')

p = seq(0,1,0.01)  # probabilities between 0 and 1
plot(p,qnorm(p),type='l')

z = rnorm(100,mean=3,sd=5)
plot(z)
```


R Built-in Distributions

<code>beta</code>	beta	<code>logis</code>	logistic
<code>binom</code>	binomial	<code>lnorm</code>	log normal
<code>cauchy</code>	Cauchy	<code>nbinom</code>	negative binomial
<code>chisq</code>	Chi-squared	<code>norm</code>	normal
<code>exp</code>	exponential	<code>pois</code>	Poisson
<code>f</code>	Fisher's f	<code>t</code>	Student's t
<code>gamma</code>	gamma	<code>unif</code>	uniform
<code>geom</code>	geometric	<code>Weibull</code>	Weibull
<code>hyper</code>	hypergeometric		

Numerous others available in packages.

Later in the class, we will discuss how to generate random numbers from scratch.

The sample Function

Sometimes you want to randomly select elements from a set:

```
x = sqrt(1:10)
y = sample(x,5)
# 5 entries from x drawn at random
```

If sample is given `n` instead of `x`, it assumes `1:n`

```
sample(10,5)
```

You can also sample *with replacement*. This means each time you choose from the original collection and can get repeated elements:

```
> sample(x,5,replace=TRUE)
[1] 2.828427 2.645751 2.449490 2.000000 2.828427
```

The sample Function

Randomly select 5 employees from a business with 20 employees.

Get External Data		
L31		
	A	B
1	Label	Name
2		1 Alison
3		2 Amy
4		3 Brigitte
5		4 Darwin
6		5 Emily
7		6 Fernando
8		7 George
9		8 Harry
10		9 Henry
11		10 John
12		11 Kate
13		12 Max
14		13 Moe
15		14 Nancy
16		15 Ned
17		16 Paul
18		17 Ramon
19		18 Rupert
20		19 Tom
21		20 Victoria

Simulation

Why do statisticians need to generate random numbers at all?

- To assess convergence to asymptotic results (eg, central limit theorem).
- To investigate the sample properties of complex statistical procedures.
- To verify mathematical results (e.g. the square of a normal is a χ_1^2)
- Within some statistical procedures (later in the course).

Evaluating the Central Limit Theorem

- CLT: if $X_1, X_2, \dots$ are i.i.d. with mean μ and variance σ^2 , then

$$\sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n X_i - \mu \right) \xrightarrow{d} N(0, \sigma^2)$$

as $n \rightarrow \infty$.

- But how close is this for a given, finite n ?
- For example: suppose we had three negative binomial random variables with $r = 1$ and $p = 0.07$?

Negative Binomial (or Pascal) Distribution

- We say $Z \sim NB(r, p)$ if the probability mass function of Z is

$$P(Z = k) = \binom{k + r - 1}{k} p^r (1 - p)^k \quad (1)$$

for $k = 0, 1, 2, \dots$

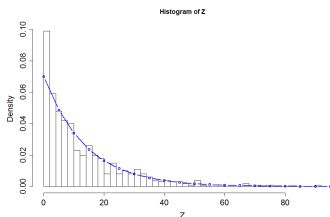
- Z denotes the number of failures till the r th Success.
- p is the probability of Success.
- The total number of Bernoulli trials is $Z + r$.
- $E(Z) = \frac{(1-p)r}{p}$ and $Var(Z) = \frac{(1-p)r}{p^2}$
- $r = 1$: Geometric Distribution

```

Z = rnbinom(500,1,0.07)
hist(Z,50,freq=FALSE)

x = seq(0,100,5)
lines(x,dnbinom(x,1,0.07),
type='b',col='blue')

```



A Simulation Experiment

```
nsim = 500                                # Define simulation parameters
n = 3
p = 0.07
mu = (1-p)/p
sig = sqrt( (1-p)/p^2 )

res = rep(0,nsim)

for(i in 1:nsim){                          # Run Simulations
  Z = rbinom(n,1,p)
  res[i] = sqrt(n)/sig*( mean(Z) - mu )
}

hist(res,50,freq=FALSE)                   # Plot Results
x = seq(-3,3,0.1)
lines(x,dnorm(x))
```

Some Comments On Good Simulation Code

- Pre-define as many simulation constants into R objects as possible.
 - This way, to change the simulation (eg, if we wanted to change p), we only need to do so in one place.
- Pre-compute numbers that will be used many times (`mu` and `sig` above).
- Store as much as possible; you never know when you will need it.
- Ideally, you can re-create *exactly* the same results from your simulation if you need to.

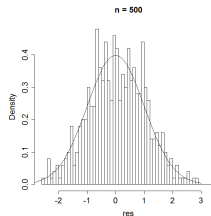
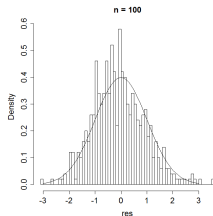
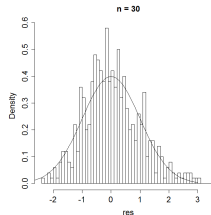
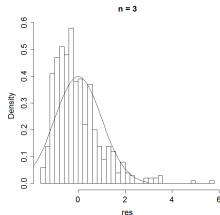
Some More Comments On Good Code

Further general guidelines:

- Organize your code in coherent .R files (functions related to a particular task, simulations for a particular set of results).
- Provide an overall description at the start of the file - what is this code doing?
- Date the last time you modified the code – even better, keep a running log of changes you have made.
- Arrange your code into coherent blocks; reading/generating data, running the simulation, collating results and make sure these are clearly separated and labeled.
- When you save simulation results (including plots from them), include as much information in the file name as possible; keep a file documenting exactly what each of these results means and contains.
- Comment everything!

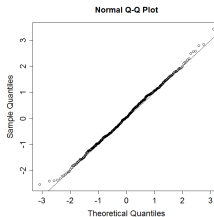
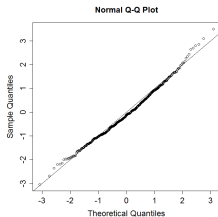
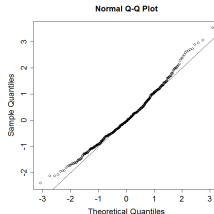
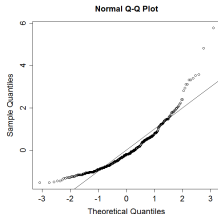
Larger n

Because I pre-defined $n=3$, it's easy to change it to 30, 100, 500.



Evaluating Densities

It is difficult to judge how good a histogram is; QQ-plots do better. From `qqnorm(res)` and `qqline(res)`:



Simulation Size Guidelines

Many simulations aim to estimate an *average* (i.e., an expectation).

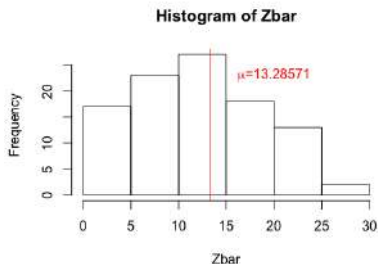
How accurate is my estimate of the expectation?

```
nsim = 100
n = 3
p = 0.07
mu = (1-p)/p
Zbar = rep(0,nsim)

for(i in 1:nsim){
  Z = rbinom(n,1,p)
  Zbar[i] = mean(Z)
}

hist(Zbar)
abline(v=mu, col='red')
text(20,23,substitute(paste(mu, "=", m), list(m=mu)),
      col='red')
```

Simulation Size Guidelines



```
> (mu = (1-p)/p)
[1] 13.28571
> (sig = sqrt( (1-p)/p^2 ))
[1] 13.77664
> sig/sqrt(n)
[1] 7.953949
> mean(Zbar)
[1] 12.27333
> sqrt(var(Zbar)/nsim)
[1] 0.6742894
```

With `nsim = 1000`

```
> mean(Zbar)
[1] 13.03567
> sqrt(var(Zbar)/nsim)
[1] 0.2414213
```

With `nsim = 10000`

```
> mean(Zbar)
[1] 13.32737
> sqrt(var(Zbar)/nsim)
[1] 0.07985549
```

How Many Simulations?

- Usually want to estimate the number to within a tolerance
- Remember that $\text{Var}(\bar{X}) = \text{Var}(X)/N$.
- Suppose from initial simulation of 100, `sd(Zbar)` about 6.7.
- So we need $6.7^2 \approx 45$ simulations so that $\text{Var}(\text{mean}(\text{Zbar}))$ is about 1.
- Calculations apply very generally:
 - Estimate a probability: expectation of an indicator; e.g.
`mean(Zbar > 10)`
 - Variance: `mean( (Zbar-mean(Zbar)) ^2 )` or `var(Zbar)`
- For probabilities, remember if $X_1, \dots, X_N$ are binary,
 $\text{Var}(\bar{X}) = \frac{p(1-p)}{N} \leq \frac{1}{4N}$.
- If you want p to within δ , you need $N \geq 1/(4\delta^2)$.
($\delta = 0.01$: $N \geq 2,500$, $\delta = 0.005$: $N \geq 10,000$)

Example: Testing Tests

Suppose we wanted to test $H_0 : p = 0.07$ from 30 samples?

- 30 negative binomial $NB(1, 0.07)$ samples $X_1, \dots, X_{30}$.
- Known mean is $0.93/0.07 = 13.286$.
- Conduct a t-test for mean of the X_i :

$$t = \frac{\sqrt{n}|\bar{X} - 13.286|}{\sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2}}$$

and reject H_0 at the 5% level if $t > 2.045$ (obtained from `qt(0.975, 29)`).

- *Should* have probability of rejecting ≈ 0.05 when $p = 0.07$.
- But does this test have reasonable α -level? Test with a simulation.

Testing α -levels

```
nsim = 10000    # We need accuracy to within 0.01 to
                 # check alpha = 0.05.

n = 30
res = rep(0,nsim)
t = rep(0,nsim)
mu = (1-p)/p

for(i in 1:nsim){
  X = rbinom(n,1,p)
  t[i] = sqrt(n)*abs( mean(X) - mu )/sd(X)
  res[i] = t[i] > qt(0.975,29)
}

mean(res)
```


Simulating Marginal and Conditional Distributions

Use the notation $P(A|B)$ for the probability of A *given* B .

- Multiplicative Law: $P(A \cap B) = P(A|B)P(B)$
- Law of Total Probability: For disjoint sets $B_1, \dots, B_n$ such that $P(\cup_{i=1}^n B_i) = 1$

$$P(A) = \sum_{i=1}^n P(A \cap B_i) = \sum_{i=1}^n P(A|B_i)P(B_i)$$

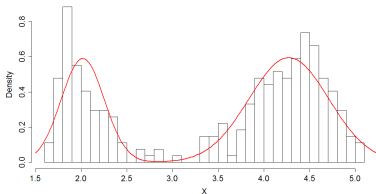
- In Bayes theorem:

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

We'll explore the computational equivalents of these here.

An Example

Data on the duration of the eruption at the 'old faithful' geyser in Yellowstone National Park:



We can represent this as being (approximately) two normal peaks

$$f(x) = p \underbrace{\frac{1}{\sqrt{2\pi}\sigma_1} e^{-(x-\mu_1)/2\sigma_1^2}}_{f_1(x)} + (1-p) \underbrace{\frac{1}{\sqrt{2\pi}\sigma_2} e^{-(x-\mu_2)/2\sigma_2^2}}_{f_2(x)} \quad (2)$$

(note that because each normal distribution integrates to 1, their weighted sum does)

but it would be nice to simulate this data.

Simulating Mixture Models

We represented the two-peaked distribution above as two re-scaled normal distributions.

But we can construct it by posing a hypothetical binary random variable Z to tell us which normal component an observation comes from

- 1 Simulate Z as Bernoulli with probability p
- 2 If $Z = 1$ simulate X from $N(\mu_1, \sigma_1^2)$, otherwise simulate X from $N(\mu_2, \sigma_2^2)$

```
Z = rbinom(1,1,p)
if(Z){ X = rnorm(1,mean=mu1,sd=sig1) }
else{ X = rnorm(1,mean=mu2,sd=sig2) }
```

Simulation and Probability

To translate simulation scheme into probability

$$Z \sim \text{Bin}(1, p), \quad X|Z = 1 \sim N(\mu_1, \sigma_1^2) \quad X|Z = 0 \sim N(\mu_2, \sigma_2^2)$$

so we have defined X *conditional* on Z .

But when we look at X by itself (i.e., throw away Z) we get the *marginal* distribution

$$P(X \leq x) = P(X \leq x|Z = 1)p + P(X \leq x|Z = 0)(1 - p)$$

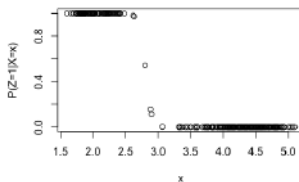
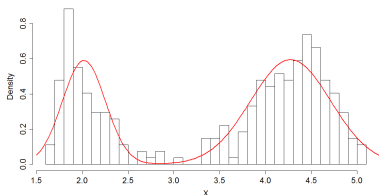
yielding the density above.

- Marginal distribution is what you get when you drop the information in Z .
- Useful way of generating random variables.

Simulation and Bayes Theorem

We might also like to know which component to assign a given observation to; that is, we're looking for

$$P(Z = 1|X = x) = \frac{P(X = x|Z = 1)P(Z = 1)}{P(X = x)}.$$



Simulation and Bayes Theorem

```
data(faithful)
head(faithful)
mixdat = faithful$eruptions
p = 0.3484043
mu1 = 2.018607; mu2 = 4.273343
sig1 = 0.2356206; sig2 = 0.4370642

# Evaluate normal densites at observed data points
dens1 = dnorm(mixdat,mu1,sd=sig1)
dens2 = dnorm(mixdat,mu2,sd=sig2)

# Use Bayes theorem to evaluate "posterior"
# probability that Z=1
PrZeq1 = p*dens1/(p*dens1+(1-p)*dens2)
plot(mixdat,PrZeq1, ylab="P(Z=1|X=x)", xlab="x")
```

Summary

- Built-in functions to calculate densities, distribution functions, quantile functions and generate random variables.
- Simulation methods to check on statistical properties of procedures.
- Simulation sizes should be large enough to accurately estimate quantities of interest.
- Can check on this with standard statistical tools – you can always add more simulations!

Simulation: Probability Made Concrete

Simulation-Based Critical Values

Permutation Tests

Rizzo, Chapter 8

Fangfang Wang
MA 551

Statistical Tests Made Concrete

From Notes # 3:

- α -level: if the null hypothesis were true, we would (mistakenly) reject α -proportion of the time.
- To check this,
 - simulate data generated from the null distribution.
 - run hypothesis test.
 - repeat many times; proportion rejected should be α .

But we can also use this to define hypothesis tests:

- Most tests reject H_0 if test statistic $>$ critical value
- But we need to choose the critical value.
 - Also by simulation!

Critical Values for Tests

Suppose we want to test a hypothesis H_0 using data $X_1, \dots, X_n$:

- Choose a statistic $t(X_1, \dots, X_n)$ that should be small when H_0 is true and large when H_0 is false.
- Reject H_0 if $t(X_1, \dots, X_n) > t_{1-\alpha}$.
 - Define $t_{1-\alpha}$ so that $P(t(X_1, \dots, X_n) > t_{1-\alpha} | H_0) = \alpha$.
- But how do we actually find $t_{1-\alpha}$ if we don't trust current theory?
 - 1 Simulate $X_1, \dots, X_n$ under H_0 .
 - 2 Evaluate $T = t(X_1, \dots, X_n)$.
 - 3 Repeat to get $T_1, \dots, T_N$.
 - 4 $t_{1-\alpha}$ given by the $(1 - \alpha)$ -quantile of $T_1, \dots, T_N$.
- Note: problematic if H_0 does not *completely* specify distribution of $X_1, \dots, X_n$.

A Negative Binomial Simulation

Back to testing the mean of a negative binomial (see Notes #3)

```
start = proc.time()
nsim = 25000
n = 30
p = 0.07; mu = (1-p)/p
t.vals = rep(0,nsim)

for(i in 1:nsim){           # Data and t-statistic
  X = rnbinom(n,1,p)
  t.vals[i] = sqrt(n)*abs( mean(X) - mu )/sd(X)
}
proc.time()-start

> t.crit = quantile(t.vals,0.95) # Simulation critical
2.288837                        # value
> qt(0.975,29)                 # t-dist critical value
[1] 2.04523
```

Vectorizing

Let's see how to vectorize this

Compiled pointwise operations:

- `+`, `-`, `*`, `/`, `^`
- `sqrt`, `log`, `exp`,

Vectorised operations:

- `mean`, `sum`, `var`, `sd`, `cumsum`, `diff`, ...

Matrix-vector operations:

- `t`, `%*%`, `diag`, `solve`, ...

Linear algebra often helps: taking column means of a matrix

```
X = matrix(rnorm(1000*5000),1000,5000)
ms = rep(0,ncol(X))
for(i in 1:ncol(X)){ ms[i] = mean(X[,i]) }
```

Alternatively, the column means of `X` can be gotten with a matrix multiplication:

```
ms2 = rep(1/nrow(X),nrow(X))%*%X           # Why?
```

Mathematically, this is $\left(\frac{1}{n}\mathbf{1}\right)^T X$.

But remember: clarity vs efficiency trade-off!

Vectorizing

Back to testing the mean of a negative binomial. Let's see how to vectorize this

```
start = proc.time()

# Generate nsim data sets over rows.
XX = matrix(rnbinom(n*nsim,1,p),nsim,n)
# Take the mean
mean.X = XX%*%rep(1/n,n)
# Subtract mean
center.X = XX - matrix(mean.X,nsim,n,byrow=FALSE)
# Average squared deviation then square-root
sd.X = sqrt( (center.X^2)%*%rep(1/(n-1),n) )
# Calculate Statistic
t.vals = sqrt(n)*abs(mean.X - mu)/sd.X

proc.time()-start
```

Testing Two Populations

- Two samples $\{X_1, \dots, X_{n_1}\}$, $\{Y_1, \dots, Y_{n_2}\}$ from distributions F_X and F_Y respectively.
- $H_0 : F_X = F_Y$, but F_X not specified.

Options:

- Two-sample t -test:

$$\frac{|\bar{X} - \bar{Y}|}{\sqrt{s_P^2(1/n_1 + 1/n_2)}} \text{ where } s_P^2 = \frac{(n_1 - 1)s_X^2 + (n_2 - 1)s_Y^2}{n_1 + n_2 - 2}.$$

- Welch's t -test or Welch-Satterthwaite t -test.

But:

- t -test critical value if you don't trust asymptotics?
- What if F_X not specified? H_0 is pretty vague?

Constructing a Null Distribution

Implement a two-sample t -test as a permutation test:

- Add a column (row below to save space) of labels

$$\begin{bmatrix} X_1 & \cdots & X_{n_1} & Y_1 & \cdots & Y_{n_2} \\ 1 & \cdots & 1 & 2 & \cdots & 2 \end{bmatrix}$$

- If the X 's and Y 's are from the same distribution, their *labels* shouldn't matter.
- So, if we randomly mix up their labels, things shouldn't change very much.
- Implementation:
 - 1 Randomly permute the labels.
 - 2 Treat permuting the labels like generating **new** X 's and Y 's.
 - 3 Evaluate t -statistic on the permuted labels; this is the *permutation distribution*.

Constructing a Null Distribution

- There are $\binom{n_1+n_2}{n_1}$ different ways to partition pooled sample into 2 subsets of sizes n_1 and n_2 .
- The collection of test statistics computed from all the permutations give permutation distribution.
- Unless sample size is very small, evaluate test statistics over all samples is computationally excessive.
- An approximate permutation test is implemented by randomly drawing a large number of samples without replacement.

An Example Data Set

Example `chickwts` data in R gives weight of chickens fed different diets. See page 217 of Rizzo.

We will focus on differences between linseed and soybean.

```
data(chickwts)
```

```
Z = chickwts[chickwts$feed=='linseed' |  
              chickwts$feed=='soybean',]
```

```
x = Z[Z$feed=='linseed',1]
```

```
y = Z[Z$feed=='soybean',1]
```

```
> t.test(x,y)
```

```
data:  x and y
```

```
t = -1.3246, df = 23.63, p-value = 0.198
```

But we'd like to verify that p-value.

A Test Statistic

We'll define a function to take `Z` and give us the t-statistic back.

```
chick.t.test = function(Z) {  
  x = Z[Z$feed=='linseed',1]  # Split into linseed  
  y = Z[Z$feed=='soybean',1]  # and soybean  
  return( abs(t.test(x,y)$statistic) )  
}
```

Defining this function is overkill (but saves space next slide).

We could also use output of `lm(weights~feed, data=Z)`.

Actually, not quite equivalent. [Why not?]

First we'll record the observed statistic

```
t.obs = chick.t.test(Z)
```

Constructing a Null Distribution

The `sample(N)` function will randomly re-arrange `1:N`.

```
> sample(5)
[1] 5 4 2 1 3
```

Now record the t statistic under random permutations of `feed`.

```
nperm = 1000           # Number of permutations
t.perm = rep(0,nperm)
temp.Z = Z              # Store a version of Z that we
                        # can change around.

for(i in 1:nperm){
  I = sample(nrow(Z)) # Generate a random permutation.
  temp.Z[,2] = Z[I,2]
  t.perm[i] = chick.t.test(temp.Z)
}
```

Assessing Significance

Now we can ask *Is the observed statistic too large according to the permutation distribution?*

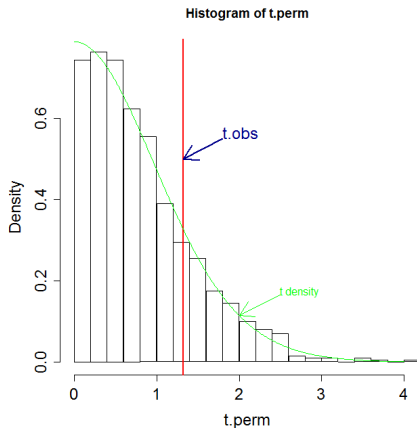
```
> mean(c(t.perm, t.obs)
      >= t.obs)
[1] 0.194
```

We can also look at the critical value

```
> quantile(t.perm, 0.95)
2.087385
```

Compare to t -value

```
> qt(0.975, 23.63)
[1] 2.06561
```



Some Philosophical Distinctions

Permutation distribution has a different data-generating model.

- Null hypothesis: X 's and Y 's generated according to the same distribution.
- Permutation test: X 's and Y 's fixed, but labels are assigned at random.

So why is the permutation test OK?

- Under H_0 all permutations of X 's and Y 's are equally likely.
- If the labels are assigned at random, the probability of rejecting is 0.05.
- This argument has nothing to do with the actual X and Y values.

More Generally

Ideas extend to test associations between quantities:

- Correlations between two continuous random variables.
- Regression of a response onto multiple covariates.
- Associations between *groups* of covariates.

Same reasoning as before:

- If they're independent, permuting one makes no difference (all permutations are equally likely).
- If X and Y (possibly multivariate) are related, permuting one (either!) breaks the relationship.

Another Example

Look at all feeds in `chickwts`; do they affect outcome weight?
We'll use the F statistic for the regression.

```
mod = lm(weight~feed,data=chickwts)
(fstat.obs = summary(mod)$fstatistic[1])

nperm = 1000
fstat.perm = rep(0,nperm)
temp.data = chickwts
for(i in 1:nperm){
  I = sample(nrow(chickwts)) # Generate a random permutation.
  temp.data$feed = chickwts$feed[I]
  fstat.perm[i] =
    summary(lm(weight~feed,data=temp.data))$fstatistic[1]
}

mean(c(fstat.perm, fstat.obs) >= fstat.obs)
```

Limitations

- Restricted to breaking relationships.
- No option to partially break relationships.
- Eg: $y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \epsilon_i$
 - Can test $\beta_1 = \beta_2 = 0$ by permuting the y_i .
 - Suppose we wanted to test just $\beta_1 = 0$?

Could permute just the x_{i1} ; but this also changes relationship between x_{i1} and $x_{i2} \Rightarrow$ changes variance of your estimated coefficients.
 - Some variations to let you do this later.
- Not always the most powerful test available.
- **But:** pretty generic when applicable.

More General Statistics

Standard test statistics are not the only measures that can be permuted.

- Kolmogorov-Smirnov test for two samples: maximal distance between empirical cdfs.
- Comparing two multivariate samples; Hotelling's T^2 , but also other measures (Rizzo's own distance covariance).
- Could compare variances, if you think that this is the most obvious difference in distributions.
- Relationships between collections of continuous covariates (eg 10 ecological covariates and 4 human land-use): largest correlation, major canonical covariate.
- Little theory to guide best statistic; choice is based on what will pick up the signal you expect to find.
- Project?

Two-sample Kolmogorov-Smirnov(K-S) test

Example 8.2 of Rizzo: Consider `chickwts` data in `R`. Test for any type of difference in the soybean and linseed groups.

$$H_0 : F = G \quad H_a : F \neq G$$

where F = distribution of weight of chicks fed soybean supplements, and G = distribution of weight of chicks fed linseed supplements.

The K-S statistic D is the maximum absolute difference between the ecdf's of the two samples:

$$D = \sup_{1 \leq i \leq N} |F_{n_1}(z_i) - G_{n_2}(z_i)| \quad (1)$$

where F_{n_1} = ecdf of $\{X_1, \dots, X_{n_1}\}$, G_{n_2} = ecdf of $\{Y_1, \dots, Y_{n_2}\}$

- $0 \leq D \leq 1$

- large values of D support the alternative

Two-sample Kolmogorov-Smirnov(K-S) test

Using `ks.test`, the observed value of $D = D(X, Y) = 0.2976190$.

```
options(warning=-1)    # supress warnings
D.obs = ks.test(x,y,exact=F)$statistic
        # 0.297619
```

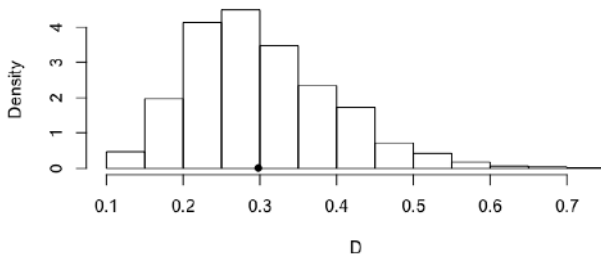
Shall we reject the null hypothesis? To determine whether this value is strong evidence for the alternative, we compute D with the *replicates* $D^* = D(X^*, Y^*)$.

```
Z = c(x, y)           # pooled sample
nperm = 1000
D.perm = numeric(nperm)
for(i in 1:nperm){
  k = sample(length(Z), size=14, replace=F)
  x1 = Z[k]
  y1 = Z[-k]
  D.perm[i] = ks.test(x1,y1, exact=F)$statistic
}
options(warn = 0)
```

Assessing Significance

Compute p-value

```
> mean(c(D.obs, D.perm) >= D.obs)
[1] 0.4615385
hist(D.perm, main="", freq=F)
points(D.obs, 0, pch=16)
```



Summary

- Probability: often very helpful to think about theory via what simulation looks like.
- For conducting tests; when in doubt, simulate!
- In [R](#), clever vectorization can buy you a lot of speed (when you need it).
- Permutation tests: randomly re-order some columns to break-up relationships in the data.
- I.e., make H_0 true; then use observed data to conduct your test.

Permutation Tests of Independence

Rizzo, Chapter 8

Fangfang Wang
MA 551

Test of Independence

A test of independence of random vectors $X \in \mathbb{R}^p$ and $Y \in \mathbb{R}^q$

$$H_0 : F_{XY} = F_X F_Y, \quad H_a : F_{XY} \neq F_X F_Y \quad (1)$$

where F_X = CDF of X , F_Y = CDF of Y , F_{XY} = CDF of (X, Y) .

- Let $\hat{\theta}$ be the test statistic.
 - Large values of $\hat{\theta}$ support H_a .
- The test will be implemented as a permutation test.
- Suppose that a random sample is in an $n \times (p + q)$ data matrix $\mathbf{Z}$ with observations in rows:

$$\mathbf{Z}_{n \times (p+q)} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,p} & y_{1,1} & y_{1,2} & \cdots & y_{1,q} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,p} & y_{2,1} & y_{2,2} & \cdots & y_{2,q} \\ \vdots & \vdots & & & & & & \vdots \\ x_{n,1} & x_{n,2} & \cdots & x_{n,p} & y_{n,1} & y_{n,2} & \cdots & y_{n,q} \end{bmatrix}$$

An Example: Iris data

- Edgar Anderson's Iris data set gives the measurements in centimeters of the four variables (i.e., petal length and width, and sepal length and width) for 50 flowers from each of 3 species of iris.
- The species are Iris setosa, versicolor, and virginica.
- Test whether the *bivariate distributions* (petal length, petal width) and (sepal length, sepal width) of iris setosa are independent.

An Example: Iris data

```
> head(iris)
Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1           5.1         3.5         1.4         0.2  setosa
2           4.9         3.0         1.4         0.2  setosa
3           4.7         3.2         1.3         0.2  setosa
4           4.6         3.1         1.5         0.2  setosa
5           5.0         3.6         1.4         0.2  setosa
> z <- as.matrix(iris[1:50, 1:4]) # setosa
> x <- z[ , 1:2]
> y <- z[ , 3:4]
```

Permutation Tests of Independence

- Let v_1 be the row labels of the X sample
- Let v_2 be the row labels of the Y sample.
- If X and Y are dependent, the samples must be paired and the ordering of labels v_2 cannot be changed independently of v_1 .
- Under independence, the samples X and Y need not be matched.
- Any permutation of the row labels of the X or Y sample generates a permutation replicate.
- The permutation test procedure for independence permutes the row indices of one of the samples (it is not necessary to permute both v_1 and v_2).

Approximate Permutation Test Procedure for Independence

- 1 Compute the observed test statistic $\hat{\theta}(X, Y) = \hat{\theta}(\mathbf{Z}, v_1, v_2)$.
- 2 For each replicate, indexed $b = 1, \dots, B$:
 - Generate a random permutation $\pi_b = \pi(v_2)$.
 - Y^* : permutation replicate of the Y sample
 - Compute $\hat{\theta}^{(b)} = \hat{\theta}^*(\mathbf{Z}, \pi_b) = \hat{\theta}(X, Y^*)$.
- 3 Compute the empirical p-value by

$$\hat{p} = \frac{1 + \#\{\hat{\theta}^{(b)} \geq \hat{\theta}\}}{1 + B}. \quad (2)$$

- 4 Reject H_0 at significance level α if $\hat{p} \leq \alpha$.

dCov Test of Independence

For $X \in \mathbb{R}^p$ and $Y \in \mathbb{R}^q$,

$$H_0 : F_{XY} = F_X F_Y, \quad H_a : F_{XY} \neq F_X F_Y. \quad (3)$$

- Multivariate nonparametric test of independence based on **distance covariance** (dCov).
 - G. J. Székely, M. L. Rizzo, and N. K. Bakirov (SRB).
Measuring and testing dependence by correlation of distances. AoS, 2007, 35(6).
- Let ϕ_X, ϕ_Y be characteristic functions of X and Y , and $\phi_{X,Y}$ the joint characteristic function of X and Y .
- Under H_0 : $\phi_{X,Y}(t, s) = \phi_X(t)\phi_Y(s)$
- Measure the distance between $\phi_{X,Y}(t, s)$ and $\phi_X(t)\phi_Y(s)$:

$$V(X, Y)^2 = \|\phi_{X,Y}(t, s) - \phi_X(t)\phi_Y(s)\|_w^2 \quad (4)$$

$$= \frac{1}{c} \int_{\mathbb{R}^{p+q}} \frac{|\phi_{X,Y}(t, s) - \phi_X(t)\phi_Y(s)|^2}{\|t\|_p^{1+p} \|s\|_q^{1+q}} dt ds. \quad (5)$$

Empirical Distance Covariance

Given the observed sample $(\mathbf{X}, \mathbf{Y}) = \{(X_k, Y_k), k = 1, \dots, n\}$, define $a_{kl} = \|X_k - X_l\|_p$, $b_{kl} = \|Y_k - Y_l\|_q$, for $k, l = 1, \dots, n$. The empirical distance covariance $V_n(\mathbf{X}, \mathbf{Y})$ is defined by

$$V_n^2(\mathbf{X}, \mathbf{Y}) = \frac{1}{n^2} \sum_{k,l=1}^n A_{k,l} B_{k,l} \quad (6)$$

where $A_{k,l} = a_{kl} - \bar{a}_{k\cdot} - \bar{a}_{\cdot l} + \bar{a}_{\cdot\cdot}$ and $B_{k,l} = b_{kl} - \bar{b}_{k\cdot} - \bar{b}_{\cdot l} + \bar{b}_{\cdot\cdot}$.

- The subscript “.” denotes that the mean is computed for the index that it replaces.
- These formulas are similar to computing formulas in analysis of variance, so the distance covariance statistic is very easy to compute.
- $V_n^2(\mathbf{X}, \mathbf{Y}) = \|\phi_{X,Y}^n(t, s) - \phi_X^n(t)\phi_Y^n(s)\|_w^2$ (Theorem 1 of SRB)
 - $\phi_{X,Y}^n(t, s) = n^{-1} \sum_{k=1}^n e^{it^\top X_k + is^\top Y_k}$
 - $\phi_X^n(t) = n^{-1} \sum_{k=1}^n e^{it^\top X_k}$, $\phi_Y^n(s) = n^{-1} \sum_{k=1}^n e^{is^\top Y_k}$

Test Statistic

■ If $E(|\mathbf{X}|_p + |\mathbf{Y}|_q) < \infty$, then

1 if $\mathbf{X}$ and $\mathbf{Y}$ are independent, $nV_n^2(\mathbf{X}, \mathbf{Y})/S_2 \xrightarrow{D} Q$

■ where Q is a quadratic form of centered Gaussian random variables with coefficients that depend on the **distributions** of X and Y , as $n \rightarrow \infty$;

2 if $\mathbf{X}$ and $\mathbf{Y}$ are dependent, then $nV_n^2(\mathbf{X}, \mathbf{Y})/S_2 \xrightarrow{P} \infty$, as $n \rightarrow \infty$.

3 where $S_2 = n^{-2} \sum_{k,l=1}^n a_{kl} n^{-2} \sum_{k,l=1}^n b_{kl}$ converges a.s. to a constant by the SLLN;

■ Implement the test based on nV_n^2 as a permutation test!

Compute Empirical Distance Covariance

```
dCov <- function(x, y) {
  x <- as.matrix(x)
  y <- as.matrix(y)
  n <- nrow(x)
  m <- nrow(y)
  if (n != m || n < 2) stop("Sample sizes must agree")
  if (! (all(is.finite(c(x, y)))))
    stop("Data contains missing or infinite values")

  Ak1 <- function(x) {
    d <- as.matrix(dist(x))
    m <- rowMeans(d)
    M <- mean(d)
    a <- sweep(d, 1, m)
    b <- sweep(a, 2, m)
    return(b + M)
  }
  A <- Ak1(x)
  B <- Ak1(y)
  dCov <- sqrt(mean(A * B))
  dCov
}
```

Test Statistic

```
> # compute empirical distance covariance  
> dCov(x, y)  
[1] 0.06436159
```

- For iris data example, $nV_n^2 = 0.06436159^2 * 50 = 0.207$.

An Example: Iris data

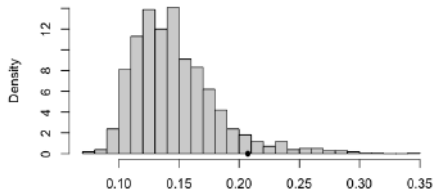
```
data(iris)
z <- as.matrix(iris[1:50, 1:4]) # Data matrix
x <- z[, 1:2]
y <- z[, 3:4]
n = nrow(z)
(D.obs = n*dCov(x, y)^2) # 0.207

nperm = 1000
D.perm = numeric(nperm)
for(i in 1:nperm){
  ix = sample(n)
  x <- z[, 1:2] #leave x as is
  y <- z[ix, 3:4] #permute rows of y
  D.perm[i] = n*dCov(x, y)^2
}
```

An Example: Iris data

```
> mean(c(D.obs, D.perm) >= D.obs)
[1] 0.05994006
```

```
hist(D.perm, nclass="scott", xlab="", main="",
      freq=FALSE)
points(D.obs, 0, cex=1, pch=16)
```



The Bootstrap

Rizzo, Chapter 7

Fangfang Wang
MA 551

Generating A Sample Distribution

- We have seen how to simulate in order to conduct tests in R.
- We can do the same thing for other measures of uncertainty.
- Variance of estimates
 - Simulate according to the model with parameters θ .
 - Estimate parameters.
 - Look at the distribution of parameter estimates over many simulations.
- Note: simulation depends on the parameters available.
- We can also evaluate bias.

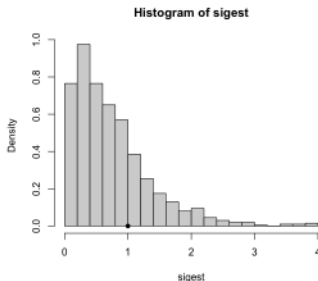
Example: Properties of the Sample Variance

Why we divide by $n - 1$? Consider $\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$ for $n = 4$ and x_i sampled from $N(0, \sigma^2)$ with $\sigma^2 = 1$:

```
nsim = 1000
sigest = rep(0,nsim)

for(i in 1:nsim){
  X = rnorm(4)
  sigest[i]=mean((X-mean(X))^2)
}

hist(sigest,20,probability=T)
points(1,0,pch=16)
```



Sample Variance

The expected value of the estimate, $E(\hat{\sigma}^2)$, is estimated by

```
mean(sigest)
```

and the bias = $E(\hat{\sigma}^2) - \sigma^2$ is (since the true variance is 1)

```
> bias = mean(sigest) - 1
```

```
[1] -0.2553418
```

and the standard error, from the standard deviation of simulated samples, is

```
> sighat.sd = sd(sigest)
```

```
[1] 0.5944814
```

Making Fewer Assumptions

Some important limitations to value of simulation:

- Only valid under the parameters you use to simulate.
 - Bias of $\hat{\sigma}^2$ is σ^2/n – changes with σ^2 .
 - But we don't know the parameters – we just have data.
 - Can always plug in our estimate, if that is biased, our estimate of bias is also biased.
- Only valid assuming the distribution you simulate from represents the data generating mechanism.

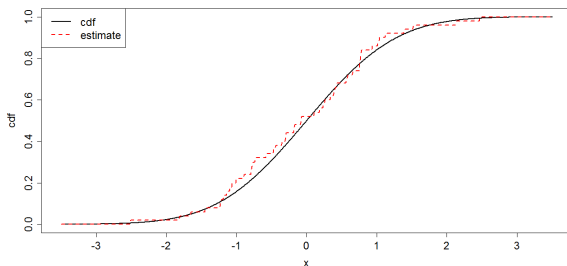
Maybe we could make more use of the data.

The Bootstrap

Introduced by Bradley Efron (AoS, 1979), > 26,000 citations from all of NSF's funding areas.

Simple idea:

- I want to simulate from distribution F , but I don't know it.
- However, I do have data $\Rightarrow$ use this as an estimate of F !



The Bootstrap

- The term “bootstrap” can refer to nonparametric bootstrap or parametric bootstrap.
- In nonparametric bootstrap, the distribution is not specified.
- Estimate the distribution of a population by resampling.
- Resampling methods treat an *observed sample* as a *finite population*, and random samples are generated (resampled) from it to estimate population characteristics and make inferences about the sampled population.
- Bootstrap methods are often used when the distribution of the target population is not specified; the sample is the only information available.

Empirical Estimates of a Distribution

- Cumulative distribution function: $F(x) = P(X \leq x)$
- Estimate from data $\{X_1, \dots, X_n\} \sim F$:

$$\hat{F}(x) = \frac{1}{n} \sum_{i=1}^n I(X_i \leq x) \quad (1)$$

“jumps” $1/n$ at each observation.

- 1 $\hat{F}(x)$ converges to $F(x)$ everywhere as $n \rightarrow \infty$. (Why?)

- 2 Interpretation of X drawn from $\hat{F}(x)$: *For each i , X takes value X_i with probability $1/n$.*

Empirical Estimates of a Distribution

- Cumulative distribution function: $F(x) = P(X \leq x)$
- Estimate from data $\{X_1, \dots, X_n\} \sim F$:

$$\hat{F}(x) = \frac{1}{n} \sum_{i=1}^n I(X_i \leq x) \quad (2)$$

“jumps” $1/n$ at each observation.

- 1 Practically: to sample from $\hat{F}$, choose one X_i at random.
- 2 To sample more “re-sample with replacement”: *each time you choose an X_i , replace it in the data set for the next sample.*
- 3 Resampling generates a random sample $X_1^*, \dots, X_n^*$, where X_i^* are iid, uniformly distributed on the set $(X_1, \dots, X_n)$.

The Bootstrap as a Plug-in Rule

Estimate $\theta(F)$ by $\theta(\hat{F})$

We are often interested in parameters that are (or are related to) expectations with respect to the distribution F .

- Population mean: $\mu = E_F(X)$
- Population variance: $\sigma^2 = E_F \{ (X - \mu)^2 \}$
- Probabilities: Let $\hat{\theta}(X_1, \dots, X_n)$ be a complicated function of the data. Then

$$P_F(\hat{\theta} \leq t) = E_F \left(I_{\{\hat{\theta} \leq t\}} \right)$$

- Quantiles: Find t_α such that $P_F(\hat{\theta} \leq t_\alpha) = \alpha$

Estimate using the same expectations with respect to $\hat{F}$!

Sampling Schemes

Some general terminology (informal)

sample from F generate a random variable X according to distribution F

resample from a data set $\{X_1, \dots, X_n\}$, choose one at random.

with replacement when I choose an X_i , replace it in the data for the next draw.

without replacement when I choose an X_i , leave it out of the data for the next draw.

Different types of samples:

bootstrap resample n observations with replacement

subsample resample $k \leq n$ observations without replacement.

Note: bootstrap samples will have repeated values; a subsample of size n is a permutation.

sample

Will *resample* objects with or without replacement and will return a vector of required size

```
# A permutation of the numbers 5:10  
sample(5:10)
```

```
# A subsample of size 3  
sample(5:10,size=3)
```

```
# A bootstrap sample  
sample(5:10,replace=TRUE)
```

```
# A subsample of size 3 with replacement  
sample(5:10,size=3,replace=TRUE)
```

If N an integer, `sample(N)` is the same as `sample(1:N)`.

The Bootstrap Recipe

Given $\{X_1, \dots, X_n\}$, and a statistic $\hat{\theta} \equiv \hat{\theta}(X_1, \dots, X_n)$ that estimates a parameter or characteristic of the population, θ .

- Repeat for $b = 1, \dots, B$:
 - Draw a *bootstrap sample* $\{X_1^*, \dots, X_n^*\}$ by resampling $X_1, \dots, X_n$ with replacement.
 - Record $\hat{\theta}_b^* \equiv \hat{\theta}(X_1^*, \dots, X_n^*)$.
- Use $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$ to approximate the sampling distribution of $\hat{\theta}$.

Bias Correction

Some statistics are biased; to assess this we consider the average bootstrap replicate

$$\bar{\theta}^* = \frac{1}{B} \sum_{b=1}^B \hat{\theta}_b^*, \quad (3)$$

then we can measure the bias in $\hat{\theta}$ by

$$\widehat{bias} = \bar{\theta}^* - \hat{\theta}. \quad (4)$$

$\widehat{bias}$: Bootstrap estimate of bias

We can even correct our estimate $\hat{\theta}$ by subtracting off the bias

$$\hat{\theta}^c = \hat{\theta} - \widehat{bias} = 2 * \hat{\theta} - \bar{\theta}^* \quad (5)$$

Bootstrap Estimation of Standard Error

- $se(\hat{\theta})$: standard error of an estimator $\hat{\theta}$
- The bootstrap estimate of $se(\hat{\theta})$ is the sample standard deviation of the bootstrap replicates $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$.

$$\widehat{se}(\hat{\theta}) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\theta}_b^* - \bar{\theta}^*)^2} \quad (6)$$

- The number of replicates needed for good estimates of standard error is not large.
 - $B = 50$ is usually large enough, and rarely is $B > 200$ necessary.
 - Efron and Tibshirani (1993): *An introduction to the Bootstrap*, p. 52.
 - Note: much larger B will be needed for confidence interval estimation.
- If $|\widehat{bias}|/\widehat{se}(\hat{\theta}) < .25$, not necessary to adjust for bias.

Standard Normal Bootstrap Confidence Interval

Number of possible ways to do confidence intervals.

Simplest: normal approximation

- Compute resampled versions of $\hat{\theta}$: $\hat{\theta}_1^*, \dots, \hat{\theta}_B^*$.
- Obtain the standard deviation $\hat{se}(\hat{\theta})$ (See Eq. (6))
- Compute the $100(1 - \alpha)\%$ CI for θ as

$$(\hat{\theta} - z_{\alpha/2} \hat{se}(\hat{\theta}), \hat{\theta} + z_{\alpha/2} \hat{se}(\hat{\theta})) \quad (7)$$

for $z_{\alpha/2}$ the normal critical value.

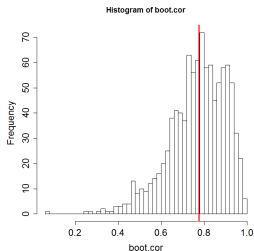
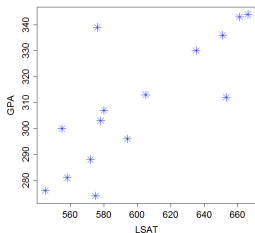
- Replace $\hat{\theta}$ in (7) with $\hat{\theta}^c$ for bias correction.

Assumes that $\hat{\theta}$ is approximately normally distributed

Example

Law school data: average LSAT and GPA for 15 law schools

Interest is in correlation between LSAT and GPA.



```
library(bootstrap)
cor(law)
> (obs.cor=cor(law)[1,2])
[1] 0.7763745

nboot = 1000
boot.cor = rep(0,nboot)
n = nrow(law)

for(i in 1:nboot){
  bootstrap.sample = law[sample(n,replace=TRUE),]
  boot.cor[i] = cor(bootstrap.sample)[1,2]
}
```

```
# Estimate the bias  
> cor.bias = mean(boot.cor)-obs.cor  
[1] -0.008115637
```

```
# Bias-Corrected Estimate  
> obs.cor.c = obs.cor-cor.bias  
[1] 0.7844901
```

```
# Bootstrap Standard Error  
> cor.se = sd(boot.cor)  
[1] 0.1319921
```

```
# Bias-Corrected Standard Normal Bootstrap CI  
> obs.cor.c + c(-1,1)*qnorm(0.975)*cor.se  
[1] 0.6742477 0.8947325
```

Percentile Bootstrap Confidence Interval

Can also use the empirical distribution of the bootstrap statistics.

Let $\hat{t}_\alpha$ denote the α -quantile of the bootstrap distribution of $\hat{\theta}^*$; i.e.,

$$P_{\hat{F}}(\hat{\theta}^* \leq \hat{t}_\alpha) = \alpha$$

Then, a $100(1 - \alpha)\%$ percentile bootstrap CI for θ is

$$(\hat{t}_{\alpha/2}, \hat{t}_{1-\alpha/2}) \tag{8}$$

This can be approximated by

$$(\hat{\theta}_{\alpha/2}^*, \hat{\theta}_{1-\alpha/2}^*) \tag{9}$$

where $\hat{\theta}_\alpha^*$ is the α th quantile in $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$.

Basic Bootstrap Confidence Interval

Alternatively, suppose b_α is the α -quantile of $\hat{\theta} - \theta$; i.e.

$P(\hat{\theta} - \theta < b_\alpha) = \alpha$. Then

$$P(\hat{\theta} - b_{1-\alpha/2} < \theta < \hat{\theta} - b_{\alpha/2}) = 1 - \alpha$$

- Replace the (unknown) distribution of $\hat{\theta} - \theta$ by the known distribution of $\hat{\theta}^* - \hat{\theta}$

- Yields $\hat{b}_{\alpha/2} = \hat{t}_{\alpha/2} - \hat{\theta}$ and the $100(1 - \alpha)\%$ CI for θ

$$(2\hat{\theta} - \hat{t}_{1-\alpha/2}, 2\hat{\theta} - \hat{t}_{\alpha/2}) \quad (10)$$

This can be approximated by

$$(2\hat{\theta}^c - \hat{\theta}_{1-\alpha/2}^*, 2\hat{\theta}^c - \hat{\theta}_{\alpha/2}^*) \quad (11)$$

where $\hat{\theta}_\alpha^*$ is the α th quantile in $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$.

Continuing Example

```
# Quantiles of Bootstrap Distribution
b0.025 = quantile(boot.cor,0.025)
b0.975 = quantile(boot.cor,0.975)

# Percentile Bootstrap Confidence interval
> c(b0.025,b0.975)
0.4645733 0.9615963

# Basic Bootstrap Confidence Interval
> 2*obs.cor.c - c(b0.975, b0.025)
0.607384 1.104407
```

- Upper limit ≥ 1 can be thresholded (remember, this interval is just meant to capture the “truth”, 95% of the time).

Percentile- t Method

- Let t_α denote the α -quantile of $T = (\hat{\theta} - \theta)/\widehat{se}(\hat{\theta})$.
- Then a $100(1 - \alpha)\%$ CI is given by

$$\left(\hat{\theta} - t_{1-\alpha/2} \widehat{se}(\hat{\theta}), \hat{\theta} - t_{\alpha/2} \widehat{se}(\hat{\theta}) \right) \quad (12)$$

- $100(1 - \alpha)\%$ Bootstrap t CI for θ

$$\left(\hat{\theta}^c - \hat{t}_{1-\alpha/2} \widehat{se}(\hat{\theta}), \hat{\theta}^c - \hat{t}_{\alpha/2} \widehat{se}(\hat{\theta}) \right) \quad (13)$$

where $\hat{t}_\alpha = \alpha$ -quantile of bootstrap distribution of $T^* = (\hat{\theta}^* - \hat{\theta})/\widehat{se}^*$ (a version of T based on a resample), i.e.,

$$P_{\hat{F}}(T^* \leq \hat{t}_\alpha) = \alpha \quad (14)$$

Note that $\widehat{se}^*$ is different from $\widehat{se}(\hat{\theta})$.

The Bootstrap t interval (studentized bootstrap interval)

Given $\{X_1, \dots, X_n\}$ and a statistic $\hat{\theta} \equiv \hat{\theta}(X_1, \dots, X_n)$.

- 1 Repeat for $b = 1, \dots, B$:
 - 1 Draw a bootstrap sample $X_1^*, \dots, X_n^*$.
 - 2 Record $\hat{\theta}_b^* \equiv \hat{\theta}(X_1^*, \dots, X_n^*)$.
 - 3 Compute $\widehat{se}(\hat{\theta}_b^*)$ by **bootstrapping** $\{X_1^*, \dots, X_n^*\}$.
 - 4 Compute $t_b^* = (\hat{\theta}_b^* - \hat{\theta}) / \widehat{se}(\hat{\theta}_b^*)$
- 2 Use $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$ to compute $\widehat{se}(\hat{\theta})$.
- 3 Use $t_1^*, \dots, t_B^*$ to represent the sampling distribution of $T^* = (\hat{\theta}^* - \hat{\theta}) / \widehat{se}^*$; and obtain $\hat{t}_\alpha$ the α th quantile of $\{t_1^*, \dots, t_B^*\}$.

Disadvantage: bootstrap nested in a bootstrap.

Conditionally-Specified Models

Frequently we only describe part of the data generating mechanism.

Eg: regression model

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

with $\epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$.

- What about x_i ? Treated as fixed (often chosen by experimenter).
- Or, frequently, $x_i \sim h(x)$, but h not specified.

Example: Multiple Linear Regression

For multiple linear regression

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \epsilon_i, \quad i = 1, \dots, n,$$

where $\epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$. Also written as

$$\mathbf{y} = X\beta + \epsilon$$

Squared error is now represented by

$$SSE(\beta) = (\mathbf{y} - X\beta)^T (\mathbf{y} - X\beta)$$

so that the derivative of squared error is

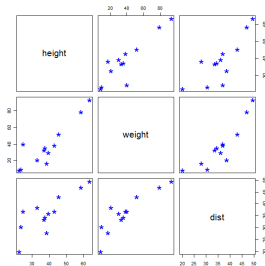
$$\frac{d}{d\beta} SSE(\beta) = 2(X^T X \beta - X^T \mathbf{y})$$

which is 0 at

$$\hat{\beta} = (X^T X)^{-1} X^T \mathbf{y}$$

A Data Set

- 12 children's height, weight and arterial distance from the wrist to the heart.
- Used as guidelines for inserting catheters.
- Model distance in terms of height and weights.



```
> mod = lm(dist~height+weight,data=heart)
> summary(mod)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	24.50804	5.12461	4.782	0.000998	***
height	0.05091	0.21060	0.242	0.814396	
weight	0.25495	0.10326	2.469	0.035624	*

Any issue?

A Simulation

We'll use the estimated coefficients and residual standard error to simulate at the observed covariates.

```
beta = mod$coef      # Start from estimated coefficients

# Design matrix
X = as.matrix(cbind(rep(1,12),heart[,2:3]))

# Predicted values (also from mod$fit)
pred = X%*%beta      # pred = mod$fit

# Residual standard error
sigma = summary(mod)$sigma
```

Vectorizing A Simulation

Create a large matrix where response for each data set is in one column.

Recall that $\mathbf{y} = X\beta + \epsilon$. Repeat the same $X\beta$ over each column, but create a matrix of simulated ϵ .

```
nsim = 1000
nobs = dim(heart)[1]    # number of observations
Ysim = matrix(pred,nobs,nsim,byrow=FALSE) +
        matrix(rnorm(nobs*nsim,sd=sigma),nobs,nsim)
```

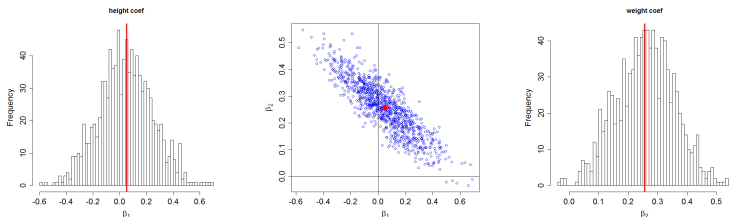
Now estimate β ; the `solve` command inverts a matrix:

```
beta.sim = solve( t(X)%*%X, t(X)%*%Ysim )
```

Because the estimate is linear in `Ysim` we can obtain them all at once.

Continuing

Simulation results:



Lovely demonstration of effect of correlated covariates. (Note that the correlation between $\hat{\beta}_1$ and $\hat{\beta}_2$ is -0.4384.)

Bootstrap options

- Re-sample (x_i, y_i) pairs and do standard bootstrap.
- Try to re-sample the ϵ_i – corresponds to our model.

Residual Bootstrap

Basically restricted to linear regression models:

$$\mathbf{y} = X\beta + \epsilon$$

- 1 Estimate $\hat{\beta}$.
- 2 Estimate residuals $\hat{\epsilon}_i = y_i - \mathbf{x}_i\hat{\beta}$, $i = 1, \dots, n$.
- 3 Bootstrap residuals to produce $\hat{\epsilon}_i^*$, $i = 1, \dots, n$.
- 4 Add bootstrapped residuals back to predictions
 $y_i^* = \mathbf{x}_i\hat{\beta} + \hat{\epsilon}_i^*$, $i = 1, \dots, n$.
- 5 Estimate $\hat{\beta}^*$ for bootstrapped (x_i, y_i^*) .
- 6 Repeat Steps 3-5 for B times and obtain $\hat{\beta}_b^*$, $b = 1, \dots, B$

Now all the bias, standard error, confidence interval statistics can be calculated with the same recipe.

Why Residual Bootstrap?

More stable, avoids ties in the $\mathbf{x}_i$, doesn't change a fixed design.

Continuing The Example

```
# Estimate errors
eps.hat = heart$dist - pred

# Now bootstrap residuals
nboot = 1000
eps.boot = eps.hat[sample(12,size=nboot*12,
                          replace=TRUE)]
eps.boot = matrix(eps.boot,12,nboot)

# Create data
Y.boot = matrix(pred,12,nboot,byrow=FALSE)
          + eps.boot

# And re-estimate
beta.boot = solve( t(X)%*%X, t(X)%*%Y.boot)
```

Usual Statistics

Calculate the same statistics as before

```
# Bias
> biases = apply(beta.boot,1,mean) - beta
      (Intercept)      height      weight
0.138147064    -0.001146391    -0.001338689
```

```
# Standard Error
> se.boot = apply(beta.boot,1,sd)
      (Intercept)      height      weight
4.38185528  0.17610865  0.08662377
```

Biases are probably not real but

```
# Bias-corrected estimator
beta.c = beta-biases
```

Confidence Intervals

```
# Lower and Upper Bounds
>lb = apply(beta.boot,1,quantile,0.025)
>ub = apply(beta.boot,1,quantile,0.975)

                lb                ub
(Intercept) 15.2245321 33.4174053
height      -0.2877086  0.4436933
weight       0.0728820  0.4254932

# 95% Confidence Interval
>cbind(2*beta - ub, 2*beta-lb)

                [,1]                [,2]
(Intercept) 15.5986750 33.7915482
height      -0.3418719  0.3895299
weight       0.0844015  0.4370127
```

Parametric Bootstrap

Residual bootstrap is not always applicable:

Eg: logistic regression;

$$y_i \in \{0, 1\}, \quad \frac{P(y_i = 1)}{P(y_i = 0)} = \exp(\mathbf{x}_i \beta)$$

Doesn't make sense to do look at residuals.

Instead, estimate $\hat{\beta}$ and create a new data set by generating each y_i^* according to

$$P(y_i^* = 1) = \frac{e^{\mathbf{x}_i \hat{\beta}}}{1 + e^{\mathbf{x}_i \hat{\beta}}} \quad (15)$$

Isn't this just estimate parameters and then simulate data from the model?

Yes! But naming is part of good salesmanship.

Examples

- Linear regression simulation on Slide 27; this is exactly a parametric bootstrap for linear regression.

- *Exercise 7.4 of Rizzo, page 212.*

Referring to the air-conditioning data set `aircondit` provided in the `boot` package.

The 12 observations are the times in hours between failures of air-conditioning equipment. Assume that the times between failures follow an exponential distribution with hazard rate λ , $\text{Exp}(\lambda)$.

Obtain the MLE of the hazard rate λ and use bootstrap to estimate the bias and standard error of the estimate.

- For $X \sim \text{Exp}(\lambda)$, $E(X) = 1/\lambda$.

Parametric Bootstrap

- 1 Given $\{X_1, \dots, X_n\}$, obtain parametric estimate $\hat{F}$ of unknown F , i.e., get estimates of parameters
- 2 Draw a bootstrap sample from $\hat{F}$ and calculate statistic of interest on bootstrap sample
 - i.e., simulate data values from parametric model using estimates as parameters, $X_1^*, X_2^*, \dots, X_n^* \sim \hat{F}$
 - $\hat{\theta}^* = \hat{\theta}(X_1^*, X_2^*, \dots, X_n^*)$
- 3 Repeat Step 2 independently a large number B of times obtaining bootstrap replications: $\hat{\theta}_1^*, \hat{\theta}_2^*, \dots, \hat{\theta}_B^*$.
- 4 Use $\{\hat{\theta}_1^*, \hat{\theta}_2^*, \dots, \hat{\theta}_B^*\}$
 - to estimate bias, see (4)
 - to estimate standard error of $\hat{\theta}$, see (6)
 - to obtain confidence interval, see (7), (9), (11), or (13)

Exercise 7.4 of Rizzo, page 212

```
library('boot')
data(aircondit)
(lambda.hat = 1/mean(aircondit$hours)) # compute mle
n = length(aircondit$hours)
B = 1000
lambda.boot = numeric(B)
for(b in 1:B){
  bootstrap.sample = rexp(n, rate = lambda.hat)
  lambda.boot[b] = 1/mean(bootstrap.sample)
}

(bias.lambda = mean(lambda.boot)-lambda.hat) # Bias
# Bias-Corrected Estimate
(lambda.hat.c = lambda.hat-bias.lambda)

(se.lambda = sd(lambda.boot) ) # Standard error
```

Summary

- Simulation (parametric bootstrap) a tool for evaluating confidence intervals about estimated parameters.
- *Bootstrap*: avoids having to know distribution of data.
- Useful for
 - Bias Correction
 - Estimate standard errors
 - Confidence intervals
- Residual bootstrap for linear regression models.

But

- Justification is asymptotic: requires enough data that empirical distribution approximates truth.
- Won't work for every problem or every statistic (but most standard stats are OK). See Notes #7.
- How about dependent data? Project!

Cross Validation Jackknife BCa Bootstrap CI

Rizzo, Chapter 7

Fangfang Wang
MA 551

Outline

- 1 Cross Validation
 - Orthogonal Polynomials
- 2 Jackknife
- 3 Confidence Interval
- 4 More examples on Bootstraps

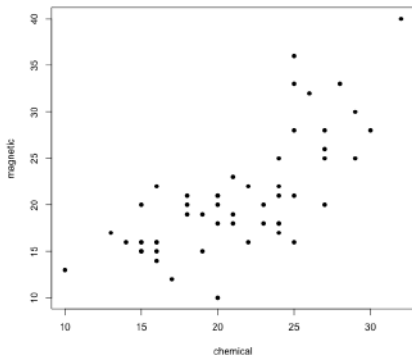
Cross Validation

- Data partitioning method
- To assess the stability of parameter estimates, the accuracy of a classification algorithm, the adequacy of a fitted model, and in many other applications.
 - A special case: Jackknife
- Partition the data into *training* and *test* sets:
 - Estimate model using the training set only.
 - Estimate misclassification rate by running the classifier on the test set.
 - Assess the fit of the model by holding back a test set from the model estimation, and then using the test set to see how well the model fits the new test data.
- “n-fold” CV partitions the data into n test sets (or test points), and then the model fitting leaves out one test set in turn, so that the models are fitted n times; “leave-one-out” procedure.

An Example, Rizzo 7.17

The `ironslag(DAAG)` data set has 53 measurements of iron content by two methods, `chemical` and `magnetic`.

```
> head(ironslag)
  chemical magnetic
1       24       25
2       16       22
3       24       17
4       18       21
5       18       20
6       10       13
```



Example Continued

Models for predicting magnetic measurement (Y) from chemical measurement (X) are:

- 1 Linear: $Y = \beta_0 + \beta_1 X + \varepsilon$
- 2 Quadratic: $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \varepsilon$
- 3 Exponential: $\log(Y) = \beta_0 + \beta_1 X + \varepsilon$
- 4 Log-log: $\log(Y) = \beta_0 + \beta_1 \log(X) + \varepsilon$

Example Continued

```
library(DAAG); attach(ironslag)
par(mfrow=c(2,2))
a <- seq(10, 40, .1)      #sequence for plotting fits

L1 <- lm(magnetic ~ chemical)
plot(chemical, magnetic, main="Linear", pch=16)
yhat1 <- L1$coef[1] + L1$coef[2] * a
lines(a, yhat1, lwd=2)

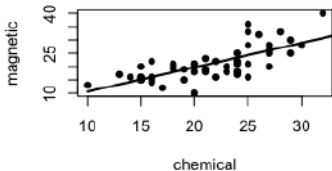
L2 <- lm(magnetic ~ chemical + I(chemical^2))
plot(chemical, magnetic, main="Quadratic", pch=16)
yhat2 <- L2$coef[1] + L2$coef[2] * a + L2$coef[3] * a^2
lines(a, yhat2, lwd=2)

L3 <- lm(log(magnetic) ~ chemical)
plot(chemical, magnetic, main="Exponential", pch=16)
logyhat3 <- L3$coef[1] + L3$coef[2] * a
yhat3 <- exp(logyhat3)
lines(a, yhat3, lwd=2)

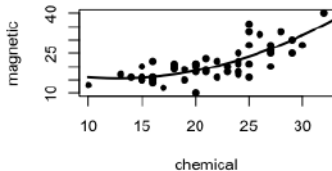
L4 <- lm(log(magnetic) ~ log(chemical))
plot(log(chemical), log(magnetic), main="Log-Log", pch=16)
logyhat4 <- L4$coef[1] + L4$coef[2] * log(a)
lines(log(a), logyhat4, lwd=2)
```

Example Continued

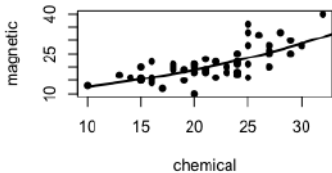
Linear



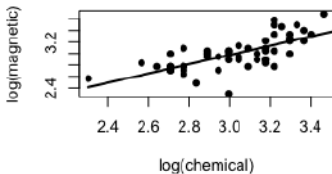
Quadratic



Exponential



Log-Log



Example Continued

- Once the model is estimated, we want to assess the fit.
- Focus on prediction error.
- Prediction error is estimated by CV, without making strong distributional assumptions about error.

Leave-one-out Cross Validation

Procedure to estimate prediction error by leave-one-out CV

- 1 For $k = 1, \dots, n$, let observation (x_k, y_k) be the test point and use the remaining observations to fit the model.
 - 1 Fit the model(s) using only the $n - 1$ observations in the training set, $(x_i, y_i), i \neq k$.
 - 2 Compute the predicted response $\hat{y}_k = \hat{\beta}_0 + \hat{\beta}_1 x_k$ for the test point.
 - 3 Compute the prediction error $e_k = y_k - \hat{y}_k$.
- 2 Estimate the mean of the squared prediction errors
$$\hat{\sigma}^2 = n^{-1} \sum_{k=1}^n e_k^2.$$

Example Continued

```
n <- length(magnetic)
e1 <- e2 <- e3 <- e4 <- numeric(n)
# for n-fold cross validation
# fit models on leave-one-out samples
for (k in 1:n) {
  y <- magnetic[-k]; x <- chemical[-k]

  J1 <- lm(y ~ x)
  yhat1 <- J1$coef[1] + J1$coef[2]*chemical[k]
  e1[k] <- magnetic[k] - yhat1

  J2 <- lm(y ~ x + I(x^2))
  yhat2 <- J2$coef[1] + J2$coef[2]*chemical[k] +
    J2$coef[3]*chemical[k]^2
  e2[k] <- magnetic[k] - yhat2

  J3 <- lm(log(y) ~ x)
  yhat3 <- exp(J3$coef[1] + J3$coef[2]*chemical[k])
  e3[k] <- magnetic[k] - yhat3

  J4 <- lm(log(y) ~ log(x))
  yhat4 <- exp(J4$coef[1] + J4$coef[2]*log(chemical[k]))
  e4[k] <- magnetic[k] - yhat4
}
```

Example Continued

```
> c(mean(e1^2), mean(e2^2), mean(e3^2), mean(e4^2))
[1] 19.55644 17.85248 18.44188 20.45424
```

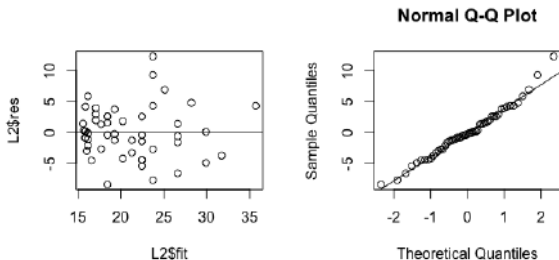
Best fit: Model 2, $Y_j = \beta_0 + \beta_1 X_j + \beta_2 X_j^2 + \varepsilon_j, j = 1, \dots, n.$

```
> summary(L2)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	24.49262	9.14657	2.678	0.0100	**
chemical	-1.39334	0.88852	-1.568	0.1232	
I(chemical^2)	0.05452	0.02081	2.620	0.0116	*

```
> cor(chemical, chemical^2)
[1] 0.991
```



Orthogonal Polynomials

Key Idea: Transform $\{1, X, X^2\}$ to **orthogonal basis** $\{1, P_1(X), P_2(X)\}$ where:

$$\sum_{j=1}^n P_1(X_j) = \sum_{j=1}^n P_2(X_j) = 0$$

$$\sum_{j=1}^n P_1(X_j)P_2(X_j) = 0 \quad (\text{Orthogonality!})$$

New Model:

$$Y_j = \alpha_0 + \alpha_1 P_1(X_j) + \alpha_2 P_2(X_j) + \varepsilon_j$$

Important Property

Same fitted values, better statistical properties!

Construction of Orthogonal Polynomials (Gram-Schmidt)

Step-by-Step Orthogonalization:

1 Start with basis: $v_0 = 1, v_1 = X, v_2 = X^2$

2 Orthogonalize:

$$P_0(X) = 1$$

$$P_1(X) = X - \frac{\langle X, 1 \rangle}{\langle 1, 1 \rangle} \cdot 1 = X - \bar{X}$$

3 Normalize:

$$\tilde{P}_1(X) = \frac{P_1(X)}{\sqrt{\sum P_1^2(X_j)}}$$

Quadratic Orthogonal Polynomial

Constructing $P_2(X)$:

$$\begin{aligned} P_2(X) &= X^2 - \text{proj}_1(X^2) - \text{proj}_{P_1}(X^2) \\ &= X^2 - \bar{X}^2 - \frac{\langle X^2, P_1 \rangle}{\langle P_1, P_1 \rangle} P_1(X) \end{aligned}$$

After simplification:

$$P_2(X) = (X - \bar{X})^2 - \frac{\sum (X_j - \bar{X})^3}{\sum (X_j - \bar{X})^2} (X - \bar{X}) - \frac{\sum (X_j - \bar{X})^2}{n}$$

Normalized version:

$$\tilde{P}_2(X) = \frac{P_2(X)}{\sqrt{\sum P_2^2(X_j)}}$$

Alternatively,

$$Y_j = \alpha_0 + \tilde{\alpha}_1 \tilde{P}_1(X_j) + \tilde{\alpha}_2 \tilde{P}_2(X_j) + \tilde{\varepsilon}_j.$$

In R: Using `poly()` Function

```
# Original model (problematic)
L2 <- lm(magnetic ~ chemical + I(chemical^2))

# Orthogonal polynomial model (recommended)
L2_ortho <- lm(magnetic ~ poly(chemical, degree = 2, raw = FALSE))

> summary(L2_ortho)
Coefficients:
                                Estimate Std. Error t value Pr(>|t|)
(Intercept)                20.7547      0.5629   36.869  < 2e-16
poly(chemical, degree = 2, raw = FALSE)1  33.2977      4.0982    8.125 1.07e-10
poly(chemical, degree = 2, raw = FALSE)2  10.7373      4.0982    2.620  0.0116
```

Key Arguments

- `raw = FALSE`: Returns orthogonal polynomials
- `raw = TRUE`: Returns raw polynomials $1, X, X^2$
- Default is `raw = FALSE`

Outline

- 1 Cross Validation
 - Orthogonal Polynomials
- 2 Jackknife
- 3 Confidence Interval
- 4 More examples on Bootstraps

The Jackknife

- The jackknife is another resampling method, a few decades earlier than the bootstrap.
- M. H. Quenouille: *Approximate tests of correlation in time series*. JRSS-B, 11:68-84, 1949.
- The jackknife is like a “leave-one-out” type of CV.
- Let $\mathbf{x} = (x_1, \dots, x_n)$ be an observed random sample.
- Define the i th jackknife sample

$$\mathbf{x}_{(i)} = (x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) \quad (1)$$

- If $\hat{\theta} = T_n(\mathbf{x})$, define the i th jackknife replicate

$$\hat{\theta}_{(i)} = T_{n-1}(\mathbf{x}_{(i)}), \quad (2)$$

$$i = 1, \dots, n.$$

The Jackknife Estimate of Bias

- The average jackknife replicate: $\bar{\theta}_{jack} = n^{-1} \sum_{i=1}^n \hat{\theta}_{(i)}$
- The Jackknife Estimate of Bias:

$$\widehat{bias}_{jack} = (n-1)(\bar{\theta}_{jack} - \hat{\theta}) \quad (3)$$

- The Jackknife Estimate of θ

$$\hat{\theta} - \widehat{bias}_{jack} = n\hat{\theta} - (n-1)\bar{\theta}_{jack}. \quad (4)$$

- ✓ Suppose θ is the variance and $\hat{\theta} = n^{-1} \sum_{i=1}^n (x_i - \bar{x})^2$. The bias is

$$bias(\hat{\theta}) = E(\hat{\theta} - \theta) = -n^{-1}\theta \quad (5)$$

Thus,

$$E(\bar{\theta}_{jack} - \hat{\theta}) = E(\bar{\theta}_{jack} - \theta) - E(\hat{\theta} - \theta) \quad (6)$$

$$= -\frac{1}{n(n-1)}\theta = \frac{1}{n-1}bias(\hat{\theta}) \quad (7)$$

Note: Jackknife bias is asymptotically smaller than the bias of any given biased estimator.

The Jackknife Estimate of Bias

- The average jackknife replicate: $\bar{\theta}_{jack} = n^{-1} \sum_{i=1}^n \hat{\theta}_{(i)}$
- The Jackknife Estimate of Bias:

$$\widehat{bias}_{jack} = (n-1)(\bar{\theta}_{jack} - \hat{\theta}) \quad (8)$$

- The Jackknife Estimate of θ

$$\hat{\theta} - \widehat{bias}_{jack} = n\hat{\theta} - (n-1)\bar{\theta}_{jack}. \quad (9)$$

- ✓ Suppose $E(\hat{\theta}) = \theta + a(\theta)/n + b(\theta)/n^2 + O(n^{-3}) = \theta + \text{Bias}$. Then

$$E(\bar{\theta}_{jack} - \hat{\theta}) = \frac{a(\theta)}{n(n-1)} + \frac{b(\theta)(2n-1)}{n^2(n-1)^2} + O(n^{-4}) \quad (10)$$

$$(n-1)E(\bar{\theta}_{jack} - \hat{\theta}) = \frac{a(\theta)}{n} + \frac{b(\theta)(2n-1)}{n^2(n-1)} + O(n^{-3}) \approx \text{Bias} \quad (11)$$

$$E(\hat{\theta} - \widehat{bias}_{jack}) = \theta + O(n^{-2}) \quad (12)$$

The Jackknife Estimate of Standard Error

- The average jackknife replicate: $\bar{\theta}_{jack} = n^{-1} \sum_{i=1}^n \hat{\theta}_{(i)}$
- The Jackknife Estimate of Standard Error:

$$\widehat{se}_{jack}(\hat{\theta}) = \sqrt{\frac{n-1}{n} \sum_{i=1}^n (\hat{\theta}_{(i)} - \bar{\theta}_{jack})^2}. \quad (13)$$

- ✓ Let $\hat{\theta} = \bar{X}$. $Var(\hat{\theta}) = \sigma^2/n$.
- ✓ Then $\hat{\theta}_{(i)} = (n-1)^{-1} \sum_{k \neq i} X_k$ and

$$\frac{n-1}{n} \sum_{i=1}^n (\hat{\theta}_{(i)} - \bar{\theta}_{jack})^2 = \frac{1}{n(n-1)} \sum_{k=1}^n (X_k - \bar{X})^2. \quad (14)$$

- ✓ $E(\widehat{se}_{jack}(\hat{\theta})^2) = n^{-1} \sigma^2$.
- J. Tukey: *Bias and confidence in not-quite large samples*. Annals of Mathematical Statistics, 29:614, 1958.
- B. Efron and C. Stein, *The jackknife estimate of variance*. The Annals of Statistics, pp. 586-596, 1981.

Bootstrap Estimation of Bias and Standard Error

- Recall: Bootstrap estimate of bias

$$\widehat{bias} = \bar{\theta}^* - \hat{\theta} = \frac{1}{B} \sum_{b=1}^B \hat{\theta}_b^* - \hat{\theta}. \quad (15)$$

Bootstrap estimate of $se(\hat{\theta})$,

$$\widehat{se}(\hat{\theta}) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B \left(\hat{\theta}_b^* - \bar{\theta}^* \right)^2}. \quad (16)$$

- What are variances of $\widehat{bias}$ and $\widehat{se}(\hat{\theta})$?
- Use jackknife to estimate the variance of these estimates.
See Rizzo, Chapter 7.3.
- Project.

Outline

- 1 Cross Validation
 - Orthogonal Polynomials
- 2 Jackknife
- 3 Confidence Interval
- 4 More examples on Bootstraps

What is a Good Confidence Interval?

- If an exact CI can be formed, bootstrap CI should closely match exact CI.
- If an exact CI cannot be formed, bootstrap CI should give accurate coverage probabilities.
- Different bootstrap CI methods have different coverage accuracies.

First and Second Order Accurate

- Let n be the sample size.
- A confidence interval is first order accurate if the error tends to 0 at rate $n^{-1/2}$, and second order accurate if the error tends to 0 at rate n^{-1} .
- Standard Normal Bootstrap CI: first order accurate
 - can perform poorly if distribution is highly skewed
- Basic bootstrap CI: first order accurate.
- Percentile Bootstrap CI: first order accurate
- Bootstrap t CI: second order accurate
 - The studentized statistic is closer to being pivotal
- Project.

Better Bootstrap Confidence Intervals

- Modified version of percentile intervals
- Second order accurate
- Correction for bias and skewness
- The better bootstrap CI is called BCa for “bias corrected” and “adjusted for acceleration.”
- $100(1 - \alpha)\%$ BCa bootstrap CI for θ :

$$(\hat{\theta}_{\alpha_1}^*, \hat{\theta}_{\alpha_2}^*) \quad (17)$$

where $\hat{\theta}_{\alpha}^*$ is the α th quantile in $\{\hat{\theta}_1^*, \dots, \hat{\theta}_B^*\}$, and

$$\alpha_1 = \Phi \left(\hat{z}_0 + \frac{\hat{z}_0 + z_{\alpha/2}}{1 - \hat{a}(\hat{z}_0 + z_{\alpha/2})} \right) \quad (18)$$

$$\alpha_2 = \Phi \left(\hat{z}_0 + \frac{\hat{z}_0 + z_{1-\alpha/2}}{1 - \hat{a}(\hat{z}_0 + z_{1-\alpha/2})} \right) \quad (19)$$

Here, $\Phi(x)$ is the CDF of $N(0,1)$.

Better Bootstrap Confidence Intervals

- $\hat{z}_0$: bias correction factor; estimate median bias of the replicates $\hat{\theta}_b^*$ for $\hat{\theta}$

$$\hat{z}_0 = \Phi^{-1} \left(\frac{1}{B} \sum_{b=1}^B I(\hat{\theta}_b^* < \hat{\theta}) \right) \quad (20)$$

What is $\hat{z}_0$ if $\hat{\theta}$ is the median of Bootstrap replicates?

- $\hat{a}$: skewness or acceleration correction factor; estimates the rate of change of $se(\hat{\theta})$

$$\hat{a} = \frac{\sum_{i=1}^n (\bar{\theta}_{jack} - \hat{\theta}_{(i)})^3}{6 \{ \sum_{i=1}^n (\bar{\theta}_{jack} - \hat{\theta}_{(i)})^2 \}^{3/2}} \quad (21)$$

where $\hat{\theta}_{(i)}$ is the i th jackknife replicate for $\hat{\theta}$.

- Efron (1987): *Better Bootstrap Confidence Intervals*, JASA, Vol. 82, No. 397, pp.171-185.

Example 7.2 of Rizzo, Continued

```
# Bias correction factor
z0 = qnorm(sum(boot.cor < obs.cor)/length(boot.cor))

# Skewness factor (jackknife est.)
n = nrow(law) # number of observations
cor.jack = numeric(n)
for (i in 1:n) {
  cor.jack[i] = cor(law[-i,])[1,2]
}
L = mean(cor.jack) - cor.jack
ahat = sum(L^3)/(6*sum(L^2)^1.5)

# 95% BCa CI
alpha = .05
zalpha1 = qnorm(alpha/2)
zalpha2 = qnorm(1-alpha/2)
adj.alpha1 = pnorm(z0 + (z0+zalpha1)/(1-ahat*(z0+zalpha1)))
adj.alpha2 = pnorm(z0 + (z0+zalpha2)/(1-ahat*(z0+zalpha2)))

quantile(boot.cor, c(adj.alpha1, adj.alpha2))
```

Example 7.2 of Rizzo, Continued

```
# 95% BCa CI  
> quantile(boot.cor, c(adj.alpha1, adj.alpha2))  
0.7244584% 94.52885%  
0.3577413 0.9446134
```

```
# 95% Percentile Bootstrap CI (Notes #6)  
> c(b0.025,b0.975)  
0.4645733 0.9615963
```

```
# 95% Basic Bootstrap CI (Notes #6)  
> 2*obs.cor.c - c(b0.975, b0.025)  
0.607384 1.104407
```

Outline

- 1 Cross Validation
 - Orthogonal Polynomials
- 2 Jackknife
- 3 Confidence Interval
- 4 More examples on Bootstraps**

Bootstraps for Sample Variance

Consider $\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$ for $n = 40$ and $x_i \sim N(0, 1)$.
See also Notes #6.

```
# Define Statistic
statistic <- function(x){
  mean( (x-mean(x))^2 )
}

# Compute the bootstrap se of a statistic
bootstrap.se = function(x, statistic, B){
  x = as.matrix(x)
  n = nrow(x)
  th = numeric(B)
  for (b in 1:B){
    bootstrap.sample = x[sample(n, replace = T),]
    th[b] = statistic(bootstrap.sample)
  }
  return(sd(th))
}
```

Bootstraps for Sample Variance, continued

```
set.seed(1)
n = 40
mydata = rnorm(n)
(obs.sigest = statistic(mydata))

nboot = 1000
boot.sigest = numeric(nboot)
se = numeric(nboot)

for(i in 1:nboot){
  bootstrap.sample = mydata[sample(n,replace=TRUE)]
  boot.sigest[i] = statistic(bootstrap.sample)
  se[i] = bootstrap.se(bootstrap.sample, statistic, 100)
}
```

Bootstraps for Sample Variance, continued

```
# Estimate the bias
(sigest.bias = mean(boot.sigest) - obs.sigest)
# Bias-Corrected Estimate
(obs.sigest.c = obs.sigest-sigest.bias)

# Bootstrap Standard Error
(sigest.se = sd(boot.sigest))

# N(0,1) Bootstrap Bias-Corrected CI
Boot.StdNorm.CI = obs.sigest.c + c(-1,1)*qnorm(0.975)*sigest.se

# Quantiles of Bootstrap Distribution
b0.025 = quantile(boot.sigest,0.025)
b0.975 = quantile(boot.sigest,0.975)

# Percentile Bootstrap CI
Boot.Percentile.CI = c(b0.025,b0.975)

# Basic Bootstrap CI
Boot.Basic.CI = 2*obs.sigest.c - c(b0.975, b0.025)
```


Bootstraps for Sample Variance, continued

```
# Bootstrap t CI
t.stat = (boot.sigest - obs.sigest)/se
Qt = quantile(t.stat, c(0.05/2, 1-0.05/2))
names(Qt) = rev(names(Qt))
Boot.t.CI = rev(obs.sigest.c - Qt*sigest.se)

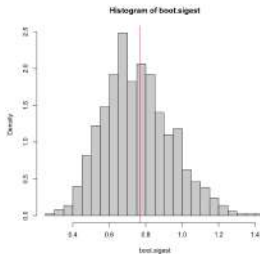
# Bootstrap BCa CI
z0 = qnorm(mean(boot.sigest < obs.sigest))
# Skewness factor (jackknife est.)
stat.jack = numeric(n)
for (i in 1:n) {
  stat.jack[i] = statistic(mydata[-i])
}
L = mean(stat.jack) - stat.jack
ahat = sum(L^3)/(6*sum(L^2)^1.5)

zalpha1 = qnorm(.05/2)
zalpha2 = qnorm(1-.05/2)
adj.alpha1 = pnorm(z0 + (z0+zalpha1)/(1-ahat*(z0+zalpha1)))
adj.alpha2 = pnorm(z0 + (z0+zalpha2)/(1-ahat*(z0+zalpha2)))
Boot.BCa.CI = quantile(boot.sigest, c(adj.alpha1, adj.alpha2))
```

Bootstraps for Sample Variance, continued

```
> hist(boot.sigest, 30, probability = T)
> abline(v=obs.sigest, col='red')
>
> tab.sigest = rbind(Boot.StdNorm.CI, Boot.Percentile.CI,
+                   Boot.Basic.CI, Boot.t.CI, Boot.BCa.CI)
> rownames(tab.sigest) = c("Std Normal", "Percentile", "Basic",
+                          "Studentized", "BCa")
> round(tab.sigest, 4)
```

	2.5%	97.5%
Std Normal	0.4272	1.1251
Percentile	0.4156	1.1100
Basic	0.4424	1.1368
Studentized	0.4694	1.4610
BCa	0.4736	1.1770



When Bootstraps Break

Bootstrap doesn't work for all statistics

- $\hat{\theta}(X_1, \dots, X_n) = \min_{ij} |X_i - X_j|$
- Minimum distance between points in the data set.
- Bootstrap sample: minimum distance is 0 (tied observations)
- Real data – almost never ties for continuous variables.

Most cases of failure $\hat{\theta}(X_1, \dots, X_n)$ is not a smooth function of $X_1, \dots, X_n$ (cannot differentiate with respect to X_i).

When Bootstraps Break

- Wasserman (2005): All of nonparametric statistics, Springer.
- Notes from Rui Castro.
- Let $X_1, \dots, X_n$ be an i.i.d. sample from an unknown distribution F with mean μ and unit variance.
- Let $\theta = |\mu|$
- Define $\hat{\theta} = |\bar{X}_n|$, where $\bar{X}_n = n^{-1} \sum_{j=1}^n X_j$.
- If $\mu = 0$, the Bootstraps fail.

When Bootstraps Break

Define

$$T_n = \sqrt{n}(\hat{\theta} - \theta) = \sqrt{n}(|\bar{X}_n| - |\mu|). \quad (22)$$

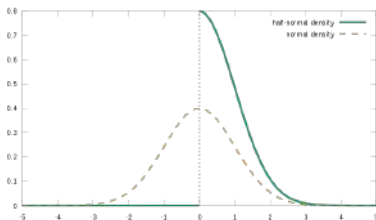
Note that

$$T_n = |\sqrt{n}(\bar{X}_n - \mu) + \sqrt{n}\mu| - |\sqrt{n}\mu| \quad (23)$$

$$\stackrel{D}{\approx} |Z + \sqrt{n}\mu| - |\sqrt{n}\mu| \quad (24)$$

where $Z \sim N(0, 1)$.

■ When $\mu = 0$, $T_n \stackrel{D}{\approx} |Z|$.



When Bootstraps Break

- Consider bootstrap samples of $T_n = \sqrt{n}(|\bar{X}_n| - |\mu|)$:

$$T_n^* = \sqrt{n}(|\bar{X}_n^*| - |\bar{X}_n|) \quad (25)$$

where $\bar{X}_n^* = n^{-1} \sum_{j=1}^n X_j^*$ and $\{X_1^*, \dots, X_n^*\}$ are the bootstrap sample.

- Asymptotic density of T_n :

$$f_{T_n}(x) = \begin{cases} 0 & \text{if } x < -\sqrt{n}|\mu| \\ \phi(x) + \phi(x + 2\sqrt{n}|\mu|) & \text{if } x \geq -\sqrt{n}|\mu| \end{cases} \quad (26)$$

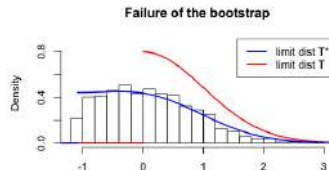
- Density of T_n^* conditional on $X_1, \dots, X_n$ is given by

$$f_{T_n^*}(x) = \begin{cases} 0 & \text{if } x < -\sqrt{n}|\bar{X}_n| \\ \phi(x) + \phi(x + 2\sqrt{n}|\bar{X}_n|) & \text{if } x \geq -\sqrt{n}|\bar{X}_n| \end{cases} \quad (27)$$

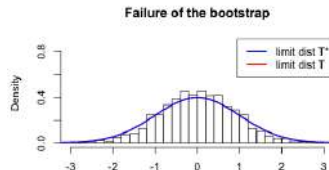
where $\phi(\cdot)$ is pdf of $N(0, 1)$.

When Bootstraps Break

When $\mu = 0$



When $\mu = 1$



When Bootstraps Break

When $\mu = 0$:

```
set.seed(1)
mu = 0
n = 100
x = rnorm(n)+mu
theta0 = abs(mean(x))

nboot = 1000
boot.theta = numeric(nboot)
for(i in 1:nboot){
  bootstrap.sample = x[sample(n,replace=TRUE)]
  boot.theta[i] = abs(mean(bootstrap.sample))
}

T.star = sqrt(n)*(boot.theta-theta0)

hist(T.star, br='FD',prob=T, ylim=c(0,1.1*2*dnorm(0)),
      xlab='', main='Failure of the bootstrap')
```


When Bootstraps Break

```
# Add density of T_n
curve(dnorm(x)+dnorm(x+2*abs(mu)*sqrt(n)), -abs(mu)*sqrt(n), 10,
      add=T, col='red', lwd=2)
curve(0*x, -10, -abs(mu)*sqrt(n), add=T, col='red', lwd=2)

# Add density of T_n*
z1 = sqrt(n)*theta0
curve(dnorm(x)+dnorm(x+2*abs(z1)), -abs(z1), 10, add=T, col='blue', lwd=2)
curve(x*0, -10, -abs(z1), add=T, col='blue', lwd=2)

legend('topright', c('limit dist T*', 'limit dist T'),
      lwd=rep(2, 2), lty=rep(1, 2), col=c('blue', 'red'))
```

Optimization and Root Finding

JMR Chapters 10 and 12

Fangfang Wang
MA 551

The Optimization Problem

We have some function $f(x)$ and we want to find

$$x^* = \operatorname{argmax} f(x) \tag{1}$$

the x that produces the largest value in $f(x)$.

If f is “nice”, we can work this out with algebra and calculus.

Otherwise, we can use a computer to search for the optimum.

Example

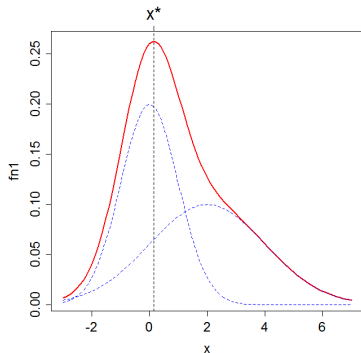
Consider the normal mixture density

$$f(x) = \frac{1}{2\sqrt{2\pi}}e^{-x^2/2} + \frac{1}{4\sqrt{2\pi}}e^{-(x-2)^2/8}$$

X comes from $N(0, 1)$ with probability 0.5 and $N(2, 2)$ with probability 0.5.

What is the mode of $f(x)$?

No algebraic solution available.



The Root-Finding Problem

We might also want to find where $f(x)$ has a specific value

$$x^+ = \{x : f(x) = c\} = \{x : f(x) - c = 0\} \quad (2)$$

Called “root-finding” because we try to find a root of $f(x) - c$.

A Classical Problem

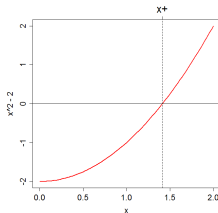
What is the value of $\sqrt{2}$?

- Classical result: no fractional expression $\sqrt{2} \neq a/b$ for all integers a and b .
- Easy expression to work with, but what are the actual digits?

Find a **numerical solution** of the problem

$$x^2 - 2 = 0$$

in $[0, \infty)$.



Reducing Optimization to Root Finding and Vice Versa

- You can always solve a root-finding problem with optimization:

$$g(x^+) - c = 0 \Rightarrow x^+ = \operatorname{argmax} -(g(x) - c)^2$$

- If $f(x)$ is *differentiable*, we can do the converse:

$$x^* = \operatorname{argmax} f(x) \Rightarrow f'(x^*) = 0$$

but you need to check $f(x^*)$ is a maximum.

- In statistics, optimization is most used, but root finding provides useful motivation.

Root-Finding 1: The Bisection Method

- Suppose that $f(x)$ is monotone (increasing or decreasing) on some range $[a \ b]$ and $f(a)f(b) < 0$.
- Then $f(x^+) = 0$ at most one $x^+ \in [a \ b]$.
- We can test whether $f(x)$ crosses zero by the sign of $f(a)f(b)$.

Bisection method searches by successively dividing intervals in two:

- 1 Start with $a < b$ such that $f(a)f(b) < 0$.
- 2 Let $c = (a + b)/2$ be the midpoint of a and b .
- 3 If $f(a)f(c) < 0$, $f(x)$ crosses 0 in $[a \ c]$, set $b = c$.
- 4 Otherwise, $f(x)$ crosses 0 in $[c \ b]$, set $a = c$.
- 5 Repeat.

Graphically and in Code

```
fn2 = function(x){ return(x^2 -2) }
```

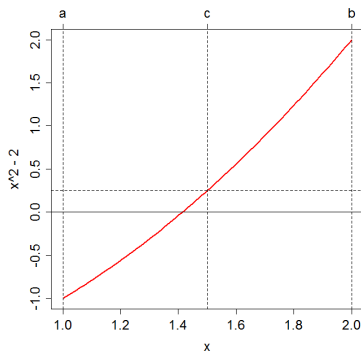
```
a = 1    # Starting values:  
b = 2    # and  $2^2 > 2$ 
```

```
fa = fn2(a); fb = fn2(b)
```

```
for(i in 1:3){  
  c = (a+b)/2; fc = fn2(c)
```

```
  if(fa*fc<0){  
    b = c; fb = fc  
  } else{  
    a = c; fa = fc  
  }  
}
```

```
}
```



Step 1: set $b = c$.

Graphically and in Code

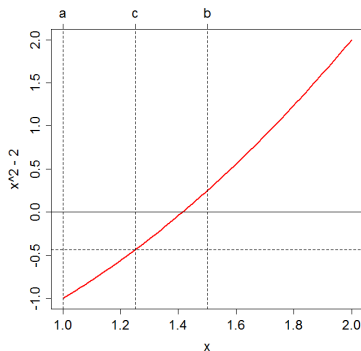
```
fn2 = function(x){ return(x^2 -2) }
```

```
a = 1    # Starting values:  
b = 2    # and  $2^2 > 2$ 
```

```
fa = fn2(a); fb = fn2(b)
```

```
for(i in 1:3){  
  c = (a+b)/2; fc = fn2(c)
```

```
  if(fa*fc<0){  
    b = c; fb = fc  
  } else{  
    a = c; fa = fc  
  }  
}
```



Step 2: set $a = c$.

Graphically and in Code

```
fn2 = function(x){ return(x^2 -2) }
```

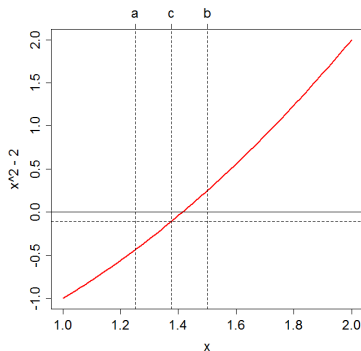
```
a = 1    # Starting values:
```

```
b = 2    # and  $2^2 > 2$ 
```

```
fa = fn2(a); fb = fn2(b)
```

```
for(i in 1:3){  
  c = (a+b)/2; fc = fn2(c)
```

```
  if(fa*fc<0){  
    b = c; fb = fc  
  } else{  
    a = c; fa = fc  
  }  
}
```



Step 3: set $a = c$.

Graphically and in Code

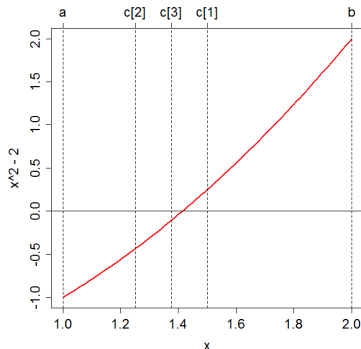
```
fn2 = function(x){ return(x^2 -2) }
```

```
a = 1    # Starting values:
b = 2    # and  $2^2 > 2$ 
```

```
fa = fn2(a); fb = fn2(b)
```

```
for(i in 1:3){
  c = (a+b)/2; fc = fn2(c)
```

```
  if(fa*fc<0){
    b = c; fb = fc
  } else{
    a = c; fa = fc
  }
}
```



Sequence of approximations
to $\sqrt{2}$.

Convergence Criteria

- It's unlikely that we will ever find $f(x) = 0$ exactly.
- So we need some way to decide that our solution is “good enough”.
- Usually decided by a tolerance.
 - For root finding, stop when $|f(c)| < \epsilon$.
 - ϵ chosen based on required accuracy and machine tolerance (default is often around $1e-8$).
- We also usually set a maximum number of iterations, so we know we will terminate sometime.

In Code

```
BisectionSearch = function(fn,a,b,tol=1e-8,maxit=100){  
  fa = fn(a); fb = fn(b);      # Initialization  
  
  tol.met = FALSE      # No tolerance met  
  iter = 0              # No iterations  
  
  while(!tol.met){  
    c = (a+b)/2; fc = fn(c)  
  
    if( fa*fc < 0 ){ b = c; fb = fc }  
    else{ a = c; fa = fc }  
  
    iter = iter + 1 # Update iterations and tolerance  
    if( abs(fc) < tol | iter > maxit ){ tol.met=TRUE }  
  }  
  return(list(sol=c,iter=iter))  
}
```

Output

Including some `print` commands in the function (see also page 189 of JMR):

```
> sol = BisectionSearch(fn2,1,2)
Iter      Value      Convergence
[1] 1       1.5       0.25
[1] 2       1.25     -0.4375
[1] 3       1.375    -0.109375
...
[1] 26      1.4142   -2.63102e-08
[1] 27      1.4142  -5.23681e-09
```

And if we compare this to R's value:

```
> sol$sol
[1] 1.414214
> sqrt(2)
[1] 1.414214
> sqrt(2)-sol$sol
[1] 1.851493e-09
```

Analysis of Convergence of Root Finding Methods

So how large is our error?

- Let $D = b - a$ for the starting guess.
- If we use $\hat{x} = (a + b)/2$, we know that $|\hat{x} - x^+|$ is at most $D/2$.
- Next iteration, we halve the size of the interval again.
- After k steps, the error can be at most $2^{-k}D = O(2^{-k})$.
- In this case, we can control error by controlling the number of iterations.

Properties (unlike Newton-Raphson, next)

- Doesn't require derivatives.
- Very simple to implement.
- Explicit convergence error from number of steps.
- **But** slow and doesn't generalize to more dimensions.

Root-Finding 2: Newton-Raphson

Suppose that $f(x)$ has a derivative.

- Start at initial guess x_0 and take a linear approximation

$$f(x) \approx f(x_0) + (x - x_0)f'(x_0) \quad (3)$$

- Now we can set the linear approximation to 0

$$f(x_0) + (x - x_0)f'(x_0) = 0 \Rightarrow x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

- Now set x_1 to be our guess and start again. $x_2 = x_1 - \frac{f(x_1)}{f'(x_1)}$

As before, stop when $|f(x_n)| < \epsilon$ or too many iterations.

Graphically and in Code

We first need to define a derivative

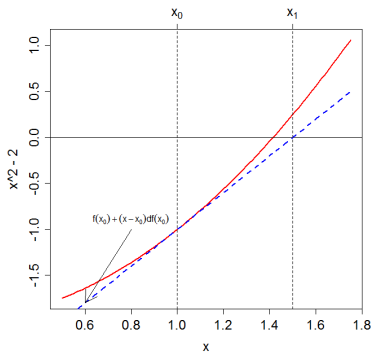
$$\frac{d}{dx} (x^2 - 2) = 2x$$

```
dfn2 = function(x){ return( 2*x ) }
```

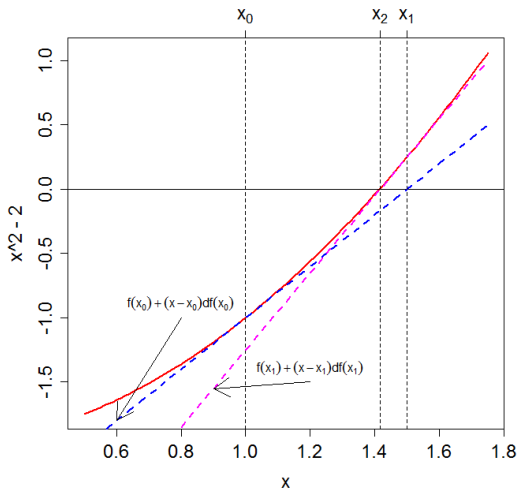
Start at 1 as an initial guess

```
x0 = 1
f0 = fn2(x0)
df0 = dfn2(x0)

x1 = x0 - f0/df0
```



The Next Iteration



And continue.

A Formal Function

```
NewtonRaphson = function(fn,dfn,x0,tol=1e-8,maxit=100){  
  f0 = fn(x0); df0 = dfn(x0);      # Initialization  
  
  tol.met = FALSE      # No tolerance met  
  iter = 0              # No iterations  
  
  while(!tol.met){  
    x0 = x0 - f0/df0  
    f0 = fn(x0); df0 = dfn(x0)  
  
    iter = iter + 1 # Update iterations  
    if( abs(f0)<tol | iter>maxit ){ tol.met=TRUE }  
  }  
  return(list(sol=x0,iter=iter))  
}
```

Validation

Very fast convergence.

```
> sol = NewtonRaphson(fn2,dfn2,1)
```

```
Iter      Estimate Convergence
```

```
[1] 1      1.5      0.25
```

```
[1] 2      1.41667 0.00694
```

```
[1] 3      1.41422 6.007e-06
```

```
[1] 4      1.41421 4.511e-12
```

```
> sol$sol
```

```
[1] 1.414214
```

```
> sqrt(2)
```

```
[1] 1.414214
```

```
> sqrt(2)-sol$sol
```

```
[1] -1.594724e-12
```

Convergence Analysis

A bit of mathematics:

Write a Taylor-series approximation to $f(x)$ near x_n as

$$f(x) = f(x_n) + (x - x_n)f'(x_n) + R_1 \quad (4)$$

where the remainder R_1 is

$$R_1 = \frac{1}{2}(x - x_n)^2 f''(\tilde{x}) \quad (5)$$

for some $\tilde{x}$ between x and x_n .

Now let's look at this approximation at the root x^+

$$0 = f(x^+) = f(x_n) + (x^+ - x_n)f'(x_n) + \frac{1}{2}(x^+ - x_n)^2 f''(\tilde{x}) \quad (6)$$

Convergence Analysis

$$0 = f(x_n) + (x^+ - x_n)f'(x_n) + \frac{1}{2}(x^+ - x_n)^2 f''(\tilde{x}) \quad (7)$$

Re-arrange this and divide by $f'(x_n)$:

$$x^+ - x_n + \frac{f(x_n)}{f'(x_n)} = -\frac{f''(\tilde{x})}{2f'(x_n)}(x^+ - x_n)^2 \quad (8)$$

Note that $x_{n+1} = x_n - f(x_n)/f'(x_n)$. So

$$|x^+ - x_{n+1}| = \left| \frac{f''(\tilde{x})}{2f'(x_n)} \right| (x^+ - x_n)^2 \quad (9)$$

- Error $\epsilon_n = x^+ - x_n$ is squared each iteration (bisection search just halves it).
- But only works if $f''(\tilde{x})/f'(x_n)$ stays small *and* we start close to x^+ .

Let $\left| \frac{f''(\tilde{x})}{2f'(x_n)} \right| \leq C$, then $|\epsilon_n| \leq C^{2^n-1} |\epsilon_0|^{2^n}$.

Convergence Issues

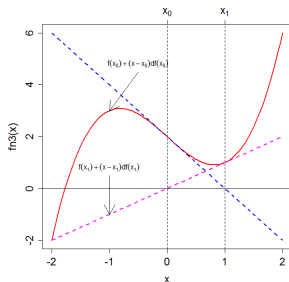
Newton-Raphson can fail in a number of ways:

- Function not smooth enough (needs second derivatives to be fast).
- $f'(x)$ is zero at the root.
- Infinite cycles.

Consider $f(x) = x^3 - 2x + 2$,
 $f'(x) = 3x^2 - 2$:

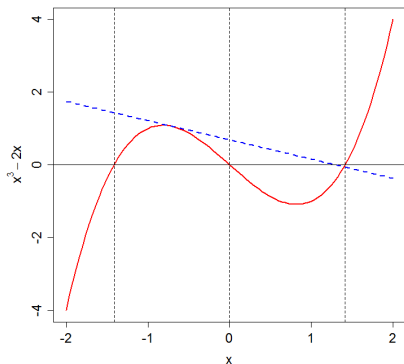
- Start $x_0 = 0$,
 $x_1 = 0 - 2/(-2) = 1$.
- $x_2 = 1 - 1/1 = 0 = x_0$

Never ends!



What If $f(x)$ Crosses 0 Multiple Times?

- Optimizers will give you one of the zeros.
- Newton-Raphson usually gives you the root closest to where you started.
- But not always!
- Common strategy: try starting from multiple places.
- Bisection search harder to analyze in this case.



Secant Method

Calculating derivatives can sometimes be inconvenient (and users often get them wrong).

Instead, use two initial guesses x_0 and x_1 .

- Draw a line through $(x_0, f(x_0))$ and $(x_1, f(x_1))$, secant of the curve $y = f(x)$:

$$g(x) = f(x_1) + (x - x_1) \frac{f(x_0) - f(x_1)}{x_0 - x_1} \quad (10)$$

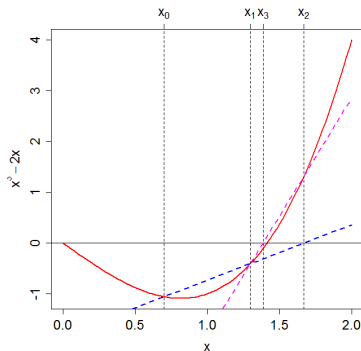
- Find the point where this crosses zero, i.e., $g(x) = 0$

$$x_2 = x_1 - f(x_1) \frac{x_0 - x_1}{f(x_0) - f(x_1)} \quad (11)$$

- Iterate.

Graphically

- Same convergence properties as Newton-Raphson.
- Same problems with local solutions, infinite loops.
- Still requires smoothness.
- Trade-off need for $f'(x)$ with extra calculation and two starting points.



Optimization 1: Newton-Raphson

More frequently (in statistics) we want to optimize.

Newton-Raphson for Optimization: (usually just “Newton’s Method”)

- To find the maximum of $f(x)$, look for $f'(x) = 0$.
 - Stationary point: local max, local min, saddle point
- Use $f'(x)$ in place of $f(x)$ in algorithms above.
- Iteration changes to

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

- Need a maximum: check $f''(x_n) < 0$ (or conversely for a minimum).
- If at the wrong sort of stationary point, try again.

The Mode of a Mixture Distribution

$$f(x) = \frac{1}{2\sqrt{2\pi}}e^{-x^2/2} + \frac{1}{4\sqrt{2\pi}}e^{-(x-2)^2/8}$$

$$f'(x) = -\frac{x}{2\sqrt{2\pi}}e^{-x^2/2} - \frac{(x-2)}{16\sqrt{2\pi}}e^{-(x-2)^2/8}$$

$$f''(x) = \frac{x^2-1}{2\sqrt{2\pi}}e^{-x^2/2} + \frac{\frac{(x-2)^2}{4}-1}{16\sqrt{2\pi}}e^{-(x-2)^2/8}$$

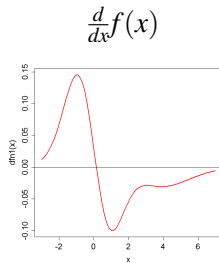
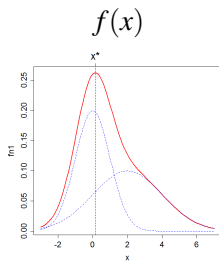
Expressed in terms of the normal density:

```
fn1 = function(x){
  return( 0.5*dnorm(x) + 0.5*dnorm(x,sd=2,mean=2) )
}

dfn1 = function(x){
  return( -x*dnorm(x)/2 - (x-2)*dnorm(x,mean=2,sd=2)/8 )
}

d2fn1 = function(x){
  return( (x^2-1)*dnorm(x)/2 + ((x-2)^2/4-1)*dnorm(x,mean=2,sd=2)/8 )
}
```

The Usual Problems Occur



Newton-Raphson will only converge close to the truth.

```
> est = NewtonRaphson(dfn1,d2fn1,0)
[1] 1 0.1516 0.00017
[1] 2 0.1525 2.701e-08
[1] 3 0.1525 7.008e-16
```

```
> est = NewtonRaphson(dfn1,d2fn1,2)
[1] 1 2.9632 -0.028714
[1] 2 16.098 -5.71e-12
```

$$f'(x) \rightarrow 0 \text{ as } |x| \rightarrow \infty$$

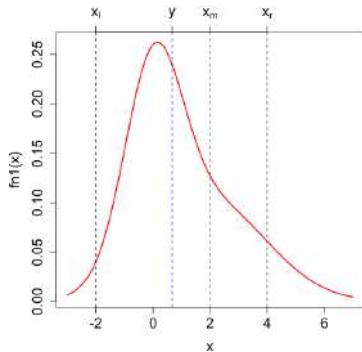
Optimization 2: Golden Section Search

Analogue to bisection search for zeros. Do not want to require derivatives.

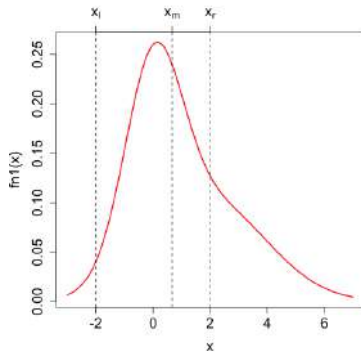
- Begin with left point x_l , right point x_r and middle x_m .
- Suppose we have $x_l < x_m < x_r$ such that $f(x_m) > f(x_l)$ and $f(x_m) > f(x_r)$.
- Choose a new point y in the larger of $[x_l, x_m]$ and $[x_m, x_r]$.
- Suppose $y \in [x_l, x_m]$,
 - If $f(y) > f(x_m)$ set $x_r = x_m$ and $x_m = y$.
 - Otherwise set $x_l = y$.
- Conversely for $y \in [x_m, x_r]$.

Graphical Golden Section

Starting Point



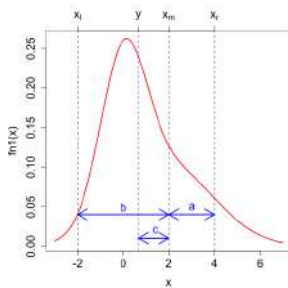
New Configuration



- Unimodality crucial – allows us to conclude where maximum *must* lie.
- y in largest interval = most efficient exploration.

The Golden Section: Choosing y

- Place y so that we always reduce the interval by the same amount.
- Choose the initial interval so that the ratio of larger to smaller interval doesn't change.
- Shift x_r to x_m ; ratio is $(b - c)/c$.
- Shift x_l to y ; ratio is a/c .
- Both should be equal to $b/a = \rho$.



$$\frac{a}{c} = \frac{b}{a} \rightarrow c = \frac{a^2}{b} \text{ substitute in } \frac{b-c}{c} = \frac{b}{a} \text{ yields } \rho^2 - 1 = \rho$$

The Golden Section

$$\rho^2 - \rho - 1 = 0$$

is solved for

$$\rho = \frac{1 + \sqrt{5}}{2} :$$

the “Golden ratio”.

- To work out how large c is, note that $a = b - c$ (we shrink the same amount in either case).
- $\frac{c}{b} = 1 - \frac{a}{b} \Rightarrow c = \frac{b}{1+\rho}$ (algebra is a bit involved).
- Or $y = x_m - \frac{x_m - x_l}{1+\rho}$.

Pseudo-Code

Start with: x_l, x_r, x_m , satisfying $f(x_m) > f(x_l)$ and $f(x_m) > f(x_r)$.

Repeat:

- 1 If $x_r - x_m > x_m - x_l$, $y = x_m + (x_r - x_m)/(1 + \rho)$
 - If $f(y) > f(x_m)$, set $x_l = x_m$ and $x_m = y$.
 - Else set $x_r = y$
- 2 Else $y = x_m - (x_m - x_l)/(1 + \rho)$
 - If $f(y) > f(x_m)$, set $x_r = x_m$ and $x_m = y$.
 - Else set $x_l = y$

until $x_r - x_l < \epsilon$ or too many iterations.

In practice, just update $f(x_l), f(x_m), f(x_r)$ from $f(y)$ or $f(x_m)$ as appropriate to avoid re-evaluating.

See [gsection.r](#).

Some Notes

- Tolerance criteria can be any of

$$|x_{n+1} - x_n| < \epsilon, \quad |f(x_{n+1}) - f(x_n)| < \epsilon,$$

Step size in x , improvement in $f(x)$ (and maximum iterations as backup)

None guarantee $|x_n - x^*| < \epsilon$; often require all to be met.

- Convergence is local – with multiple maxima in a function, each of these will find just one.

Why?

Optimization has multiple scientific uses.

In statistics; most important is maximum likelihood estimation.

$X_1, \dots, X_n \sim f(x; \theta)$; estimate θ .

Choose θ to make $X_1, \dots, X_n$ *most probable*.

$$\hat{\theta} = \operatorname{argmax} P(X_1, \dots, X_n | \theta) = \operatorname{argmax} \prod_{i=1}^n f(X_i | \theta)$$

is the *maximum likelihood estimator*.

Usually work with the log probability

$$\hat{\theta} = \operatorname{argmax} \sum_{i=1}^n \log f(X_i | \theta)$$

Sometimes calculable analytically, but not always.

Exercise 12.8.2 of JMR, Page 239

Use the golden-section search algorithm to find all of the local maxima of the function $f(x) = |x| \log(|x|/2) \exp(-|x|)$ for $x \neq 0$ and $f(0) = 0$, within the interval $[-10, 10]$.

```
fn = function(x){
  if (x==0){fn=0}
  else{
    fn = abs(x)*log(abs(x)/2)*exp(-abs(x))
  }
  return(fn)
}

xx = seq(-10,10,0.1);

fnx = rep(0,length(xx))
for (j in 1:length(xx)){
  fnx[j] = fn(xx[j])
}
plot(xx, fnx,type='l')

source('gsection.r')
gsection(fn, -0.5,0.25,0.2)
gsection(fn, 3,5.25,5)
```

Summary

- Root finding and optimization as crucial numerical tools.
- Can be converted into each other.
- Locally, Newton-Raphson methods make convergence towards the truth very fast.
- Bisection/Golden Section methods don't require derivatives.
- You always run the risk of not converging, or only finding a local optimum.

Multivariate Optimization

JMR Chapter 12

Fangfang Wang
MA 551

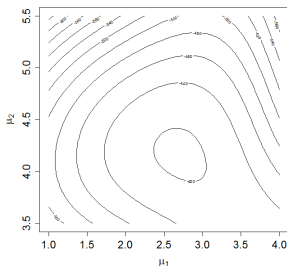
Multivariate Optimization

- We have seen Golden Section and Newton-Raphson searches for one-dimensional optima.
- It is rarely the case that we only want to optimize one thing.
- So we need some strategies for working in multiple dimensions.

Example: means of a normal mixture:

$$\sum_{i=1}^n \log \frac{1}{2} (\phi(X_i - \mu_1) + \phi(X_i - \mu_2))$$

where $\phi(x)$ is the normal density.



On Maximum Likelihood Estimation

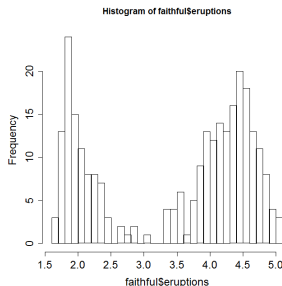
General principle of obtaining parameter estimates

Choose the parameters that make the data as probable as possible.

In the case of the old-faithful data, it looks like we could use two normal distributions

$$P(X) = \frac{1}{2}(\phi(X - \mu_1) + \phi(X - \mu_2))$$

with centers around 2 and 4.5. (Probably smaller standard deviations, too, but lets keep it simple for now).



But we'd like to use the X_i to decide on μ_1 and μ_2 .

Log Likelihood

Probability distribution for any X is

$$P(X) = \frac{1}{2}(\phi(X - \mu_1) + \phi(X - \mu_2)) \quad (1)$$

so that the probability of $X_1, \dots, X_n$ is $\prod_{i=1}^n P(X_i)$.

Usually more convenient to work with log probability as a function of parameters μ_1 and μ_2 (doesn't change where the maximum is).

$$\ell(\mu_1, \mu_2) = \sum_{i=1}^n \log P(X_i) = \sum_{i=1}^n \log \frac{1}{2}(\phi(X_i - \mu_1) + \phi(X_i - \mu_2)) \quad (2)$$

This is the **log likelihood** that we want to maximize for μ_1 and μ_2 .

Note: we could also fit standard deviations, proportions for each normal, but that won't plot so well.

Co-ordinate Ascent

Just optimize one parameter at a time.

- 1 Start with $\mu_{(0)} = (\mu_{0,1}, \mu_{0,2})$
- 2 Run an optimizer to choose a new μ_1 keeping μ_2 fixed.

$$\mu_{1,1} = \operatorname{argmax}_{\mu_1} \ell(\mu_1, \mu_{0,2})$$

- 3 Run an optimizer to choose μ_2 keeping μ_1 fixed at its current value

$$\mu_{1,2} = \operatorname{argmax}_{\mu_2} \ell(\mu_{1,1}, \mu_2)$$

Then $\mu_{(1)} = (\mu_{1,1}, \mu_{1,2})$.

- 4 Iterate until the updates do not move the solution much.

Some Coding Notes

- Use, say, `gsection` from Notes #8 to do the 1d optimization.

```
gsection <- function(ftn, x.l, x.r, x.m, tol=1e-9) { ...}
```

- But this assumes a function with only one input.
- We'll re-write this so the function can take a vector of inputs `mu` (in this case (μ_1, μ_2)).
- It also needs a dimension (`dim`) to tell it which element to work on.
- And it still needs upper and lower starting values for `mu[dim]`.
- We've also got data `x` to add to the function, so we need to add an extra argument throughout.
- This is fairly specialized to the problem; similar things apply to Newton's method functions (but we also need vectors of derivatives).

In Code

```
CoordinateAscent = function(mu,mul,mur,fn,X,tol=1e-8,maxit=1000)
{
  iter = 0                # Initialization
  tol.met = FALSE
  muhist = c()

  while(!tol.met){ # Tolerance will be checked by how much we
    oldmu = mu      # move mu over one cycle

    for(dim in 1:length(mu)){ # But we'll update the history
      iter = iter + 1        # at each coordinate
      mu = GoldenSection(fn,mul,mur,mu,dim,X)$xm
      muhist = rbind(muhist,mu)
    }

    if(max(abs(mu - oldmu))<tol | iter>maxit){tol.met = TRUE}
    else{ oldmu = mu }
  }

  return(list(mu=mu, iter=iter, muhist = muhist))
}
```

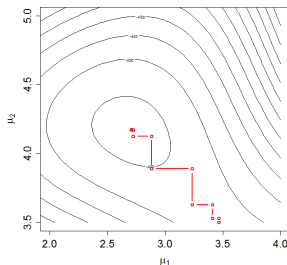
Results

We need to re-define our function to take a vector (μ_1, μ_2):

```
fn = function(mu,X) {  
  return(sum(log(0.5*(dnorm(X-mu[1])+dnorm(X-mu[2])))))  
}
```

And define a box of values containing the maximum

```
mul = c(1,3.5) # lower bounds  
mur = c(4,5.5) # upper bounds  
mu = c(3.6,3.5) # start
```



```
opt = CoordinateAscent(mu,mul,mur,fn,X)
```

Some Notes

- Using `GoldenSection` requires a box for the optimal value as well as a starting point.
- Alternative: `NewtonRaphson` could be used at each 1-dimensional step (but would also require modification).
- One co-ordinate at a time = very slow convergence (long set of zig-zag lines).
- Convergence criterion: $\|\mu_{(k+1)} - \mu_{(k)}\|$.

Steepest Ascent

Perhaps we can work out a better direction to move in?

- We'd like to move uphill as quickly as possible.
- Let $f : \mathbb{R}^p \rightarrow \mathbb{R}$ be a function with continuous partial derivatives everywhere.
- Let $\mathbf{x} = (x_1, \dots, x_p)' \in \mathbb{R}^p$.
- Define the gradient $\nabla f(\mathbf{x}) = (df/dx_1, \dots, df/dx_p)'$ – this is the direction that f changes fastest.
- So consider an update of the form $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \nabla f(\mathbf{x}_k)$ for some α .
- To choose α : 1-d optimization on the function

$$f^*(\alpha) = f(\mathbf{x}_k + \alpha \nabla f(\mathbf{x}_k))$$

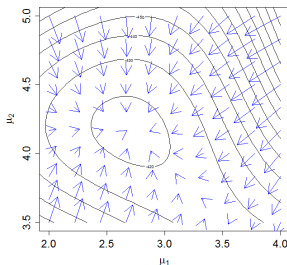
Our Example

We need derivatives of the log likelihood

$$\frac{d\ell(\mu_1, \mu_2)}{d\mu_1} = \sum_{i=1}^n \frac{(X_i - \mu_1)\phi(X_i - \mu_1)}{\phi(X_i - \mu_1) + \phi(X_i - \mu_2)}$$

$$\frac{d\ell(\mu_1, \mu_2)}{d\mu_2} = \sum_{i=1}^n \frac{(X_i - \mu_2)\phi(X_i - \mu_2)}{\phi(X_i - \mu_1) + \phi(X_i - \mu_2)}$$

Visualized as arrows:



Steepest Ascent Continued

Three-step method: at iteration k with current estimate $\mathbf{x}_k$

- 1 Calculate the gradient $\mathbf{g}_k = \nabla f(\mathbf{x}_k)$.
- 2 Find the α_k that maximizes $f^*(\alpha) = f(\mathbf{x}_k + \alpha \mathbf{g}_k)$.
 - Line search.
- 3 Set $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{g}_k$.

Line search: any one-dimensional optimization

- Requires either specialized code (as in [GoldenSection](#) above), or defining a function within a function (see example later).
- Should guarantee that $f(\mathbf{x}_{k+1}) > f(\mathbf{x}_k)$.
- Known starting point, but no known limits.

GoldenSection Line Search

Consider a golden section search over $f^*(\alpha) = f(\mathbf{x}_k + \alpha \mathbf{g}_k)$.

- We know that $f(\mathbf{x}_k)$ is increasing in the direction of $\mathbf{g}_k$, so we can require $\alpha > 0$. That is, the left end point is $a_l = 0$.
- Right end point is more tricky; we want to be past the top of the 'bump' in the line search.
- One strategy:
 - Start with a_r small (say, some multiples of tolerance).
 - Keep doubling a_r until $f^*(a_r)$ decreases. The maximum must be between the last three values.
 - To prevent going on forever, set a maximum place to stop.

Things are actually not so complicated with a Newton-Raphson method, but you need more derivatives.

Line Search Function

```

GoldenSectionLineSearch = function(fns,X,tol,maxtry)
{
  # We need to work out where to put our limits for the Golden
  # Section search.

  ar = c(0,2*tol,4*tol)                #Start near tolerance
  fval = c(fns(0,X),fns(2*tol,X),fns(4*tol,X))

  try = 3                               #Keep doubling
  while( fval[try]>fval[try-1] & try < maxtry ){ #the end point
    ar = c(ar,2*ar[try])
    try = try+1
    fval = c(fval,fns(ar[try],X))
  }
  if(try == maxtry){al=ar[try-1];ar=ar[try];am=ar} #Last two points
  else{al=ar[try-2];ar=ar[try];am=ar[try-1]} #if not at a hump,
                                              #otherwise last three.

  # Now call GoldenSection for the line search

  return( GoldenSection(fns,al,ar,am,1,X,tol=tol,maxit=maxit) )
}

```

Putting It Together

```
SteepestAscent = function(mu,fn,dfn,X,tol=1e-8,maxit=1000,maxtry=25)
{
  tol.met=FALSE; iter = 0; iterhist = mu

  while(!tol.met){
    iter = iter+1; oldmu = mu; g = dfn(mu,X)

    # Define a new function -- I'm using the fact that mu and g are
    # going to be taken from the StepestAscent workspace
    # when fns is called.
    fns = function(alpha,X){ return(fn(mu+alpha*g,X)) }

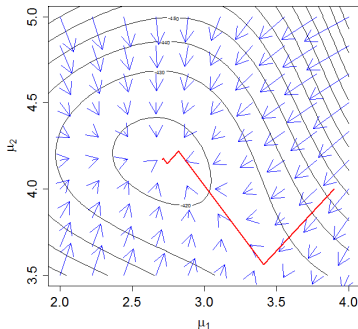
    # Conduct the line search
    linesearch = GoldenSectionLineSearch(fns,X,tol,maxtry)

    # And update the estimate
    mu = mu + linesearch$xm*g
    iterhist = rbind(iterhist,mu)

    if( max(abs( mu-oldmu )) < tol | iter > maxit){ tol.met=TRUE }
  }
  return(list(mu=mu,iter=iter,iterhist=iterhist))
}
```

Some Notes

- More direct than coordinate ascent.
- Still guaranteed to increase $f(\mathbf{x}_k)$ at each step.
- Still some zig-zagging across ridges.



Multivariate Newton-Raphson

More derivatives can help. Let $f : \mathbb{R}^p \rightarrow \mathbb{R}$. Take a Taylor expansion in p dimensions. For $\mathbf{x}, \mathbf{x}^* \in \mathbb{R}^p$,

$$f(\mathbf{x}) \approx f(\mathbf{x}^*) + (\mathbf{x} - \mathbf{x}^*)^T \nabla f(\mathbf{x}^*) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^*)^T H(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*) \quad (3)$$

where $H(\mathbf{x}^*)$ is the *Hessian* at $\mathbf{x}^*$

$$H(\mathbf{x}^*) = \begin{bmatrix} \frac{d^2 f(\mathbf{x}^*)}{dx_1 dx_1} & \cdots & \frac{d^2 f(\mathbf{x}^*)}{dx_1 dx_p} \\ \vdots & \ddots & \vdots \\ \frac{d^2 f(\mathbf{x}^*)}{dx_p dx_1} & \cdots & \frac{d^2 f(\mathbf{x}^*)}{dx_p dx_p} \end{bmatrix}. \quad (4)$$

Try to maximize the quadratic approximation to f .

Multivariate Newton-Raphson

Want to maximize the approximation

$$\tilde{f}(\mathbf{x}) = f(\mathbf{x}^*) + (\mathbf{x} - \mathbf{x}^*)^T \nabla f(\mathbf{x}^*) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^*)^T H(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*). \quad (5)$$

Setting the gradient to zero

$$\nabla \tilde{f}(\mathbf{x}) = \nabla f(\mathbf{x}^*) + H(\mathbf{x}^*)(\mathbf{x} - \mathbf{x}^*) = 0, \quad (6)$$

or

$$\mathbf{x} = \mathbf{x}^* - H(\mathbf{x}^*)^{-1} \nabla f(\mathbf{x}^*). \quad (7)$$

This gives the Newton-Raphson update

$$\mathbf{x}_{k+1} = \mathbf{x}_k - H(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k), \quad (8)$$

(with k now meaning iteration rather than dimension)

1-d Newton-Raphson obtained exactly the same way.

A First Multivariate Newton-Raphson Function

```
NewtonRaphson2 = function(mu,dfn,d2fn,X,tol=1e-8,maxit=100)
{
  tol.met=FALSE; iter = 0; iterhist = mu

  while(!tol.met){
    iter = iter + 1
    oldmu = mu

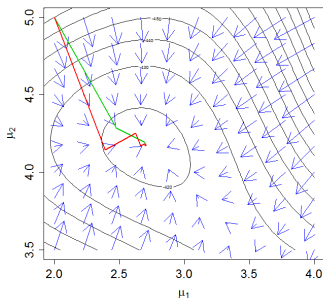
    g = dfn(mu,X)           # Gradient
    H = d2fn(mu,X)          # Hessian
    mu = mu - solve(H,g)     # Update
    iterhist = rbind(iterhist,mu)

    if( (max(abs( mu-oldmu ))<tol & max(abs(g))<tol) | iter>maxit)
      { tol.met=TRUE }
  }
  return(list(mu=mu,iter=iter,iterhist=iterhist))
}
```

Convergence: requires both last step *and* the gradient to be sufficiently small.

Some Notes and Results

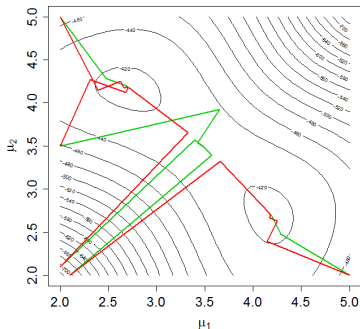
- Nominally very fast convergence.
- But no guarantee that you will find a maximum instead of a minimum (or a saddle).
- Can have problems with singular Hessians.



Green = Newton-Raphson,
Red = Steepest Ascent.

Multi-Modal Objectives

When there are multiple local minima, you end up in one of them.



Newton-Raphson can end up in a saddle point, too.

Usually, you start your optimization in multiple places (often chosen at random) and pick the best end-point.

Least Squares and Gauss-Newton Methods

Regression is one of the most commonly used statistical methods.

Instead of a linear regression, what if we had some non-linear function we wanted to fit to data?

Eg: the Michaelis-Menten model

$$Y_i = \frac{\theta_1 x_i}{\theta_2 + x_i} + \epsilon_i. \quad (9)$$

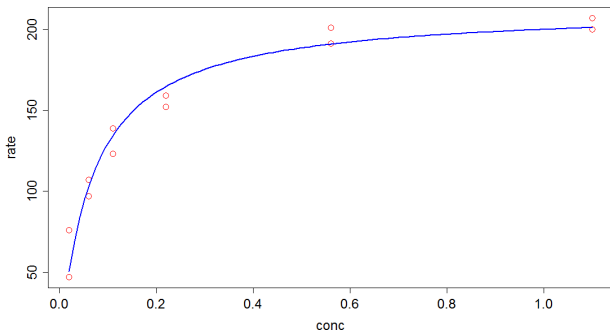
Estimate θ_1 and θ_2 by minimizing squared error; i.e., maximizing

$$-\frac{1}{2} \sum_{i=1}^n \left(Y_i - \frac{\theta_1 x_i}{\theta_2 + x_i} \right)^2. \quad (10)$$

More generally, we model $Y_i = f(x_i, \theta) + \epsilon_i$.

Example: Puromycin Data

Data from an enzymatic reaction that saturates as the concentration of Puromycin increases.



Blue line gives model of increased response subject to saturation.

A Bit of Notation

Using the model $Y_i = f(x_i, \theta) + \epsilon_i$, $i = 1, 2, \dots, n$, with vector θ of interest.

In vector form

$$Y_{n \times 1} = f(X, \theta) + E \quad (11)$$

and in particular we will want to look at the matrix

$$J(X, \theta) = \frac{df(X, \theta)}{d\theta} \quad (12)$$

where the entries are

$$J(X, \theta)_{ij} = \frac{df(x_i, \theta)}{d\theta_j} \quad (13)$$

rows = observations, columns = elements of θ .

Gauss-Newton Methods

We have the objective function

$$F(\theta) = -\frac{1}{2} \sum_{i=1}^n (Y_i - f(x_i, \theta))^2 \quad (14)$$

with gradient

$$\nabla F(\theta) = \sum_{i=1}^n \frac{df(x_i, \theta)}{d\theta} (Y_i - f(x_i, \theta)) = J(X, \theta)^T (Y - f(X, \theta)) \quad (15)$$

and Hessian

$$H(\theta) = - \sum_{i=1}^n \frac{df(x_i, \theta)}{d\theta} \frac{df(x_i, \theta)}{d\theta}^T - \sum_{i=1}^n \frac{d^2 f(x_i, \theta)}{d\theta d\theta^T} (Y_i - f(x_i, \theta)) \quad (16)$$

$$\approx -J(X, \theta)^T J(X, \theta), \quad (17)$$

because second term should be small.

Gauss-Newton Iteration

$$\theta_{k+1} = \theta_k + \left[J(\theta_k, X)^T J(\theta_k, X) \right]^{-1} J(\theta_k, X)^T (Y - f(\theta_k, X))$$

```
GaussNewton = function(theta,fn,dfn,Y,x,tol=1e-8,maxit=100)
{
  tol.met=FALSE; iter = 0; iterhist = theta

  while(!tol.met){
    iter = iter + 1
    oldtheta = theta

    f = fn(theta,x)
    g = dfn(theta,x)

    theta = theta + solve(t(g)%*%g,t(g)%*%(Y-f))
    iterhist = rbind(iterhist,theta)

    if(max(abs( theta-oldtheta )) < tol | iter > maxit)
      { tol.met=TRUE }
  }
  return(list(theta=theta,iter=iter,iterhist=iterhist))
}
```

Setting Up Puromycin

Need functions that return vector $f(X, \theta)$

```
mmfn = function(theta,x){ return( theta[1]*x/(theta[2]+x) ) }
```

and matrix $J(X, \theta)$

```
dmmfn = function(theta,x){  
  return( cbind(x/(theta[2]+x), -theta[1]*x/(theta[2]+x)^2) )  
}
```

We can then call

```
opt = GaussNewton(c(205,0.08),mmfn,dmmfn,Puromycin[,2],Puromycin[,1])
```

Note that $J(\theta_k, X)^T J(\theta_k, X)$ is always positive definite;
Gauss-Newton tends to have fewer convergence problems than
standard Newton-Raphson (but not none).

Levenberg-Marquardt

- We have seen that Newton's method works very well close to the optimum.
- But it can get stuck in local minima instead of maxima, or in saddle points.
- On the other hand, steepest-ascent always increases the objective function; but can converge slowly.
- Perhaps we find a way to trade these off.

A Trade-Off

Recall the Newton update

$$\mathbf{x}_{k+1} = \mathbf{x}_k - H(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k). \quad (18)$$

We'll modify this by adding a diagonal element of $H(\mathbf{x}_k)$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - (H(\mathbf{x}_k) - \lambda_k I)^{-1} \nabla f(\mathbf{x}_k). \quad (19)$$

- When $\lambda_k \approx 0$, we get Newton's method.
- For λ_k very large, our step is

$$\mathbf{x}_{k+1} \approx \mathbf{x}_k + \frac{1}{\lambda_k} \nabla f(\mathbf{x}_k) \quad (20)$$

the steepest ascent direction.

Choosing λ_k

Some things to keep in mind

- We want to end up in a maximum.
 - $H(\mathbf{x}_k) - \lambda_k I$ should be negative definite; i.e., we want all the eigenvalues of $H(\mathbf{x}_k) - \lambda_k I$ to be negative.
 - Note that if $\{e_1, \dots, e_p\}$ are the eigenvalues of $H(\mathbf{x}_k)$, the eigenvalues of $H(\mathbf{x}_k) - \lambda_k I$ are $\{e_1 - \lambda_k, \dots, e_p - \lambda_k\}$.
 - Make $\lambda_k > \text{largest positive } e_j$.
- We also want to make sure that $f(\mathbf{x}_k)$ increases at each iteration.
- In practice, taking the Eigen-decomposition of $H(\mathbf{x}_k)$ is computationally expensive and unnecessary.
- Instead, we will use rules to increase or decrease λ_k at each iteration.

Levenberg-Marquardt Pseudo Code

- Choose tolerances $\epsilon_1, \epsilon_2, \epsilon_3$, maximum iterations.
- Start at $\mathbf{x}_0$, calculate $f(\mathbf{x}_0)$, $\nabla f(\mathbf{x}_0)$, $H(\mathbf{x}_0)$.
- Set λ_0 to be larger than the maximum eigenvalue of $H(\mathbf{x}_0)$ plus some tolerance (make it near zero if $H(\mathbf{x}_0)$ is neg. def).
- While $\|\mathbf{x}_{k+1} - \mathbf{x}_k\| > \epsilon_1$, $\|\nabla f(\mathbf{x}_{k+1})\| > \epsilon_2$,
 $f(\mathbf{x}_{k+1}) - f(\mathbf{x}_k) > \epsilon_3$, $k < \text{maxit}$
 - 1 $\tilde{\mathbf{x}}_{k+1} = \mathbf{x}_k - (H(\mathbf{x}_k) - \lambda_k I)^{-1} \nabla f(\mathbf{x}_k)$.
 - 2 If $f(\tilde{\mathbf{x}}_{k+1}) > f(\mathbf{x}_k)$, set $\mathbf{x}_{k+1} = \tilde{\mathbf{x}}_{k+1}$, $\lambda_{k+1} = \lambda_k/2$.
 - 3 Else, while $f(\tilde{\mathbf{x}}_{k+1}) \leq f(\mathbf{x}_k)$
 - $\lambda_k = 2\lambda_k$, $\tilde{\mathbf{x}}_{k+1} = \mathbf{x}_k - (H(\mathbf{x}_k) - \lambda_k I)^{-1} \nabla f(\mathbf{x}_k)$

Set $\mathbf{x}_{k+1} = \tilde{\mathbf{x}}_{k+1}$, $\lambda_{k+1} = \lambda_k$.

Idea: keep increasing λ_k until we get an increase in $f(\mathbf{x}_k)$; otherwise try to decrease to get back to Newton's method.

Some Notes

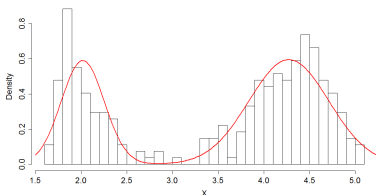
- Levenberg-Marquardt avoids steepest-ascent's wiggles.
- Levenberg-Marquardt, gradient-ascent algorithm, and Newton-Raphson could get stuck at a saddle point.
- Algorithm that can escape from saddle points? Project!

Other Methods: `optim`

- Optimization is a large and ongoing field of research.
- Multiple optimization tools in various R packages, specialized to different purposes.
- Some general-purpose tools are available in the `optim` function: among the possible `method` options:
 - `BFGS`, `L-BFGS-B`, `CG` – like Newton-Raphson; details differ about how Hessians are calculated.
 - `Nelder-Mead` only uses function values (see next slide).
 - `SANN` simulated annealing (random searching algorithm – next slide).

Most do *minimization* instead of *maximization* – just put a minus sign out front.

Mixture Model Example



Fit all parameters; $\phi_{\mu,\sigma}(x) = N(\mu, \sigma)$ density

$$-\sum \log((1 - \theta_3)\phi_{\theta_1, \theta_4}(X_i) + \theta_3\phi_{\theta_2, \theta_5}(X_i)) \quad (21)$$

```
fn = function(theta,X)
{
  -sum( log( (1-theta[3])*dnorm(X,mean=theta[1],sd=theta[4])+
            theta[3]*dnorm(X,mean=theta[2],sd=theta[5])) )
}

res = optim( c(2,4.3,0.5,0.5,0.5),fn,
            method = "Nelder-Mead",X=faithful$eruptions )
```

Some Other Methods

Nelder-Mead Simplex

- Like Golden section in higher dimensions.
- In a plane: take three points forming a triangle.
- Reflect the worst point, and expand, otherwise shrink.
- Analogs in higher dimensions.

Simulated Annealing

- Useful in finding global optima in the presence of large numbers of local optima.
- Start somewhere and randomly try a nearby point.
- Decide to move to that point probabilistically, but with lower probability for moving downhill.
- Slowly make the size of moves smaller and insist more on moving uphill.
- Theoretical guarantees that you'll find the "best" maximum (but you'll never know if you have); very computationally expensive.

Some Other Problems

Constrained Optimization

- $x = \operatorname{argmax} f(x)$ but $x \geq 0$; or insist on a more complex region.
- Eg, in mixture model (μ_1, μ_2) gives the same value as (μ_2, μ_1) so constrain $\mu_1 > \mu_2$.
- Multiple ways of incorporating constraints, depending on type of constraint, size of problem etc.

Discrete Optimization

- Sometimes x takes a discrete set of values – integers, category labels etc.
- Example: search over θ_i either 1 or 0 depending on whether to include X_i in a regression model.
- Usually multiple dimensions – far too many possible combinations to search over.
- Many heuristic algorithms developed.

Summary

- Multivariate Optimization considerably more tricky than univariate.
- Trade-offs between local minima, saddle points and speed of convergence.
- Requires some care with coding and checking that the answer you get really is reasonable.
- Many sophisticated methods available.
- Project.

Global Maximum

Fangfang Wang
MA 551

Outline

- 1 Visualizing Multivariate Data
- 2 Global Maximum
- 3 Maximum Likelihood Estimation of Logistic Regression

Graphics Functions for Multivariate Data in R

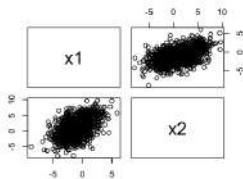
There are several packages in R for making graphs, such as [graphics](#), [lattice](#), [MASS](#), and [rgl](#).

Method	in (graphics)	in (package)
Matrix of scatterplots	<code>pairs</code>	<code>spiom</code> (lattice)
3D scatterplot		<code>cloud</code> (lattice) <code>scatterplot3d</code> (scatterplot3d)
Bivariate density surface	<code>persp</code>	<code>wireframe</code> (lattice)
Contour plot	<code>contour</code> , <code>image</code> <code>contourLines</code> <code>filled.contour</code>	<code>contourplot</code> (lattice) <code>contour</code> (MASS) <code>levelplot</code> (lattice)

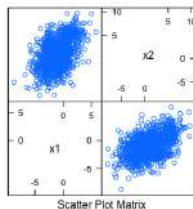
Example 1. Scatterplot matrix for bivariate normal generator

```
y1 = rnorm(1000)
y2 = rnorm(1000)
x1 = -1 + 2*y1
x2 = 1 + 3*0.5*y1 + 3*sqrt(1-0.5^2)*y2
mydata = data.frame(x1, x2) # bivariate normal
```

```
> pairs(mydata)
```

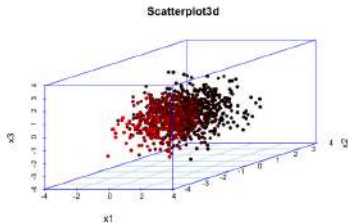
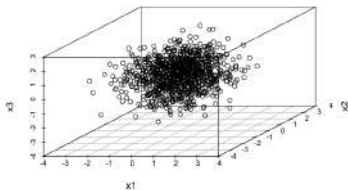


```
> library(lattice)
> splom(mydata)
```



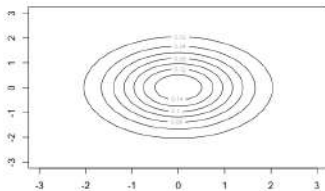
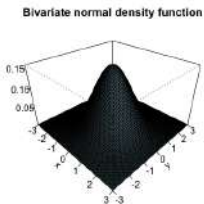
Example 2. 3D scatterplot

```
x1 = rnorm(1000)
x2 = rnorm(1000)
x3 = rnorm(1000)
library(scatterplot3d)
scatterplot3d(x1, x2, x3)
scatterplot3d(x1, x2, x3, highlight.3d=TRUE,
              col.axis="blue", col.grid="lightblue",
              main="Scatterplot3d", pch=20)
```



Example 3. Surface & contour plots of bivariate normal

```
f = function(x,y){
  (1/(2*pi))*exp(-0.5*(y^2+x^2))
}
x = y = seq(-3, 3, length=100)
z = outer(x,y,f)
persp(x, y, z, theta = 45, phi = 30, expand = 0.6,
  ltheta = 120, shade = 0.75, ticktype = "detailed",
  xlab = "x", ylab = "y", zlab = " ", col="lightblue",
  main="Bivariate normal density function")
contour(x,y,z)
```



Outline

- 1 Visualizing Multivariate Data
- 2 Global Maximum
- 3 Maximum Likelihood Estimation of Logistic Regression

Global maximum

- A simple way of using local search techniques to find a global maximum is to consider several different starting points, and hope that for one of them its local maximum is in fact the global maximum.
- Use randomization to choose the starting point.

Global maximum: an example

Consider the function (JMR, chapter 12 exercise 6):

$$f(x, y) = -(x^2 + y^2 - 2)(x^2 + y^2 - 1)(x^2 + y^2)(x^2 + y^2 + 1)(x^2 + y^2 + 2)(2 - \sin(x^2 - y^2) \cos(y - e^y))$$

- If y is too large then $\exp(y)$ is `Inf` and $\cos(y - e^y)$ will return `NaN`. This can cause problems when trying to maximize f .
- A solution is to just set the term $\cos(y - e^y)$ to 0 when this happens.

Global maximum: an example

Code up the function f as follows:

```
fn = function(x) {
  s = sum(x^2)
  y = -(s-2)*(s-1)*s*(s+1)*(s+2)*
      ( 2-sin(x[1]^2-x[2]^2)*cos(x[2]-exp(x[2])) )
  # If x[2] is too large, then exp(x[2]) is Inf
  # and cos(x[2]-exp(x[2])) is NaN.
  # Just set cos(x[2]-exp(x[2])) to 0 in this case
  if (is.nan(y)) {
    y = -(s-2)*(s-1)*s*(s+1)*(s+2)*2
  }
  return(y)
}
```

Global maximum: an example

The gradient ∇f suffers similar problems:

```
gradf = function(x) {
# bug: if x[2] is too large then exp(x[2]) is Inf
# and cos(x[2]-exp(x[2])) is NaN,
# just set cos(x[2]-exp(x[2])) and sin(x[2]-exp(x[2])) to 0.

s = sum(x^2)
f1 = -2*x[1]*((s-1)*s*(s+1)*(s+2) + (s-2)*s*(s+1)*(s+2) +
  (s-2)*(s-1)*(s+1)*(s+2) + (s-2)*(s-1)*s*(s+2) +
  (s-2)*(s-1)*s*(s+1))*(2-sin(x[1]^2-x[2]^2)*cos(x[2]-exp(x[2])))) +
  2*x[1]*(s-2)*(s-1)*s*(s+1)*(s+2)*cos(x[1]^2-x[2]^2)*cos(x[2]-exp(x[2]))

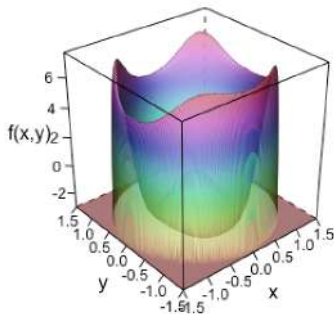
f2 = -2*x[2]*((s-1)*s*(s+1)*(s+2) + (s-2)*s*(s+1)*(s+2) +
  (s-2)*(s-1)*(s+1)*(s+2) + (s-2)*(s-1)*s*(s+2) +
  (s-2)*(s-1)*s*(s+1))*(2-sin(x[1]^2-x[2]^2)*cos(x[2]-exp(x[2])))) -
  2*x[2]*(s-2)*(s-1)*s*(s+1)*(s+2)*cos(x[1]^2-x[2]^2)*cos(x[2]-exp(x[2])) -
  (1 - exp(x[2]))*(s-2)*(s-1)*s*(s+1)*(s+2)*sin(x[1]^2-x[2]^2)*sin(x[2]-exp(x[2]))

if (is.nan(f1)) {
  f1 = -2*x[1]*((s-1)*s*(s+1)*(s+2) + (s-2)*s*(s+1)*(s+2) +
    (s-2)*(s-1)*(s+1)*(s+2) + (s-2)*(s-1)*s*(s+2) + (s-2)*(s-1)*s*(s+1))*2
}

if (is.nan(f2)) {
  f2 = -2*x[2]*((s-1)*s*(s+1)*(s+2) + (s-2)*s*(s+1)*(s+2) + (s-2)*(s-1)*(s+1)*(s+2) +
    (s-2)*(s-1)*s*(s+2) + (s-2)*(s-1)*s*(s+1))*2
}
return(c(f1, f2))
}
```

Global maximum: an example

Surface plot using
`wireframe(lattice)`



- Several local maxima in $[-1.5, 1.5] \times [-1.5, 1.5]$.
- Use `runif(2, -1.5, 1.5)` to generate $(x_0, y_0) \in [-1.5, 1.5] \times [-1.5, 1.5]$.
- Using several randomly chosen starting points, use Steepest Ascent method to find all of the local maxima of f , and thus the global maximum.

Surface plot using wireframe (lattice)

```
x = seq(-1.5, 1.5, .02)
y = seq(-1.5, 1.5, .02)
xyz = data.frame(matrix(0, length(x)*length(y), 3) )
names(xyz) = c('x','y','z')
n = 0
for (i in 1:length(x)) {
  for (j in 1:length(y)) {
    n = n+1
    xyz[n,] = c(x[i], y[j], max(fn(c(x[i],y[j])), -3))
    # f is truncated below at -3
  }
}
library(lattice)
wireframe(z~x*y, data=xyz, scales=list(arrows=FALSE),
          zlab='f(x,y)', shade=T)
```

- The `wireframe` function requires a formula $z \sim x * y$, where $z = f(x, y)$ is the surface to be plotted; the syntax for `wireframe` requires that x , y , and z have the same number of rows.
- See also `persp` on Page 6.

Global maximum: an example

```
##### Find the global maximum
source("ascent.r")
source("line.search.r")
source("gsection.r")

# apply steepest ascent using random starting points
n = 100 # number of starting points
x = matrix(nrow = n, ncol = 2)
fmax = c()

for (i in 1:n) {
  x[i,] = ascent(fn, gradf, runif(2, -1.5, 1.5), n.max=500)
  fmax = c(fmax, fn(x[i,]) )
  x[i,] = round(x[i,], digits=4)
}
```

Global maximum: an example

```
ascent <- function(f, grad.f, x0, tol = 1e-9, n.max = 100)
{
  # steepest ascent algorithm
  # find a local max of f starting at x0
  # function grad.f is the gradient of f
  # by Jones, Maillardet, and Robinson, 2nd Edition.

  x.old <- x0
  x <- line.search(f, x0, grad.f(x0))
  n <- 1
  while ((f(x) - f(x.old) > tol) & (n < n.max)) {
    x.old <- x
    x <- line.search(f, x, grad.f(x))
    n <- n + 1
  }
  return(x)
}
```

Global maximum: an example

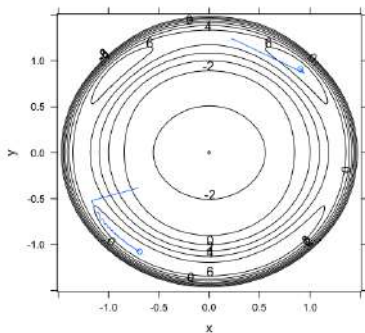
```

line.search <- function(f, x, y, tol = 1e-9, a.max = 2^5) {
  # f is a real function that takes a vector of length d
  # x and y are vectors of length d
  # line.search uses gsection to find a >= 0 such that
  # g(a) = f(x + a*y) has a local maximum at a,
  # within a tolerance of tol if no local max is found
  # then we use 0 or a.max for a,
  # the value returned is x + a*y
  # by Jones, Maillardet, and Robinson, 2nd Edition.
  if (sum(abs(y)) == 0) return(x) # g(a) constant
  g <- function(a) return(f(x + a*y))
  # find a triple a.l < a.m < a.r such that
  # g(a.l) <= g(a.m) and g(a.m) >= g(a.r)
  # a.l
  a.l <- 0; g.l <- g(a.l)
  # a.m
  a.m <- 1; g.m <- g(a.m)
  while ((g.m < g.l) & (a.m > tol)) {
    a.m <- a.m/2; g.m <- g(a.m)
  }
  # if a suitable a.m was not found then use 0 for a
  if ((a.m <= tol) & (g.m < g.l)) return(x)
  # a.r
  a.r <- 2*a.m; g.r <- g(a.r)
  while ((g.m < g.r) & (a.r < a.max)) {
    a.m <- a.r; g.m <- g.r
    a.r <- 2*a.m; g.r <- g(a.r)
  }
  # if a suitable a.r was not found then use a.max for a
  if ((a.r >= a.max) & (g.m < g.r)) return(x + a.max*y)
  # apply golden-section algorithm to g to find a
  a <- gsection(g, a.l, a.r, a.m)
  return(x + a*y)
}

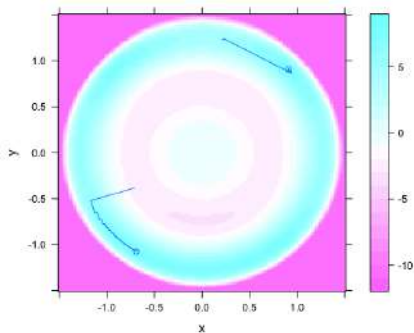
```

Contour lines

Plot contours with trajectories on top



Level plot:



Global maximum: an example

Modified version of steepest ascent that returns intermediate values:

```
ascent2 <- function(f, grad.f, x0, tol = 1e-9, n.max = 100)
{
  # steepest ascent algorithm
  # find a local max of f starting at x0
  # function grad.f is the gradient of f
  # by Jones, Maillardet, and Robinson, 2nd Edition.

  x <- rbind(x0, line.search(f, x0, grad.f(x0)))
  n <- 2
  while ((f(x[n,]) - f(x[n-1,]) > tol) & (n <= n.max)) {
    x <- rbind(x, line.search(f, x[n,], grad.f(x[n,])))
    n <- n + 1
  }
  return(x)
}
```

Global maximum: an example

```
# Apply ascent2 using random starting points
n = 2
xlist = list()
set.seed(1)
for (i in 1:n) {
  xlist[[i]] = ascent2(fn, gradf, runif(2, -1.5, 1.5))
}

# Plot contour lines
contourplot(z~x*y, data=xyz, line.list=xlist,
panel = function(x, y, z, line.list, ...) {
  panel.contourplot(x, y, z, ...)
  for (ln in line.list) {
    panel.lines(ln[,1], ln[,2], ...)
    panel.points(ln[dim(ln)[1],1], ln[dim(ln)[1],2], ...)
  }
},
cuts=20, contour=F, labels=F, region=T, colorkey=F)
```

- region: A logical flag, indicating whether regions between contour lines should be filled as in a level plot.

Outline

- 1 Visualizing Multivariate Data
- 2 Global Maximum
- 3 Maximum Likelihood Estimation of Logistic Regression

Maximum Likelihood Estimation of Logistic Regression

- The file “shuttle.csv” contains data from the Challenger space shuttle which exploded after takeoff on January 28, 1986.

https://en.wikipedia.org/wiki/Space_Shuttle_Challenger_disaster

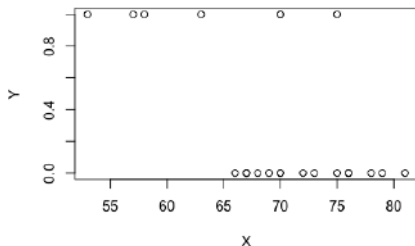
- It contains three variables: *flight*, *ndo* and *temp*.
 - 1 The variable *flight* is simply a code for the launch numbers (prior to Jan. 28, 1986) and can be ignored in what follows.
 - 2 The variable *ndo* is the number of damage o-rings, which was determined after the solid rocket boosters were retrieved from the Atlantic ocean following each launch.
 - 3 The variable *temp* is the temperature at the time of each launch.

```
> mydata = read.csv("shuttle.csv")
> head(mydata)
  flight ndo temp
1      1   0  66
2      2   1  70
3      3   0  69
4      5   0  68
5      6   0  67
6      7   0  72
```

Simple Logistic Regression

```
> table(mydata$ndo)

 0  1  2
16  5  2
> X = mydata$temp
> c(min(X), max(X))
[1] 53 81
> Y =as.numeric(mydata$ndo>0)
> table(Y)
Y
 0  1
16  7
```



- All the previous launches were at temperatures between 53 and 81 degrees.
- The temperature expected for the Challenger launch on Jan 28, 1986 was 31 degrees.

Simple Logistic Regression

- Let Y_i be an indicator of o-ring damage during the i th launch.
- Let $\pi_i = P(Y_i = 1)$.
- Consider the following logistic regression model for the probability of o-ring damage:

$$\ln \frac{\pi_i}{1 - \pi_i} = \beta_0 + \beta_1 X_i, \quad (1)$$

where X_i is the temperature at the i th launch.

- Assume all the launches are independent (which seems reasonable).
- The log-likelihood function is

$$\ell(\beta_0, \beta_1) = \sum_{i=1}^n \{Y_i(\beta_0 + \beta_1 X_i) - \log(1 + \exp(\beta_0 + \beta_1 X_i))\}. \quad (2)$$

[Why?]

Simple Logistic Regression

Fit the logistic regression model using the `glm` function in R:

```
> glm.fit = glm(Y~X, family=binomial("logit")); summary(glm.fit)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-1.0611	-0.7613	-0.3783	0.4524	2.2175

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	15.0429	7.3786	2.039	0.0415 *
X	-0.2322	0.1082	-2.145	0.0320 *

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 28.267 on 22 degrees of freedom
 Residual deviance: 20.315 on 21 degrees of freedom
 AIC: 24.315

Number of Fisher Scoring iterations: 5

Given $\hat{\beta}_0$ and $\hat{\beta}_1$, the fitted logistic response function is

$$\hat{\pi}_i = \left(1 + \exp(-(\hat{\beta}_0 + \hat{\beta}_1 X_i))\right)^{-1}. \quad (3)$$

Using coordinate ascent + golden section algorithm

- Find the MLE of β_0 and β_1 using the coordinate ascent + golden section algorithm.
- Use the starting values $\beta_0 = \ln \frac{\bar{y}}{1-\bar{y}}$ and $\beta_1 = 0$.

```
loglike = function(beta, dat){  
  # dat is a n-by-2 data matrix;  
  # dat[,1] is y and dat[,2] is x  
  y = dat[,1]  
  x = dat[,2]  
  term1 = sum(y*(beta[1]+beta[2]*x))  
  term2 = sum( log( 1+exp(beta[1]+beta[2]*x) ) )  
  return(term1-term2)  
}
```

Using coordinate ascent + golden section algorithm

```
> betal = c(-40,-5)    # lower bounds
> betar = c(40,5)      # upper bounds
> beta.int = c(0.3043478/(1-0.3043478),0) # start
> source('GoldenSection.r')
> source('CoordinateAscent.r')
> source('gsection.r')
> opt = CoordinateAscent(beta.int,betal,betar,loglike,yx)
> opt$mu
[1] 14.3458160 -0.2219666
> opt$iter
[1] 1002
```

Recall that $\pi_i = P(Y_i = 1)$ and $\ln \frac{\pi_i}{1-\pi_i} = \beta_0 + \beta_1 X_i$,
 $i = 1, \dots, 23$.

Using the `glm` function in R, $\hat{\beta}_0 = 15.0429$ and $\hat{\beta}_1 = -0.2322$.
 What are their standard errors?

Generating Random Numbers

JMR Chapter 20; Rizzo Chapter 3

Fangfang Wang
MA 551

(Note JMR Chs 14 - 17 Assumed Knowledge, but a good reference)

Outline

- 1 Generating pseudo random numbers
- 2 Inversion transform
- 3 Rejection methods
- 4 Box-Muller method
- 5 Other methods

Random Number Generation

- One of the fundamental tools required in computational statistics is the ability to simulate random variables from specified probability distributions.
- As we have seen, R has built-in functions for simulating all the common random variables. (See Notes #3)

`rnorm` Generates normal random numbers.

`rexp` Generates exponential random numbers.

- The purpose of this lecture is to see how to do this for ourselves, so that we have the tools for simulating the random variables that R does not provide for us.
- Consider discrete random variables, the inversion and rejection methods for simulating continuous random variables, techniques for simulating normals, mixtures, and convolution.

Where Do Random Numbers Come From?

- Generated from
 - Physical processes (background radiation, radio-active decay etc)
 - Fairly impractical in a computer
 - External generators (human, mouse movements, etc...)
 - Hard to control, model.
 - They are not replicable or reproduced exactly
- Computer-generated *pseudo-random* numbers
 - Deterministic (you'll get the same answer from the same starting point), but looks close to random.
 - Any experiment performed using pseudo-random numbers can be repeated exactly.

Congruential Generators

Simplest effective pseudo-random number generator

$$X_{n+1} = (AX_n + B) \bmod m \quad (1)$$

- Modulus operator returns the remainder of a division, after one number is divided by another
- Results in numbers between 0 and $m - 1$.
- Example: $A = 5, B = 1, m = 4$, and $X_1 = 2$. Then $\{X_2, X_3, X_4, \dots\} =$
- To obtain *uniform* pseudo-random numbers between 0 and 1, take $U_n = X_n/m$

Example

JMR recommends

$A = 1664525$

$B = 1013904223$

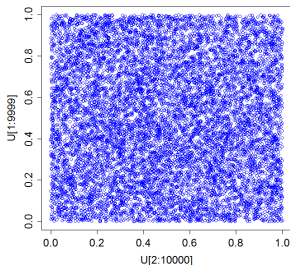
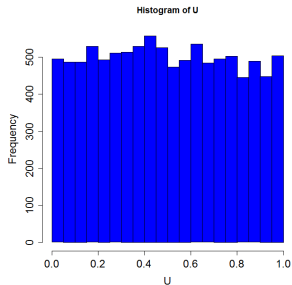
$m = 2^{(32)}$

$X = \text{numeric}(10000)$

$X[1] = 3$

for(i in $2:10000$) { $X[i] = (A * X[i-1] + B) \% m$ }

$U = X/m$



Seeds and Repeatability

- Pseudo-random number generators are *deterministic*: if you start them in the same place, you get the same answer.
- Typically, R chooses a *seed* as a starting point; often from system clock.
- But you can specify this with an integer giving where in the generator's cycle you want to start:

```
> set.seed(36)
> runif(5)
[1] 0.6223777 0.6754986 0.8022900 0.2603083 0.7597607
> set.seed(36)
> runif(5)
[1] 0.6223777 0.6754986 0.8022900 0.2603083 0.7597607
> runif(5)
[1] 0.01990291 0.95542781 0.43666244 0.08922046 0.36051968
```

- Instead of storing everything in a simulation, this lets you re-run it *exactly*.

Outline

- 1 Generating pseudo random numbers
- 2 Inversion transform**
- 3 Rejection methods
- 4 Box-Muller method
- 5 Other methods

From Uniform to Discrete Random Variables

From here on assume we can generate $U(0, 1)$ rv's – how do we get to others?

- Bernoulli rv's with $Ber(p)$. If $U \sim U(0, 1)$, then

$$P(U < p) = p. \quad (2)$$

So $X = I(U < p) \sim Ber(p)$.

- More generally, let X be a discrete rv taking values in $\{0, 1, 2, \dots\}$, with cdf F_X and pmf p_X ; that is,

$$F_X(n) = \sum_{i=0}^n P(X = i) = \sum_{i=0}^n p_i, \quad n = 0, 1, 2, \dots \quad (3)$$

We can generate X by taking $U \sim U(0, 1)$ and

$$i : F_X(i-1) < U \leq F_X(i) \quad (4)$$

Then

$$P(X = i) = P(F_X(i-1) < U \leq F_X(i)) = F_X(i) - F_X(i-1) = p_i.$$

Example: Poisson rv

Simulating a $\text{Poisson}(3)$, $P(X = k) = \frac{\lambda^k e^{-k}}{k!}$:

```
U = runif(1)
X = 0
while( ppois(X,3) < U) { X = X+1 }
```

Check that this produces the right distribution!

Often CDF $F_X(i)$ is not easy to calculate, but PMF $P(X = i)$ is; note we can update $F_X(i)$ within the `while` loop:

```
U = runif(1)
X = 0
FX = dpois(0,3)
while( FX < U) { X = X+1; FX = FX + dpois(X,3) }
```

`dpois` much cheaper than `ppois` to calculate.

Example: Geometric rv

- Recall: $X \sim NB(r, p)$ if its pmf is

$$p_X(k) = \binom{k+r-1}{k} p^r (1-p)^k, k = 0, 1, 2, \dots \quad (5)$$

Geometric distribution if $r = 1$, denoted by $\text{Geom}(p)$. (Notes #3)

- CDF of $X \sim \text{Geom}(p)$: $F_X(x) = 1 - (1-p)^{x+1}$.

Note that $F_X(x-1) < U \leq F_X(x)$ is equivalent to

$$\log(1-U)/\log(1-p) - 1 \leq x < \log(1-U)/\log(1-p). \quad (6)$$

Simulating a $\text{Geom}(.25)$:

```
p = 0.25
```

```
U = runif(1)
```

```
x = ceiling(log(1-U)/log(1-p)) - 1
```

How to generate $NB(4, 0.25)$?

Some Special Cases

- Uniform on the integers $1, \dots, N$; use the `ceiling` command

```
> N = 100; n = 10;  
> (URanNum = N*runif(n))  
[1] 82.09463 64.70602 78.29328 55.30363 52.97196 78.93562  
[9] 73.23137 69.27316  
> ceiling( URanNum )  
[1] 83 65 79 56 53 79 3 48 74 70
```

- We can now generate bootstrap samples:

```
nobs = nrow(faithful) #272  
I = ceiling( nobs*runif(nobs))  
faithboot = faithful[I,]
```

Continuous Random Variables: Inversion Method

The *Inversion* (Transform) method is the simplest way to generate continuous rv's.

If X has cdf F_X , then $F_X(X) \sim U(0, 1)$; in other words, X has the same distribution as $F_X^{-1}(U)$, where $F_X^{-1}(\cdot)$ is the inverse distribution function or quantile function. Why?

Only problem is that $F_X^{-1}(u)$ easy to obtain only in special cases.

Important Special Cases

Uniform $U(a, b)$ Density: $I(x \in [a, b]) / (b - a)$

cdf $F(x) = (x - a) / (b - a)$ if $a \leq x \leq b$

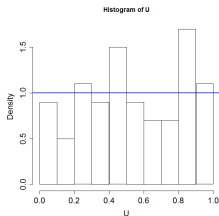
inverse cdf $F^{-1}(U) = (b - a)U + a$

Exponential $\exp(\lambda)$ Density $\lambda e^{-\lambda x}$

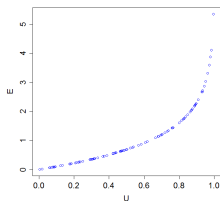
cdf $F(x) = 1 - e^{-\lambda x}$ if $x > 0$

inverse cdf $F^{-1}(U) = -\lambda^{-1} \log(1 - U)$

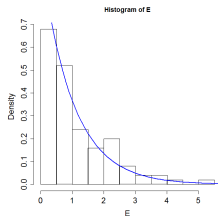
Uniform



Inverse CDF



Exponential(1)

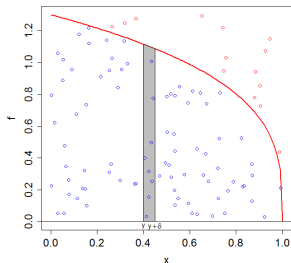


Outline

- 1 Generating pseudo random numbers
- 2 Inversion transform
- 3 Rejection methods**
- 4 Box-Muller method
- 5 Other methods

Continuous Random Variables: Rejection Method

- When F^{-1} is not easy to calculate explicitly – could try numerically.
- Alternatively, rejection method is sometimes faster. (acceptance-rejection method; rejection sampling)
- Simplest case: X has pdf $f_X(\cdot)$ with support on $[a, b]$ and $f_X(x) \leq k$.
 - 1 Generate $Y \sim U(a, b)$ and $Z \sim U(0, k)$, independently.
 - 2 Set $X = Y$ if $Z < f_X(Y)$, otherwise try again.



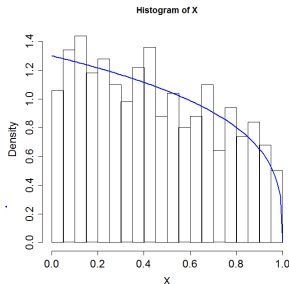
Let X_1 denote the first **accepted** value. Consider a small interval $(y, y + \delta)$. Because (Y, Z) uniform on the rectangle:

$$\begin{aligned}
 P(y < X_1 < y + \delta) &= \frac{\int_y^{y+\delta} f_X(x) dx}{\int_a^b f_X(x) dx} \\
 &= \int_y^{y+\delta} f_X(x) dx.
 \end{aligned}$$

In Code

Consider a $Beta(a, b)$ distribution, i.e., $X \sim f(x) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} x^{a-1} (1-x)^{b-1}$, with $a = 1$ and $b = 1.3$. Note that $f(x) \leq 1.3$. [Why?]

```
X = rep(0,1000)
for(i in 1:1000){
  Accept = FALSE
  while(!Accept){
    Y = runif(1)
    Z = runif(1,0,1.3)
    if( Z<dbeta(Y,1,1.3) ){
      Accept = TRUE
    }
  }
  X[i] = Y
}
```



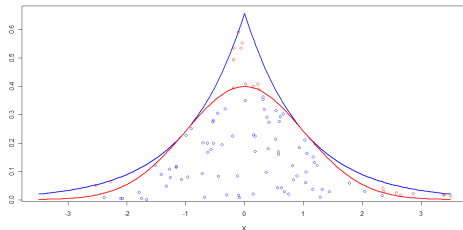
Cheaply (no fixed n):

```
Y = runif(1000)
Z = runif(1000,0,1.3)
Accept = Z < dbeta(Y,1,1.3)
X = Y[Accept]
```

Acceptance rate?

Generalized Rejection Method

- For densities on the whole real line, we can't use a uniform distribution.
- But, if we can simulate from $h(x)$ where $kh(x) \geq f_X(x)$, we can use the same ideas.



Call $kh(x)$ or $h(x)$ the *envelope* for $f_X(x)$.

- 1 Generate $Y \sim h(\cdot)$
- 2 Generate $Z \sim U(0, kh(Y))$
- 3 Accept Y if $Z < f_X(Y)$

Justification

General rejection method is justified because the (Y, Z) pairs are uniformly distributed over the region below $kh(x)$.

$$\begin{aligned}
 P((Y, Z) \in (y, y + dy) \times (z, z + dz)) \\
 &= P(Z \in (z, z + dz) | Y \in (y, y + dy)) P(Y \in (y, y + dy)) \\
 &= \frac{dz}{kh(y)} h(y) dy \\
 &= dz dy / k
 \end{aligned}$$

because $P(Z \in (z, z + dz) | Y \in (y, y + dy))$ is uniform on $[0, kh(y)]$.

This means the points we accept are uniformly distributed on the region under $f_X(x)$ and therefore the x -coordinates have density $f_X(x)$.

Example: Standard Normal

- In picture above, we used a Laplace distribution $h(x) = \frac{1}{2}e^{-|x|}$ as an envelope for standard normal.
- To generate from Laplace, use $V \sim \text{Ber}(0.5)$ and $U \sim U(0, 1)$. Because $2(V - 0.5) \in \{-1, 1\}$ and $-\log U \sim \text{Exp}(1)$, we have

$$2(V - 0.5) \log U \sim h(\cdot). \quad [\text{Why?}] \quad (7)$$

- To find k , ratio of densities is

$$\frac{\frac{1}{\sqrt{2\pi}}e^{-x^2/2}}{\frac{1}{2}e^{-|x|}} = \frac{\sqrt{2}}{\sqrt{\pi}}e^{|x|-x^2/2} \leq \sqrt{\frac{2e}{\pi}} \quad (8)$$

Example Continued

We'll fix the size of Y and Z and just see how many X we get after rejection:

```
# Generate Laplace r.v.'s
V = 2*((runif(1000)>0.5)-0.5)
U = runif(1000)
Y = V*log(U)

# Uniforms
Z = runif(1000)*0.5*exp(-abs(Y))*sqrt(2*exp(1)/pi)

# Which are accepted?
Accept = Z < dnorm(Y)

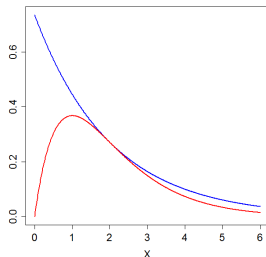
# Now we get our sample
X = Y[Accept]
```

Efficiency

- In last example above, we accept about 75% of tries.
- Higher acceptance probability = less computational work.
- Overall acceptance rate = $1/k$
 - area under $kh(x) = k$, area under $f_X(x) = 1$.
- Alternatively, number tries before accepting is Geometric($1/k$) with expected value $k - 1$.
- Two things we can affect
 - Choice of $h(x)$ (harder)
 - Choice of k .
- See optimizing Gamma on next slide.

Example: Gamma distribution

- For $m, \lambda > 0$, the $\Gamma(m, \lambda)$ pdf is $f(x) = \frac{\lambda^m}{\Gamma(m)} x^{m-1} e^{-\lambda x}$, for $x > 0$.
- Use an exponential envelope $h(x) = \mu e^{-\mu x}$, for $x > 0$. [Choice of μ ?]
- To envelop f , we need to find



$$k^* = \sup_{x>0} \frac{f(x)}{h(x)} = \frac{\lambda^m}{\Gamma(m)\mu} \sup_{x>0} x^{m-1} e^{(\mu-\lambda)x} \quad (9)$$

- 1 Let $m > 1$ (Note that $k^* = \infty$ if $m < 1$ [why?])
- 2 Choose $\mu < \lambda$.
- 3 Then $k^* = \frac{\lambda^m}{\Gamma(m)\mu} \frac{(m-1)^{m-1} e^{-(m-1)}}{(\lambda-\mu)^{m-1}}$. [Why?]
- 4 Further, set $\mu = m^{-1}\lambda$ and thus $k^* = \frac{m^m e^{-(m-1)}}{\Gamma(m)}$. [Why?]

Simulating Normal Random Variables

Note if $X \sim N(0, 1)$, then $\sigma X + \mu \sim N(\mu, \sigma^2)$, easy once we can generate $N(0, 1)$.

- 1** Central limit theorem: $\text{Var}(U) = 1/12$, so

$$\left(\sum_{i=1}^{12} U_i \right) - 6 \approx N(0, 1). \quad (10)$$

12 is a bit small; could add more terms + rescale, but this is computationally expensive.

- 2** Improved rejection method

- In fact, $Z \sim U(0, U\sqrt{2e/\pi})$ where $U \sim U(0, 1)$.
- We can also throw away V and just decide to make Y negative 1/2 time based on Z .

- 1** $U \sim U(0, 1)$, $Z \sim U(0, U\sqrt{2e/\pi})$, $Y = -\log(U)$
- 2** If $Z < \phi(Y)/2$ return $-Y$; Else if $\phi(Y)/2 < Z < \phi(Y)$ return Y ; Else go back to Step 1.

Improved rejection method for generating $N(0,1)$

Generate $N(0,1)$:

$$\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}.$$

Key ideas:

- Use an easy proposal on $[0, \infty)$: $\text{Exp}(1)$.
- Use an envelope ke^{-y} that dominates $2\phi(y)$ for $y \geq 0$.
- Use the same uniform variable to assign the sign ($\pm$) with prob $1/2$.

Symmetry reduction to $[0, \infty)$:

Because ϕ is symmetric,

$$X \sim N(0, 1) \iff \begin{cases} Y = |X| \text{ has density } f_+(y) = 2\phi(y), y \geq 0, \\ \text{and } \text{sign}(X) \text{ is } \pm \text{ with prob. } 1/2 \text{ independently.} \end{cases}$$

So it suffices to sample $Y \geq 0$ from $2\phi(y)$ and then attach a fair sign.

Improved rejection method for generating $N(0,1)$

Let $U \sim U(0, 1)$ and define

$$Y = -\log U.$$

Then $Y \sim \text{Exp}(1)$ with density

$$h(y) = e^{-y}, \quad y \geq 0.$$

Compute the constant k for all $y \geq 0$:

$$\frac{2\phi(y)}{e^{-y}} = \frac{2}{\sqrt{2\pi}} \exp\left(-\frac{y^2}{2} + y\right) = \frac{2}{\sqrt{2\pi}} \exp\left(\frac{1}{2} - \frac{(y-1)^2}{2}\right).$$

This is maximized at $y = 1$, hence the optimal choice is

$$k = \max_{y \geq 0} \frac{2\phi(y)}{e^{-y}} = \sqrt{\frac{2e}{\pi}}.$$

Acceptance probability is $1/k \approx 0.76$.

Uniform under the envelope (key geometric fact)

Standard rejection sampling draws (Y, Z) uniformly under $kh(y)$:

$$Y \sim h, \quad Z \mid Y = y \sim U(0, kh(y)).$$

Here, since $Y = -\log U$ implies $e^{-Y} = U$, we have

$$kh(Y) = ke^{-Y} = kU.$$

So we can equivalently generate

$$U \sim U(0, 1), \quad Y = -\log U, \quad Z \sim U(0, kU).$$

Then (Y, Z) is uniform over $\{(y, z) : 0 \leq z \leq ke^{-y}\}$. We accept only points under the target:

$$0 \leq Z < \phi(Y).$$

Geometrically,

- big region: under ke^{-y}
- small region: under $\phi(y)$.

Accepted points are uniform under $\phi(y)$.

Conditional on acceptance ($Z < \phi(y)$),

$$Z \mid (Y = y, \text{acc}) \sim U(0, \phi(y)).$$

Hence

$$\mathbb{P}(Z < \phi(y)/2 \mid Y = y, \text{acc}) = \frac{1}{2}.$$

So split this region vertically:

$$\begin{cases} 0 \leq Z < \phi(Y)/2 & \Rightarrow X = -Y, \\ \phi(Y)/2 \leq Z < \phi(Y) & \Rightarrow X = +Y. \end{cases}$$

This replaces an extra coin flip.

Improved rejection algorithm: Let $k = \sqrt{2e/\pi}$, $\phi(y) = \frac{1}{\sqrt{2\pi}} e^{-y^2/2}$.

- 1 Draw $U \sim U(0, 1)$; set $Y = -\log U$.
- 2 Draw $Z \sim U(0, kU)$.
- 3 If $Z < \phi(Y)/2$, return $X = -Y$.
- 4 Else if $\phi(Y)/2 < Z < \phi(Y)$, return $X = Y$.
- 5 Else reject and go back to Step 1.

Outline

- 1 Generating pseudo random numbers
- 2 Inversion transform
- 3 Rejection methods
- 4 Box-Muller method**
- 5 Other methods

Box-Muller for Gaussians

- Clever construction method. Look at *pairs* of Gaussians (X, Y) , where X and Y are independent $N(0, 1)$.
- Now in polar co-ordinates (R, Θ) , we have $R^2 \sim \exp(0.5)$, $\Theta \sim U(0, 2\pi)$.
- So if we have $U_1, U_2 \sim U(0, 1)$, then $\sqrt{-2 \log U_1} \cos(2\pi U_2)$ and $\sqrt{-2 \log U_1} \sin(2\pi U_2)$ are Gaussian!

This yields the following

- 1 $U_1, U_2 \sim U(0, 1)$
- 2 $X = \sqrt{-2 \log U_1} \cos(2\pi U_2), Y = \sqrt{-2 \log U_1} \sin(2\pi U_2)$

to obtain independent normal $X, Y \sim N(0, 1)$.

More Efficient Box-Muller

Trigonometric functions are expensive.

- Instead, if (A, B) uniform on the circle with polar coordinates (S, Ψ) , $S^2 = A^2 + B^2 \sim U(0, 1)$ and $\Psi \sim U(0, 2\pi)$. S and Ψ are independent.
- $(\sqrt{-2 \log S^2}, \Psi)$ has the same distribution as (R, Θ) .
- Also $\cos(\Psi) = A/S$, $\sin(\Psi) = B/S$.

Improved algorithm is

- 1 $U, V \sim U(-1, 1)$ (uniform on box)
- 2 If $S^2 = U^2 + V^2 > 1$ retry (rejection: uniform on circle)
- 3 Set $W = \sqrt{(-2 \log S^2)/S^2}$
- 4 $X = UW, Y = VW$.

Outline

- 1 Generating pseudo random numbers
- 2 Inversion transform
- 3 Rejection methods
- 4 Box-Muller method
- 5 Other methods**

Constructive Methods

Remember, direct transforms are generally fastest:

$$\text{Exp}(\lambda) \quad -\lambda^{-1} \log(U) \text{ if } U \sim U(0, 1).$$

$$\text{Ber}(p) \quad I(U < p) \text{ if } U \sim U(0, 1).$$

$$\text{Laplace} \quad 2(V - 1/2)E \text{ if } E \sim \text{Exp}(1) \text{ and } V \sim \text{Ber}(0.5).$$

$$\text{Bin}(n, p) \quad \sum_{i=1}^n Z_i \text{ if } Z_1, \dots, Z_n \sim \text{Ber}(p).$$

$$\chi_d^2 \quad \sum_{i=1}^d X_i^2 \text{ if } X_1, \dots, X_d \sim N(0, 1).$$

$$F_{d_1, d_2} \quad \frac{Z_{d_1}/d_1}{Z_{d_2}/d_2} \text{ if } Z_{d_1} \sim \chi_{d_1}^2 \text{ and } Z_{d_2} \sim \chi_{d_2}^2.$$

Many other relationships; some derived, some constructed.

Convolutions

- Let $X_1, \dots, X_n$ be iid, and $X_j \sim X$.
- The distribution function of $S = X_1 + \dots + X_n$ is called the **n -fold convolution** of X and denoted $F_X^*(n)$.
- To simulate a convolution: generate $X_1, \dots, X_n$ and compute the sum.
- Binomial r.v.'s are a sum of independent Bernoulli's:
 $X \sim \text{Bin}(n, p) \Rightarrow X = \sum_{i=1}^n Z_i$ where $Z_i \sim \text{Ber}(1, p)$

```
n = 10
p = 0.45
X = sum( runif(n) < p )
```
- $NB(r, p)$ is the convolution of r iid $\text{Geom}(p)$.


```
p = 0.25; r = 4
U = runif(r)
x = sum( ceiling(log(1-U)/log(1-p)) - 1 )
```


Mixtures

- A random variable X is a discrete mixture if the cdf of X is a weighted sum

$$F_X(x) = \sum_i \theta_i F_{X_i}(x) \quad (11)$$

for some sequence of rv's $X_1, X_2, \dots$, and $\theta_i > 0$ such that $\sum_i \theta_i = 1$. The constants θ_i are called the mixing weights or mixing probabilities.

- A random variable X is a continuous mixture if the cdf of X is

$$F_X(x) = \int F_{X|Y=y}(x) f_Y(y) dy \quad (12)$$

for a family $X|Y = y$ indexed by the real numbers y and weighting function f_Y such that $\int f_Y(y) dy = 1$.

Simulation of a convolution and a mixture of normal variables

Suppose $X_1 \sim N(0, 1)$ and $X_2 \sim N(3, 1)$ are independent.

- The notation $S = X_1 + X_2$ denotes the convolution of X_1 and X_2 . The distribution of S is $N(3, 2)$. To simulate the convolution:

- 1 Generate $x_1 \sim N(0, 1)$.
- 2 Generate $x_2 \sim N(3, 1)$.
- 3 Deliver $s = x_1 + x_2$.

- 50% normal mixture $X : F_X(x) = 0.5F_{X_1}(x) + 0.5F_{X_2}(x)$.

The distribution of X is distinctly non-normal; it is bimodal. To simulate the mixture:

- 1 Generate an integer $k \sim \text{Ber}(1, 0.5)$.
- 2 If $k = 0$ deliver x from $N(0, 1)$; if $k = 1$ deliver x from $N(3, 1)$.

The negative binomial distribution

- Recall: $NB(r, p)$ is the convolution of r iid $\text{Geom}(p)$.
- The negative binomial distribution is a Poisson-Gamma Mixture.
 - a mixture of $\text{Poisson}(\Lambda)$, where Λ has a gamma distribution;
 - continuous mixture
 - Specifically, if $X|\Lambda = \lambda \sim \text{Poisson}(\lambda)$ and $\Lambda \sim \text{Gamma}(r, \beta)$, then $X \sim NB(r, p)$ with $p = \beta/(1 + \beta)$.

$$\begin{aligned}
 \int_0^\infty f_{\text{Poisson}}(k) f_{\text{Gamma}}(\lambda) d\lambda &= \int_0^\infty \frac{e^{-\lambda}}{k!} \lambda^k \frac{\beta^r}{\Gamma(r)} \lambda^{r-1} e^{-\beta\lambda} d\lambda \\
 &= \frac{1}{k!} \frac{\beta^r}{\Gamma(r)} \left(\frac{1}{1 + \beta} \right)^{k+r} \Gamma(k + r) \\
 &= \binom{k + r - 1}{k} p^r (1 - p)^k
 \end{aligned}$$

- The negative binomial distribution was originally derived as a limiting case of the gamma-Poisson distribution.

The negative binomial distribution

```
gamma.sim <- function(lambda, m) {
  # sim a gamma(lambda, m) rv using rejection
  # with an exp envelope
  # assumes m > 1 and lambda > 0
  f <- function(x) lambda^m*x^(m-1)*exp(-lambda*x)/gamma(m)
  h <- function(x) (lambda/m)*exp(-(lambda/m)*x)
  k <- m^m*exp(1-m)/gamma(m)
  while (TRUE) {
    X <- -log(runif(1))*m/lambda
    Y <- runif(1, 0, k*h(X))
    if (Y < f(X)) return(X)
  }
}

## Generate a Poisson-Gamma mixture
r = 4; beta = 3
(lambda = gamma.sim(beta,r)) #lambda is random
U = runif(1); x = 0
FX = dpois(x,lambda)
while( FX < U){ x = x+1; FX = FX + dpois(x,lambda) }
```

Summary

- Random number generation actually *pseudo*-random.
- But deterministic random variables allow you to repeat simulations easily.
- Once you can get uniform random variables, get others by
 - 1 inversion transform
 - 2 rejection methods
 - 3 being very clever (convolutions, mixtures, etc.)
- Other approaches: MCMC, Adaptive rejection sampling, Squeeze methods, etc.

Monte Carlo Integration

JMR Ch 21-22; Rizzo Ch 5

Fangfang Wang
MA 551

Outline

1 Numerical Integration

2 Monte Carlo Integration

3 Variance Reduction

- Importance Sampling

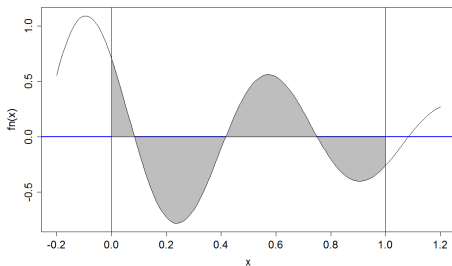
The Problem

We want to evaluate

$$I = \int_a^b g(x) dx \quad (1)$$

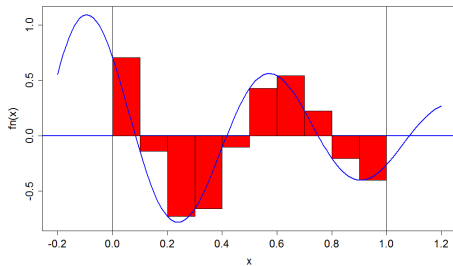
but don't know how to take the definite integral of g .

$$\int_0^1 e^x \sin(3\pi(x + 0.25)) dx \quad (2)$$



Approximation with Simple Functions

Let's just use rectangles.



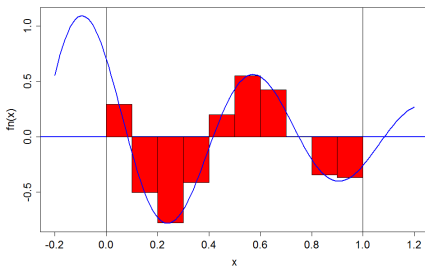
- Choose a grid of points $q_1, \dots, q_N$, where $q_1 = a$ and $q_N = b$.
- Approximate $g(x)$ by $g(q_t)$ between q_t and q_{t+1} .
- Area under approximation is $\hat{I} = \sum_{t=1}^{N-1} (q_{t+1} - q_t)g(q_t)$.

The midpoint rule

Begin with the *midpoint* rule variant of the rectangle method

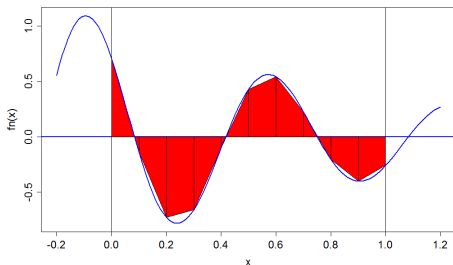
$$\hat{I} = \sum_{t=1}^{N-1} (q_{t+1} - q_t) g(m_t) \quad (3)$$

where $m_t = (q_{t+1} + q_t)/2$ is the midpoint between the two.



The Trapezoidal Rule

Perhaps straight lines would be better: still easy to work with.



- Area under the trapezoid is $(q_{t+1} - q_t) \frac{g(q_{t+1}) + g(q_t)}{2}$. (check!)
- For points spaced h apart:

$$\hat{I} = \frac{h}{2} (g(q_1) + 2g(q_2) + \dots + 2g(q_{N-1}) + g(q_N)) \quad (4)$$

Simpson's Rule

Maybe quadratic interpolation? For three points q_{t-1} , q_t , q_{t+1} ,

$$\begin{aligned}\hat{g}(x) = & g(q_{t-1}) \frac{(x - q_t)(x - q_{t+1})}{(q_{t-1} - q_t)(q_{t-1} - q_{t+1})} + g(q_t) \frac{(x - q_{t-1})(x - q_{t+1})}{(q_t - q_{t-1})(q_t - q_{t+1})} \\ & + g(q_{t+1}) \frac{(x - q_{t-1})(x - q_t)}{(q_{t+1} - q_{t-1})(q_{t+1} - q_t)}\end{aligned}$$

goes through all of $(q_{t-1}, g(q_{t-1}))$, $(q_t, g(q_t))$, and $(q_{t+1}, g(q_{t+1}))$.

Assuming a spacing of size h , the integral between q_{t-1} and q_{t+1} is (after some calculation)

$$\int_{q_{t-1}}^{q_{t+1}} \hat{g}(x) dx = \frac{h}{3} (g(q_{t-1}) + 4g(q_t) + g(q_{t+1})). \quad (5)$$

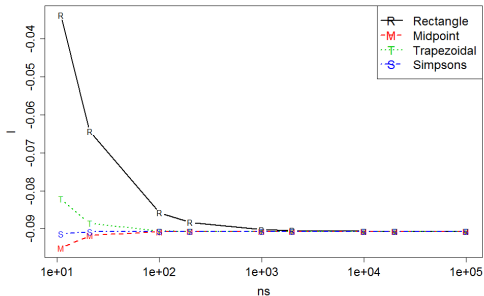
Putting these intervals together (with N odd) we get

$$\hat{I} = \frac{h}{3} (g(q_1) + 4g(q_2) + 2g(q_3) + 4g(q_4) + \dots + 4g(q_{N-1}) + g(q_N)). \quad (6)$$

Numerical integration

- Numerical integration = approximate $g(x)$ with something you can integrate.
- Rectangle, midpoint, trapezoidal, Simpson's rule – divide (finite) interval into small chunks and approximate within each chunk.

Estimates all converge to the same value:



Outline

1 Numerical Integration

2 Monte Carlo Integration

3 Variance Reduction

- Importance Sampling

The Monte Carlo Integral

- The term Monte Carlo is used to refer to techniques involving computer simulation.
- Monte Carlo integration is numerical integration using simulation.
- Monte Carlo integration is a statistical method based on random sampling.
- Monte Carlo methods were developed in the late 1940's after World War II.
- Two simulation-based approaches to integration:
 - hit-and-miss method
 - (improved) Monte Carlo method

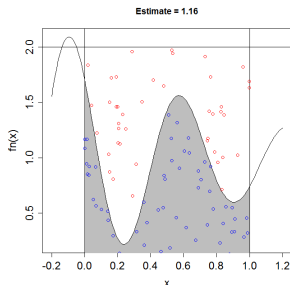
Hit-and-Miss Method

To evaluate $I = \int_a^b g(x)dx$,

- 1 Draw $X_1, \dots, X_N$ uniform on $[a \ b]$.
- 2 Draw $Y_1, \dots, Y_N$ uniform on $[c \ d]$ for c and d bounding $g(x)$ from below and above.
- 3 Estimate from proportion of $Y_i \leq g(X_i)$

$$\tilde{I} = \frac{(b-a)(d-c)}{N} \sum_{i=1}^N I(Y_i \leq g(X_i)) + c(b-a).$$

- Similar to rejection sampling.
- Hit-and-miss method converges very slowly.
- Next, we give a better Monte Carlo integration technique, which is the technique people usually refer to as 'Monte Carlo integration'.



(Improved) Monte Carlo Integral

- Recall that if $X \sim f(\cdot)$ the expectation of $g(X)$ is

$$\mathbb{E}g(X) = \int g(x)f(x)dx. \quad (7)$$

- To evaluate

$$I = \int_a^b g(x)dx, \quad (8)$$

we introduce $f(x) = \frac{1}{b-a}I(a \leq x \leq b)$, then

$$I = (b-a) \int g(x)f(x)dx = (b-a)\mathbb{E}g(X), \quad (9)$$

where $X \sim U(a, b)$. Hence, I can be **estimated** by taking $X_1, \dots, X_N \sim U(a, b)$ and

$$\hat{I} = (b-a) \frac{1}{N} \sum_{i=1}^N g(X_i). \quad (10)$$

- We will be more interested in the **expectation** than the integral.

Convergence and Variance

Generally, consider $I = \int g(x)f(x)dx$. Suppose $f(x)$ is the pdf of a certain rv, say X . Because

$$I = \mathbb{E}g(X), \quad (11)$$

we estimate I by $\hat{I} = \frac{1}{N} \sum_{i=1}^N g(X_i)$ where $X_i \sim iid f()$,

$$\mathbb{E}\hat{I} = \mathbb{E}g(X), \quad (12)$$

$$\text{Var}(\hat{I}) = \frac{1}{N} \text{Var}(g(X)). \quad (13)$$

Moreover, $\hat{I}$ converges to a normal distribution and

$$\hat{I} - I = O_p(N^{-1/2}) \quad (14)$$

Aside: $X_N = O_p(N^{-1/2})$ means

$\forall \epsilon, \exists M > 0 : P(N^{1/2}|X_N| > M) < \epsilon.$

Comparison with Quadrature

Error rate is $N^{-1/2}$, compares to N^{-2} or N^{-4} for trapezoidal or Simpson's rule.

But for a d -dimensional integral, the error is

- $N^{-2/d}$ for Trapezoidal rule.
- $N^{-4/d}$ for Simpson's rule.
- $N^{-1/2}$ for Monte Carlo integration.

So for $d > 4$, MC beats trapezoidal rule, for $d > 8$ it beats Simpson's rule.

Note that MC integration works on infinite domains if using a density $f(\cdot)$ with infinite support.

Example

An alternative way to calculate $\Gamma(m)$.

- $\Gamma(m) = \int_0^\infty x^{m-1} e^{-x} dx$.
- For m an integer $\Gamma(m) = (m-1)!$.
- To calculate by MC integration, recognize that $f(x) = e^{-x}$ is an exponential density, so that (by the LLN and CLT)

$$\Gamma(m) = \int_0^\infty x^{m-1} e^{-x} dx \approx \frac{1}{N} \sum_{i=1}^N X_i^{m-1} \quad (15)$$

where $X_1, \dots, X_m \sim \text{Exp}(1)$.

```
> m = 5
> N = 1000
> X = rexp(N)
> mean( X^(m-1) )
[1] 20.55042
```

Variance

We also know that the variance of this estimate is

$$\frac{1}{N} \left(\int_0^{\infty} x^{2m-2} e^{-x} dx - \left[\int_0^{\infty} x^{m-1} e^{-x} dx \right]^2 \right) \quad (16)$$

```
> ( VI = (mean(X^(2*(m-1))) - mean(X^(m-1))^2)/N )
[1] 23.10942
```

Or the standard deviation is

```
> sqrt(VI)
[1] 4.807226
```

So in order to reduce the standard deviation below 1, we would need $4.8^2 < 25$ times as many points.

```
> N = 25000
> X = rexp(N)
> mean( X^(m-1) )
[1] 23.14568
```

Efficiency

- If $\hat{I}_1$ and $\hat{I}_2$ are two estimators for I , then $\hat{I}_1$ is more efficient (in a statistical sense) than $\hat{I}_2$ if

$$\text{Var}(\hat{I}_1) < \text{Var}(\hat{I}_2) \quad (17)$$

- The percent reduction in variance achieved by using $\hat{I}_1$ instead of $\hat{I}_2$ is

$$100 \left(\frac{\text{Var}(\hat{I}_2) - \text{Var}(\hat{I}_1)}{\text{Var}(\hat{I}_2)} \right) \quad (18)$$

- If the variances of estimators are unknown, we can estimate efficiency by substituting a sample estimate of the variance for each estimator.
- Note that variance can always be reduced by increasing the number of replicates, so computational efficiency is also relevant.

Outline

- 1 Numerical Integration
- 2 Monte Carlo Integration
- 3 Variance Reduction**
 - Importance Sampling

Changing the Distribution

First trick: suppose we want

$$I = \mathbb{E}g(X), \quad X \sim f_1(\cdot) \quad (19)$$

but can cheaply generate data from $f_2(\cdot)$, then

$$\hat{I} = \frac{1}{N} \sum_{i=1}^N \frac{g(X_i)f_1(X_i)}{f_2(X_i)}, \quad X_1, \dots, X_N \sim f_2 \quad (20)$$

has

$$\mathbb{E}\hat{I} = \int g(x) \frac{f_1(x)}{f_2(x)} f_2(x) dx = \int g(x) f_1(x) dx \quad (21)$$

Variance Reduction

In many situations it would be useful to work out how to sample so as to reduce MC variance as much as possible.

Two things that we will consider

- What distribution are the X_i drawn from?
- Dependence in the X_i .

Consider the MC approximation:

$$I = \int g(x)dx = \int \frac{g(x)}{f(x)}f(x)dx \approx \frac{1}{N} \sum_{i=1}^N \frac{g(X_i)}{f(X_i)} \quad (22)$$

where $X_1, \dots, X_n$ are iid from f .

The density $f(x)$ is called importance function.

Importance Sampling

What's the best choice?

- $f(x) > 0$ on the set $\{x : |g(x)| > 0\}$.
- Obvious choice is to minimize:

$$\text{Nvar}(\hat{I}) = \int \left(\frac{g(x)}{f(x)} \right)^2 f(x) dx - \left[\int \frac{g(x)}{f(x)} f(x) dx \right]^2$$

- If $g(x)/f(x) = \text{constant}$, $\text{var}(\hat{I}) = 0$.
- But this means we could sample from $M^{-1}g(x)$ where $M = \int g(x) dx$ is the quantity of interest!
- Closer $g(x)/f(x)$ is to a constant, the smaller the variance of $\hat{I}$.

Choose $f(x)$ so $X_1, \dots, X_N$ is easily simulated and $g(x)/f(x)$ is 'close' to constant.

Example

Consider $\int_0^1 (1 - x^2) dx = 2/3$.

1 Let $U_1, \dots, U_N \sim U(0, 1)$:

$$\hat{I}_1 = \frac{1}{N} \sum_{i=1}^N (1 - U_i^2) \quad (23)$$

$$\text{var}(\hat{I}_1) = \frac{1}{N} \left(\int_0^1 (1 - x^2)^2 dx - 4/9 \right) = \frac{4}{45N}. \quad (24)$$

2 But if sample $X_i \sim \text{Beta}(1, 2)$, i.e., $f(x) = 2(1 - x)$, then

$$I = \int_0^1 \frac{(1 - x^2)}{2(1 - x)} 2(1 - x) dx = \int_0^1 \frac{1}{2} (1 + x) f(x) dx, \quad \hat{I}_2 = \frac{1}{2N} \sum_{i=1}^N (1 + X_i)$$

and

$$\text{var}(\hat{I}_2) = \frac{1}{N} \left(\int_0^1 \frac{1}{2} (1 - x^2)(1 + x) dx - 4/9 \right) = \frac{1}{72N}. \quad (25)$$

3 Note the density $3(1 - x^2)/2$ would give zero variance, but isn't a Beta.

Simulation Results

```
N = 1000; nsim = 1000
Est1 = rep(0,nsim)
Est2 = rep(0,nsim)
for(i in 1:nsim){
  X = runif(N)
  Est1[i] = mean( 1 - X^2 )

  X = rbeta(N,1,2)
  Est2[i] = mean( 0.5*(1+X) )
}

> var(Est1)/var(Est2)
[1] 6.072158

# Ratio of Analytic Variances
> (4/45)/(1/72)
[1] 6.4
```

Slightly More Realistic

Evaluate Normal probability of being greater than 2:

$$\int_2^{\infty} \phi(x) dx = \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-x^2/2} I(x > 2) dx \quad (26)$$

Two approaches:

- 1** Simulate X_i from $N(0, 1)$ and estimate proportion $X_i > 2$:

$$\hat{I}_1 = \frac{1}{N} \sum_{i=1}^N I(X_i > 2). \quad (27)$$

- 2** Importance sampling: simulate X_i from $N(2, 1)$ – put more data where it matters.

$$\hat{I}_2 = \frac{1}{N} \sum_{i=1}^N e^{(-X_i^2 + (X_i - 2)^2)/2} I(X_i > 2) = \frac{e^2}{N} \sum_{i=1}^N e^{-2X_i} I(X_i > 2). \quad (28)$$

```
for(i in 1:1000){
  X = rnorm(1000)
  Est1[i] = mean( X > 2 )
  Est2[i] = mean(exp(2-2*X) * (X>2))
}
> var(Est1)/var(Est2)
[1] 18.47664
```

A Warning About Tails

In the $\Gamma(m)$ example, we could have chosen X_i from $N(0,1)$, then we would have

$$\hat{I} = \frac{1}{N} \sum_{i=1}^N X_i^{m-1} e^{-X_i} \sqrt{2\pi} e^{X_i^2/2} I(X_i > 0)$$

Hence the variance is

$$\begin{aligned} \text{var}(\hat{I}) &= \frac{2\pi}{N} \int_0^\infty x^{2m-2} e^{-2x} e^{x^2} \phi(x) dx - \Gamma(m)^2 \\ &= \frac{\sqrt{2\pi}}{N} \int_0^\infty x^{2m-2} e^{-2x+x^2/2} dx - \Gamma(m)^2 \\ &= \infty. \end{aligned}$$

Lesson is that tails of $f(x)$ should never get small with respect to $g(x)$; that is, $f(x)$ should decrease slower than $g(x)$, as $x \rightarrow \pm\infty$.

Summary

- Treat integrals as expectations and just simulate.
- But, distribution you simulate from can be important.
- Importance sampling: attempt to simulate where you will see the largest area.

Next, Stratified sampling, Antithetic sampling, and Control variates

Variance Reduction

JMR Ch 21-22; Rizzo Ch 5

Fangfang Wang
MA 551

Outline

- 1 Importance Sampling
- 2 Stratified Sampling
- 3 Stratified Importance Sampling
- 4 Antithetic Sampling
- 5 Control Variates

Example 5.10 of Rizzo

Consider $I = \int_0^1 \frac{e^{-x}}{1+x^2} dx$. The candidates for the importance functions are

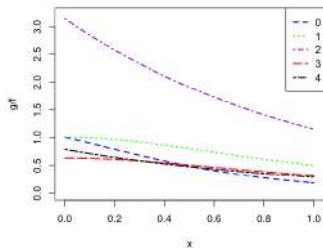
$$f_0 = 1, \quad 0 < x < 1,$$

$$f_1 = e^{-x}, \quad 0 < x < \infty,$$

$$f_2 = \frac{1}{\pi(1+x^2)}, \quad -\infty < x < \infty,$$

$$f_3 = \frac{e^{-x}}{1-e^{-1}}, \quad 0 < x < 1,$$

$$f_4 = \frac{4}{\pi(1+x^2)}, \quad 0 < x < 1.$$



f_1 and f_2 are inefficient; $g(x)/f_3(x)$ is most nearly constant.

Example 5.10 of Rizzo, Cont'd

```

N = 1000
Est0 = Est1 = Est2 = Est3 = Est4 = numeric(1000)

g = function(x){ exp(-x-log(1+x^2))*(x>0)*(x<1) }

for (i in 1:1000){
  x = runif(N)      #using f0
  Est0[i] = mean(g(x))

  u = runif(N)      #using f1: Exp(1)
  x = -log(u)
  Est1[i] = mean( g(x)/exp(-x) )

  u = runif(N)      #using f2: Cauchy
  x = tan(pi*(u-0.5))
  ind.overflow = c(which(x > 1), which(x < 0))
  x[ind.overflow] = 2 #to catch overflow errors in g(x)
  Est2[i] = mean(g(x)/dcauchy(x))

  u = runif(N)      #using f3: truncated Exp(1)
  x = - log(1-u*(1-exp(-1)))
  Est3[i] = mean( g(x)/(exp(-x)/(1-exp(-1))) )

  u = runif(N)      #using f4: truncated Cauchy
  x = tan(pi*u/4)
  Est4[i] = mean( g(x)/(4/((1 + x^2)*pi)) )
}

```

Example 5.10 of Rizzo, Cont'd

```
> c(var(Est1)/var(Est0), var(Est2)/var(Est0), var(Est3)/var(Est0), var(Est4)/var(Est0))  
[1] 2.7992634 14.4089281 0.1589998 0.3136715
```

- f_3 and possibly f_4 produce smallest variance, while f_2 produces the highest variance.
- f_1 and f_2 do not reduce error; f_1 and f_2 are inefficient

The importance function should be a function that is supported on exactly the set where $|g(x)| > 0$, and such that the ratio $g(x)/f(x)$ is nearly constant.

Outline

- 1 Importance Sampling
- 2 Stratified Sampling**
- 3 Stratified Importance Sampling
- 4 Antithetic Sampling
- 5 Control Variates

Stratified Sampling

Suppose we want to evaluate

$$I = \int_0^1 g(x) dx. \quad (1)$$

1 Simple MC estimator:

$$\hat{I} = \frac{1}{2N} \sum_{i=1}^{2N} g(X_i), \quad X_i \stackrel{iid}{\sim} U(0, 1) \quad (2)$$

2 $I = \int_0^{1/2} g(x) dx + \int_{1/2}^1 g(x) dx \doteq \theta_1 + \theta_2$. Define

$$\hat{\theta}_1 = \frac{1}{2N} \sum_{i=1}^N g(Y_i), \quad Y_i \stackrel{iid}{\sim} U(0, 0.5)$$

$$\hat{\theta}_2 = \frac{1}{2N} \sum_{i=1}^N g(Z_i), \quad Z_i \stackrel{iid}{\sim} U(0.5, 1)$$

and $\{Y_i\}$ are independent of $\{Z_i\}$. Then $E(\hat{\theta}_1 + \hat{\theta}_2) = I$ but $\text{Var}(\hat{\theta}_1 + \hat{\theta}_2) < \text{Var}\hat{I}$ if $\theta_1 \neq \theta_2$. [Why?]

3 $\hat{\theta}_1 + \hat{\theta}_2$ is called stratified estimator.

Example 5.10 of Rizzo, Cont'd

Evaluate $I = \int_0^1 \frac{e^{-x}}{1+x^2} dx$.

```
g <- function(x) {  
  exp(-x - log(1+x^2)) * (x > 0) * (x < 1)  
}
```

```
N <- 1000  
T2 <- numeric(2)  
estimates <- matrix(0, 10, 2)  
for (i in 1:10) {  
  estimates[i, 1] <- mean(g(runif(N)))  
  T2[1] <- mean(g(runif(N/2, 0, .5)))  
  T2[2] <- mean(g(runif(N/2, .5, 1)))  
  estimates[i, 2] <- mean(T2)  
}
```

```
> apply(estimates, 2, mean)  
[1] 0.5232837 0.5248492  
> apply(estimates, 2, var)  
[1] 8.677316e-05 3.648629e-05
```

Stratified Sampling

To evaluate $I = \int g(x)dx$,

- Divide the interval into strata and estimate the integral on each of the stratum with smaller variance.
- The sum of these estimates converges to I w.p.1.
- The number of replicates m and number of replicates m_j to be drawn from each of k strata are fixed so that $m = m_1 + \dots + m_k$, with the goal that

$$\text{Var}(\hat{I}_k) < \text{Var}(\hat{I})$$

where $\hat{I}_k$ is the stratified estimator and $\hat{I}$ is the standard MC estimator based on m replicates.

Stratified Sampling

Consider $I = \int_0^1 g(x)dx$.

1 $I = \text{E}g(U)$, $U \sim U(0, 1)$. Simple MC estimator:

$$\hat{I}_M = N^{-1} \sum_{i=1}^N g(U_i), \quad U_i \stackrel{iid}{\sim} U(0, 1). \quad (3)$$

2 $I = \sum_{j=1}^k I_j$, where $I_j = \int_{(j-1)/k}^{j/k} g(x)dx = \frac{1}{k} \text{E}g(X_j)$, and $X_j \sim U((j-1)/k, j/k)$. Let $m = N/k$ and define

$$\hat{I}_j = \frac{1}{km} \sum_{i=1}^m g(X_{j,i}), \quad X_{j,i} \stackrel{iid}{\sim} U((j-1)/k, j/k) \quad (4)$$

3 $\text{Var}(\hat{I}_1 + \dots + \hat{I}_k) \leq \text{Var}(\hat{I}_M)$. The inequality is strict except in the case where all the strata have identical means.

Sketch of proof

1 Note that

$$\text{Var}(\hat{I}_j) = \frac{1}{N} \left[\int_{(j-1)/k}^{j/k} g(x)^2 dx - k I_j^2 \right] \quad (5)$$

Thus

$$\text{Var}(\hat{I}_1 + \dots + \hat{I}_k) = \frac{1}{N} \left[\int_0^1 g(x)^2 dx - k \sum_{j=1}^k I_j^2 \right] \quad (6)$$

2 Note also that

$$\text{Var}(\hat{I}_M) = \frac{1}{N} \left[\int_0^1 g(x)^2 dx - I^2 \right] \quad (7)$$

3 $\text{Var}(\hat{I}_1 + \dots + \hat{I}_k) \leq \text{Var}(\hat{I}_M)$ because

$$\left(\sum_{j=1}^k I_j \right)^2 \leq k \sum_{j=1}^k I_j^2 \quad (8)$$

More reduction in variance is achieved using stratification when the means of the strata are widely dispersed than if the means of the strata are approximately equal; For integrands that are monotone functions, stratification should be an effective way to reduce variance.

Outline

- 1 Importance Sampling
- 2 Stratified Sampling
- 3 Stratified Importance Sampling**
- 4 Antithetic Sampling
- 5 Control Variates

Stratified Importance Sampling

- A modification to the importance sampling method of estimating $I = \int_{-\infty}^{\infty} g(x)dx$.
- Recall Importance Sampling method: $\hat{I} = N^{-1} \sum_{i=1}^N \frac{g(X_i)}{f(X_i)}$ where X_i has pdf f and CDF F .
- Write I as

$$I = \int_{-\infty}^{a_1} g(x)dx + \int_{a_1}^{a_2} g(x)dx + \dots + \int_{a_{k-1}}^{\infty} g(x)dx \quad (9)$$

$$= \sum_{j=1}^k \int_{a_{j-1}}^{a_j} g(x)dx \quad (10)$$

where $a_j = F^{-1}(j/k)$ for $j = 1, \dots, k-1$, $a_0 = -\infty$ and $a_k = \infty$. That is, the real line is divided into intervals corresponding to equal areas $1/k$ under f .

- Let $g_j(x) = g(x)1_{(a_{j-1} \leq x < a_j)}$ and $\theta_j = \int_{a_{j-1}}^{a_j} g_j(x)dx$.

Stratified Importance Sampling

- Importance sampling estimator of θ_j : $\hat{\theta}_j = N^{-1} \sum_{i=1}^N \frac{g_j(X_{j,i})}{f_j(X_{j,i})}$, where $f_j(x) = kf(x)$ for $a_{j-1} \leq x < a_j$ and $\{X_{j,i}\}$ has pdf $f_j()$.
- Stratified Importance Sampling Estimator: $\hat{I}^{SI} = \sum_{j=1}^k \hat{\theta}_j$.
 $E(\hat{I}^{SI}) = I$; $\text{Var}(\hat{I}^{SI}) = \sum_{j=1}^k N^{-1} \text{Var} \left(\frac{g_j(X_j)}{f_j(X_j)} \right)$, $X_j \sim f_j$
- Consider Importance Sampling Estimator:

$$\hat{I} = (kN)^{-1} \sum_{i=1}^{kN} \frac{g(Y_i)}{f(Y_i)}, \quad Y_i \sim f \quad (11)$$

$$\text{Var}(\hat{I}) = (kN)^{-1} \text{Var} \left(\frac{g(Y)}{f(Y)} \right), \quad Y \sim f.$$

- $\text{Var}(\hat{I}^{SI}) < \text{Var}(\hat{I})$, because

$$\text{Var} \left(\frac{g_j(X_j)}{f_j(X_j)} \right) = k^{-1} \int_{a_{j-1}}^{a_j} \frac{g(x)^2}{f(x)} dx - \theta_j^2$$

$$\text{and } k \sum_{j=1}^k \theta_j^2 > (\sum_{j=1}^k \theta_j)^2 = I^2$$

Stratified Importance Sampling

In Example 5.10 of Rizzo, $I = \int_0^1 \frac{e^{-x}}{1+x^2} dx$, the best result was obtained with importance function $f(x) = \frac{e^{-x}}{1-e^{-1}}$, for $0 < x < 1$.

Now divide $(0,1)$ into 5 subintervals, (a_{j-1}, a_j) , $j = 1, \dots, 5$.
On the j th subinterval, $\{X_{j,1}, \dots, X_{j,N}\}$ are independently generated from

$$f_j(x) = \frac{5e^{-x}}{1 - e^{-1}}, \quad a_{j-1} < x < a_j \quad (12)$$

for $j = 1, \dots, 5$. The stratified importance sampling estimator:

$$\hat{I}^{SI} = \sum_{j=1}^5 \hat{\theta}_j \quad (13)$$

where $\hat{\theta}_j = N^{-1} \sum_{i=1}^N \frac{g_j(X_{j,i})}{f_j(X_{j,i})}$.

Outline

- 1 Importance Sampling
- 2 Stratified Sampling
- 3 Stratified Importance Sampling
- 4 Antithetic Sampling**
- 5 Control Variates

Antithetic Sampling

So far, we've assumed independent X_i . Can we somehow use dependence?

If X and Y each are unbiased estimates of a parameter θ , with variance σ^2

$$\text{Var}\left(\frac{X + Y}{2}\right) = \frac{\sigma^2}{2} + \frac{1}{2}\text{Cov}(X, Y). \quad (14)$$

- If X and Y are independent, then $\text{Cov}(X, Y) = 0$.
- If $\text{Cov}(X, Y) < 0$, the variance of their average is *less* than if they were independent.

Antithetic sampling: generate pairs (X_i, Y_i) so that $g(X_i)$ and $g(Y_i)$ are negatively correlated.

Antithetic Sampling

Suppose we want to evaluate

$$I = \int_0^1 g(x) dx. \quad (15)$$

Consider N independent pairs $\{(X_1, Y_1), \dots, (X_N, Y_N)\}$, where $X_i, Y_i \sim U(0, 1)$ and $\text{cov}(g(X_i), g(Y_i)) < 0$. Define

$$\hat{I} = \frac{1}{N} \sum_{i=1}^N \frac{g(X_i) + g(Y_i)}{2}. \quad (16)$$

Then

$$\text{Var}(\hat{I}) = \frac{\sigma^2}{2N} [1 + \text{cor}(g(X_i), g(Y_i))]. \quad (17)$$

where $\text{Var}(g(X_i)) = \text{Var}(g(Y_i)) = \sigma^2$. But how are we to achieve this?

Antithetic Sampling

- Suppose that $g(x)$ is monotone increasing on $[0, 1]$;
- Find x^* so that $g(1 - x^*) = I$;
- Take $X \sim U(0, 1)$, and consider

$$\text{cov}(g(X), g(1 - X)) \quad (18)$$

$$= E[(g(X) - I)(g(1 - X) - I)] \quad (19)$$

$$= E[g(X)(g(1 - X) - I)] \quad (20)$$

$$< g(x^*) \int_0^{x^*} (g(1 - x) - I) dx + g(x^*) \int_{x^*}^1 (g(1 - x) - I) dx \quad (21)$$

$$= 0, \quad (22)$$

because $\int_0^1 (g(1 - x) - I) dx = \int_0^1 (g(x) - I) dx = 0$.

- By symmetry the same result also holds for g decreasing.

Antithetic Sampling

- Suppose that g is monotone.
- To evaluate $I = \int_0^1 g(x)dx$ using antithetic sampling approach, generate N independent random variables $\{X_1, \dots, X_N\}$ from $U(0, 1)$ and define

$$\hat{I} = \frac{1}{N} \sum_{i=1}^N \frac{g(X_i) + g(1 - X_i)}{2}. \quad (23)$$

with

$$\text{Var}(\hat{I}) = \frac{\sigma^2}{2N} [1 + \text{cor}(g(X_i), g(1 - X_i))] < \frac{\sigma^2}{2N}. \quad (24)$$

and $\text{Var}(g(X_i)) = \text{Var}(g(1 - X_i)) = \sigma^2$.

Example

Back to $\int_0^1 (1 - x^2) dx$:

```
N = 500
for(i in 1:1000){
  X = runif(2*N)
  Est1[i] = mean( 1 - X^2 ) # Simple MC integration

  X = rbeta(2*N,1,2) # Beta
  Est2[i] = mean( 0.5*(1+X) ) # Importance sampling

  X = runif(N)
  Est3[i] = mean( c(1-X^2, 1-(1-X)^2) ) # Antithetic
}

> c(var(Est1),var(Est2),var(Est3))
[1] 9.048686e-05 1.395101e-05 1.130340e-05
```

More Generally

Two requirements

- 1 g must be monotone; otherwise we can have covariance positive.
 - Break range of integration into pieces where g is monotone.
 - Integrate on each piece separately with antithetic sampling.
- 2 X_i should be uniform; means X_i and $1 - X_i$ have the same distribution.
 - Same idea can be carried out with an inverse transform: want to evaluate

$$I = Eg(X)$$

and X has pdf f and CDF F .

If U_i is uniform, both $F^{-1}(U_i)$ and $F^{-1}(1 - U_i)$ have cdf F and $\text{cov}(g(F^{-1}(U_i)), g(F^{-1}(1 - U_i))) < 0$ if g is monotone.

CDF of $N(0,1)$

Consider the CDF of $N(0,1)$:

$$\Phi(x) = \int_{-\infty}^x \phi(t)dt = \int_{-\infty}^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt \quad (25)$$

$$\mathbf{1} \quad x > 0: \Phi(x) = 0.5 + \int_0^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt$$

$$\mathbf{2} \quad x < 0: \Phi(x) = 0.5 - \int_0^{-x} \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt = 1 - \Phi(-x).$$

Evaluate $I = \int_0^x e^{-t^2/2} dt$ for $x > 0$.

$$\mathbf{1} \quad I = \int_0^1 x e^{-(xu)^2/2} du$$

$$\mathbf{2} \quad \text{Thus, } I = E(xe^{-(xU)^2/2}) \text{ where } U \sim U(0,1).$$

$$\mathbf{3} \quad \text{Let } g(u) = x e^{-(xu)^2/2} \text{ for } 0 \leq u \leq 1. \quad g(u) \text{ is monotone.}$$

CDF of $N(0,1)$

Evaluate $I = E(xe^{-(xU)^2/2})$ for $x > 0$.

- 1 Generate $u_1, \dots, u_N \sim U(0, 1)$
- 2 Compute $g(u_j) = xe^{-(xu_j)^2/2}$, $g(1 - u_j) = xe^{-(x(1-u_j))^2/2}$.
- 3 Define $\hat{I} = N^{-1} \sum_{j=1}^N \frac{g(u_j) + g(1-u_j)}{2}$.

```
MC.Phi <- function(x, R = 10000, antithetic = TRUE) {
  ## MC.Phi computes CDF of N(0,1), i.e., Phi(x) for x>0,
  ## with or without antithetic sampling.
  ## By M. Rizzo
  u <- runif(R/2)
  if (!antithetic) v <- runif(R/2) else
    v <- 1 - u
  u <- c(u, v)
  cdf <- numeric(length(x))
  for (i in 1:length(x)) {
    g <- x[i] * exp(-(u * x[i])^2 / 2)
    cdf[i] <- 0.5 + mean(g) / sqrt(2 * pi)
  }
  cdf
}
```

CDF of $N(0,1)$

Compare the simple Monte Carlo integration approach and the antithetic sampling approach:

```
x <- seq(.1, 2.5, length=5)
Phi <- pnorm(x)
set.seed(123)
MC1 <- MC.Phi(x, anti = FALSE)
set.seed(123)
MC2 <- MC.Phi(x)

> print(round(rbind(x, Phi, MC1, MC2), 5))
      [,1]      [,2]      [,3]      [,4]      [,5]
x    0.10000  0.70000  1.30000  1.90000  2.50000
Phi  0.53983  0.75804  0.90320  0.97128  0.99379
MC1  0.53983  0.75825  0.90418  0.97311  0.99594
MC2  0.53983  0.75805  0.90325  0.97132  0.99370
```


CDF of $N(0,1)$

Estimate the approximate reduction in variance for given $x > 0$:

```
m <- 1000
MC1 <- MC2 <- numeric(m)
x <- 1.95
for (i in 1:m) {
  MC1[i] <- MC.Phi(x, R = 1000, anti = FALSE)
  MC2[i] <- MC.Phi(x, R = 1000)
}

> c(sd(MC1), sd(MC2))
[1] 0.006938491 0.000444236
> 100*((var(MC1) - var(MC2))/var(MC1))
[1] 99.59008
```

The antithetic sampling approach achieved approximately 99.6% reduction in variance at $x = 1.95$!

Outline

- 1 Importance Sampling
- 2 Stratified Sampling
- 3 Stratified Importance Sampling
- 4 Antithetic Sampling
- 5 Control Variates**

Control Variates

An alternative to antithetic variables is the use of control variates.

The basic idea is to use one variable Y with known mean μ to control another variable X with unknown mean. In particular, we define the “controlled” version of X

$$X^* = X - \alpha(Y - \mu) \quad (26)$$

so $EX^* = E(X)$, but

$$\text{var}(X^*) = \text{var}(X) + \alpha^2 \text{var}(Y) - 2\alpha \text{cov}(X, Y) \quad (27)$$

which is minimized at $\alpha = \text{cov}(X, Y)/\text{var}(Y)$. The minimum value of $\text{var}(X^*)$ is $\text{var}(X)(1 - \rho^2)$. The percent reduction in variance is $100\rho^2$ if $\rho \neq 0$.

Here Y is a *control variate*: we use correlation between X and Y , and that we know its expectation, to obtain X^* with smaller variance.

Control Variates

Strategy: we want $I_g = \mathbb{E}g(\mathbf{X})$

- 1 We have estimator $\hat{I}_g = \frac{1}{N} \sum_{i=1}^N g(\mathbf{X}_i)$;
- 2 We know $I_h = \mathbb{E}h(\mathbf{X})$ so form $\hat{I}_h = \frac{1}{N} \sum_{i=1}^N h(\mathbf{X}_i)$ *using the same $\mathbf{X}_i$* ;
- 3 Now make the corrected estimator $\hat{I}_g^* = \hat{I}_g - \alpha(\hat{I}_h - I_h)$.
- 4 Note: we could apply importance sampling to both $\hat{I}_g$ and $\hat{I}_h$.

We know how $\hat{I}_h$ differs from what it should be; use that correlation to improve $\hat{I}_g$.

What about α ? Use sample correlation

$$\alpha = \frac{\text{cov}(\hat{I}_g, \hat{I}_h)}{\text{var}(\hat{I}_h)} = \frac{\text{cov}(g(\mathbf{X}), h(\mathbf{X}))}{\text{var}h(\mathbf{X})} \approx \frac{\sum_{i=1}^N (g(\mathbf{X}_i) - \hat{I}_g)(h(\mathbf{X}_i) - \hat{I}_h)}{\sum_{i=1}^N (h(\mathbf{X}_i) - \hat{I}_h)^2}$$

Example

Compute $I = \int_0^1 e^u du$. Two approaches:

1 Simple MC approach:

$I = Ee^U$, $U \sim U(0, 1)$. Consider $\hat{I}_{MC} = N^{-1} \sum_{j=1}^N e^{U_j}$, $U_j \sim U(0, 1)$.

$$N\text{Var}(\hat{I}_{MC}) = \text{Var}(e^U) = (e^2 - 1)/2 - (e - 1)^2 = 0.242.$$

2 Control Variate:

Let $I_h = \int_0^1 u du = .5$ and form $\hat{I}_h = \frac{1}{N} \sum_{j=1}^N U_j$.

The corrected estimator $\hat{I}^* = \hat{I}_{MC} - \alpha(\hat{I}_h - 0.5)$, where

$$\alpha = \frac{\text{COV}(\hat{I}_{MC}, \hat{I}_h)}{\text{var}(\hat{I}_h)} \stackrel{\text{Why?}}{=} \frac{\text{COV}(e^U, U)}{\text{Var}(U)} = 18 - 6e = 1.69.$$

```
set.seed(123)                                > mean(T1)
m <- 10000                                    [1] 1.713146
a <- 18 - 6*exp(1)                            > mean(T2)
U <- runif(m)                                 [1] 1.717288
T1 <- exp(U)                                  > (var(T1)-var(T2))/var(T1)
T2 <- exp(U) - a*(U-1/2)                     [1] 0.9835984
```

Example 5.10 of Rizzo, Cont'd

Back to $I = \int_0^1 \frac{e^{-x}}{1+x^2} dx$.

1 $I = Eg(U)$ where $g(x) = \frac{e^{-x}}{1+x^2}$, $U \sim U(0, 1)$.

$\hat{I}_{MC} = N^{-1} \sum_{j=1}^N g(U_j)$ where $U_j \sim U(0, 1)$.

2 Let $I_h = \int_0^1 e^{-u} du = 1 - e^{-1}$ and form $\hat{I}_h = \frac{1}{N} \sum_{j=1}^N e^{-U_j}$.

3 The corrected estimator $\hat{I}^* = \hat{I}_{MC} - \alpha(\hat{I}_h - (1 - e^{-1}))$, where

$$\alpha = \frac{\text{COV}(g(U), h(U))}{\text{Var}h(U)}.$$

```
g = function(u) exp(-u)/(1+u^2)
h = function(u) exp(-u)

set.seed(1)
u = runif(1000)
a = cov(g(u), h(u))/var(h(u))
cor(g(u), h(u)) #0.9995116

Est0 = Est5 = numeric(1000)
for (i in 1:1000){
  u = runif(1000)
  Est0[i] = mean(g(u)) # MC
  Est5[i] = mean(g(u)) - a*(mean(h(u)) - (1-exp(-1)))
}
> round(var(Est5)/var(Est0), 6)
[1] 0.000998
```

Example

Estimate $\int_0^1 (1+x)^{-1} dx$, and use that we know $\int_0^1 (1+x) dx = 3/2$.

```
X = runif(1000)
gx = 1/(1+X)    # Integral we want to estimate
hx = (1+X)      # Integral that we know

> alpha = cov(gx,hx)/var(hx)    # Correction factor
[1] 0.4759076

# Corrected Estimates
> I2 = mean(gx) - alpha*(mean(hx)-3/2)
[1] 0.6971686
```

Variances:

MC	Antithetic	Control Variate
1.91e-05	1.23e-06	5.79e-07

Choices of Control Variate

- We'd like to maximize correlation between $g(X_i)$ and $h(X_i)$ as much as possible.
- We'd also like to choose $h(x)$ to be as close to $g(x)$ as we can, and $\int h(x)dx$ is known.

Normal probability example $\int_0^1 e^{-x^2/2} dx$:

- $h_1(x)$: $e^{-x^2/2} \approx 1 - x^2/2$ and $\int_0^1 (1 - x^2/2) dx = 5/6$.
 $\hat{I}_1 = \frac{1}{N} \sum_{j=1}^N h_1(X_j)$ where $X_j \sim U(0, 1)$.
- $h_2(x) = 1 + x$ and $\int_0^1 (1 + x) dx = 1.5$.
 $\hat{I}_2 = \frac{1}{N} \sum_{j=1}^N h_2(X_j)$ where $X_j \sim U(0, 1)$.

Summary

- Treat integrals as expectations and just simulate.
- Importance sampling: attempt to simulate where you will see largest area.
- Stratified Sampling: divide the interval into strata of which the means are widely dispersed.
- Antithetic sampling: adding negatively correlated variables reduces variance.
- Control variates: if you can do something similar analytically, use this to reduce variance.
- Most useful in higher dimensions (but harder to do variance reduction)
- Project!

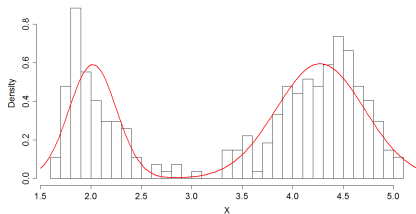
EM Algorithm for Gaussian Mixtures

Bishop Chapter 9, Givens&Hoeting Chapter 4

Fangfang Wang

Revisit the Old Faithful data

(Notes #3) Data on the duration of the eruption at the 'old faithful' geyser in Yellowstone National Park:



We can represent this as being (approximately) two normal peaks

$$f(x) = \underbrace{\pi_1 \frac{1}{\sqrt{2\pi}\sigma_1} e^{-(x-\mu_1)/2\sigma_1^2}}_{\phi_{\mu_1, \sigma_1}(x)} + \underbrace{\pi_2 \frac{1}{\sqrt{2\pi}\sigma_2} e^{-(x-\mu_2)/2\sigma_2^2}}_{\phi_{\mu_2, \sigma_2}(x)} \quad (1)$$

and $\pi_1 + \pi_2 = 1$. Here, π_1 and π_2 are referred to as the mixing weights/coefficients (see Notes #11).

Revist the Old Faithful data

Re-interpretation of the Old Faithful eruption data:

- Draw a binary random number Z with $P(Z = 1) = \pi_1$ and $P(Z = 0) = \pi_2 = 1 - \pi_1$
- If $Z = 1$, $X \sim N(\mu_1, \sigma_1^2)$
- If $Z = 0$, $X \sim N(\mu_2, \sigma_2^2)$.

We have $\{X_1, \dots, X_n\}$. Pretend there is also $\{Z_1, \dots, Z_n\}$ giving the component that X_i belonged to.

But we don't see $Z_1, \dots, Z_n$; they're latent (similar to random effects).

Strategy below: if we observed Z_i , things would be easy (just divide observations into groups and estimate parameters). We want to do something like that, but account for the fact that we don't know Z_i .

Likelihood With Latent Variables

The probability of observing X_i (without knowing Z_i) is

$$P(X_i) = P(X_i|Z_i = 1)P(Z_i = 1) + P(X_i|Z_i = 0)P(Z_i = 0) \quad (2)$$

$$= \pi_1 \phi_{\mu_1, \sigma_1}(X_i) + \pi_2 \phi_{\mu_2, \sigma_2}(X_i) \quad (3)$$

where $\pi_2 = 1 - \pi_1$.

There are 5 parameters $\theta = (\pi_1, \mu_1, \mu_2, \sigma_1, \sigma_2)^\top$ with log likelihood

$$\ell(\theta) = \sum_{i=1}^n \log \{ \pi_1 \phi_{\mu_1, \sigma_1}(X_i) + \pi_2 \phi_{\mu_2, \sigma_2}(X_i) \} \quad (4)$$

Must be maximized numerically.

Likelihood With Latent Variables

First, let's consider the distribution of Z_i *given* X_i :

$$p_i \doteq P(Z_i = 1 | X_i, \theta) \tag{5}$$

$$= \frac{P(X_i | Z_i = 1)P(Z_i = 1)}{P(X_i | Z_i = 1)P(Z_i = 1) + P(X_i | Z_i = 0)P(Z_i = 0)} \tag{6}$$

$$= \frac{\pi_1 \phi_{\mu_1, \sigma_1}(X_i)}{\pi_1 \phi_{\mu_1, \sigma_1}(X_i) + (1 - \pi_1) \phi_{\mu_2, \sigma_2}(X_i)} \tag{7}$$

- π_1 is the prior probability of $Z_i = 1$, and p_i is the corresponding posterior probability once we have observed X_i .
- p_i can also be viewed as the *responsibility* that 1st component takes for 'explaining' the observation X_i .

Likelihood With Latent Variables

Recall that

$$\ell(\theta) = \sum_{i=1}^n \log \{ \pi_1 \phi_{\mu_1, \sigma_1}(X_i) + (1 - \pi_1) \phi_{\mu_2, \sigma_2}(X_i) \} \quad (8)$$

1 Maximizing ℓ wrt μ_k ,

$$\mu_1 = \frac{\sum_{i=1}^n p_i X_i}{\sum_{i=1}^n p_i}, \quad \mu_2 = \frac{\sum_{i=1}^n (1 - p_i) X_i}{\sum_{i=1}^n (1 - p_i)} \quad (9)$$

2 Maximizing ℓ wrt σ_k^2 ,

$$\sigma_1^2 = \frac{\sum_{i=1}^n p_i (X_i - \mu_1)^2}{\sum_{i=1}^n p_i}, \quad \sigma_2^2 = \frac{\sum_{i=1}^n (1 - p_i) (X_i - \mu_2)^2}{\sum_{i=1}^n (1 - p_i)} \quad (10)$$

3 Maximizing ℓ wrt π_1 ,

$$\pi_1 = \frac{1}{n} \sum_{i=1}^n p_i \quad (11)$$

No closed-form solution!

Iterative Scheme ...

- Start with initial values θ_0 , tolerance $\epsilon > 0$, maxit
- While $\max |\theta_{k+1} - \theta_k| > \epsilon$ and $k < \text{maxit}$, iterate:
 - 1 Update probabilities

$$p_i = \frac{\pi_{1,k} \phi_{\mu_{1,k}, \sigma_{1,k}^2}(X_i)}{\pi_{1,k} \phi_{\mu_{1,k}, \sigma_{1,k}^2}(X_i) + (1 - \pi_{1,k}) \phi_{\mu_{2,k}, \sigma_{2,k}^2}(X_i)} \quad (12)$$

- 2 Update elements of θ_{k+1} :

$$\pi_{1,k+1} = \frac{1}{n} \sum_{i=1}^n p_i, \quad (13)$$

$$\mu_{1,k+1} = \frac{\sum_{i=1}^n p_i X_i}{\sum_{i=1}^n p_i}, \quad \sigma_{1,k+1}^2 = \frac{\sum_{i=1}^n p_i (X_i - \mu_{1,k+1})^2}{\sum_{i=1}^n p_i} \quad (14)$$

$$\mu_{2,k+1} = \frac{\sum_{i=1}^n (1 - p_i) X_i}{\sum_{i=1}^n (1 - p_i)}, \quad \sigma_{2,k+1}^2 = \frac{\sum_{i=1}^n (1 - p_i) (X_i - \mu_{2,k+1})^2}{\sum_{i=1}^n (1 - p_i)} \quad (15)$$

EM In Mixture Models

An elegant and powerful method for finding ML solutions for models with latent variables is called the expectation-maximization algorithm, or EM algorithm.

- We first choose some initial values for $\theta = (\pi_1, \mu_1, \mu_2, \sigma_1, \sigma_2)^\top$. Then we alternate between the following two updates that we shall call the E step and the M step.
- In the E step (or expectation step), we use the current values for θ to evaluate p_i , given by (12);
- Then in the M step (or maximization step), use these p_i s to re-estimate θ , using (13), (14), and (15).

Each update to θ resulting from an E step followed by an M step is guaranteed to increase the log likelihood.

In practice, the algorithm is deemed to have converged when the change in the log likelihood function, or alternatively in the parameters, falls below some threshold.

Application to Old Faithful

```
## EM Algorithm
EMGaussians = function(theta0, X) {
  den1 = dnorm(X, theta0[2], theta0[4])
  den2 = dnorm(X, theta0[3], theta0[5])

  Ep = theta0[1]*den1/(theta0[1]*den1+(1-theta0[1])*den2)
      # posterior

  p1 = mean(Ep)
  mu1 = sum(Ep*X)/sum(Ep)
  mu2 = sum((1-Ep)*X)/sum(1-Ep)
  sig1 = sqrt(sum(Ep*(X-mu1)^2)/sum(Ep))
  sig2 = sqrt(sum((1-Ep)*(X-mu2)^2)/sum(1-Ep))
  theta = c(p1, mu1, mu2, sig1, sig2)
  return(theta)
}
```

Application to Old Faithful

```
## EM Iterations
iterEM = function(theta, X, tol=1e-8, maxit=100) {
  tol.met = FALSE; iter = 0; iterhist = theta

  while(!tol.met){
    iter = iter + 1
    oldtheta = theta
    theta = EMGaussians(oldtheta, X)
    iterhist = rbind(iterhist,theta)
    if( max(abs(theta-oldtheta))<tol | iter>maxit)
      { tol.met=TRUE }
  }

  return( list(theta=theta,iter=iter,iterhist=iterhist) )
}
```

Application to Old Faithful

```
theta0 = c(0.5, 2.2, 4.5, 0.5, 0.5)
# initial vals: theta=(pi, mu1, mu2, sig1, sig2)
results = iterEM(theta0, faithful$eruptions)

> results$theta
[1] 0.3484046 2.0186078 4.2733434 0.2356218 0.4370631
> results$iter
[1] 32
```

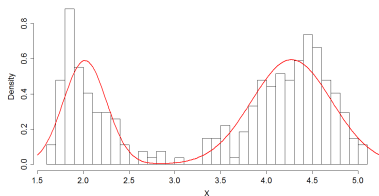
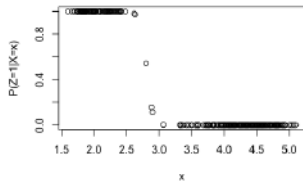
Agrees with results from `optim` to 2 decimal places (See Notes #9, page 34).

```
fn = function(theta,X){
  -sum( log( theta[1]*dnorm(X,mean=theta[2],sd=theta[4])+
    (1-theta[1])*dnorm(X,mean=theta[3],sd=theta[5])) )
}
res=optim(theta0,fn,method="Nelder-Mead",X=faithful$eruptions)
> res$par
[1] 0.3483709 2.0185460 4.2733486 0.2355770 0.4371153
```

Application to Old Faithful

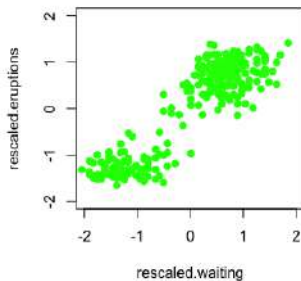
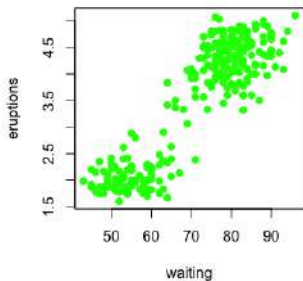
Notes #3

```
### Evaluate normal densities at observed data points
X = faithful$eruptions
dens1 = dnorm(X,results$theta[2],results$theta[4])
dens2 = dnorm(X,results$theta[3],results$theta[5])
# Use Bayes theorem to evaluate posterior
# probability that Z=1
pi.1 = results$theta[1]
PrZeq1 = pi.1*dens1/(pi.1*dens1+(1-pi.1)*dens2)
plot(X,PrZeq1, ylab="P(Z=1|X=x)", xlab="x")
```



EM for Gaussian mixtures

Re-scaled Old Faithful data set in a 2-d Euclidean space:



Mixtures of Gaussians

1 Gaussian mixture distribution

$$P(\mathbf{x}) = \sum_{j=1}^J \pi_j N(\mathbf{x} | \mu_j, \Sigma_j), \quad \mathbf{x} \in \mathbb{R}^d \quad (16)$$

with $\sum_{j=1}^J \pi_j = 1$.

2 Equivalent formulation of Gaussian mixture via latent variables

- For $\mathbf{x}_i \in \mathbb{R}^d$, introduce a random vector $\mathbf{Z}_i = (Z_{i,1}, \dots, Z_{i,J})^\top$, with $Z_{i,j} \in \{0, 1\}$ and $\sum_{j=1}^J Z_{i,j} = 1$.
 - $\mathbf{Z}_i$ describes the cluster assignment of $\mathbf{x}_i$.
 - If $\mathbf{x}_i$ is assigned to cluster k , then $Z_{i,k} = 1$ and $Z_{i,j} = 0$ for $j \neq k$.
- $P(Z_{i,j} = 1) = \pi_j$, $\sum_{j=1}^J \pi_j = 1$.
- Marginal distribution of $\mathbf{Z}_i$:

$$P(\mathbf{Z}_i = \mathbf{z}_i) = P(Z_{i,1} = z_{i,1}, \dots, Z_{i,J} = z_{i,J}) = \prod_{j=1}^J \pi_j^{z_{i,j}} \quad (17)$$

Mixtures of Gaussians

- 1 The conditional distribution of $\mathbf{X}_i$ given a particular value for $\mathbf{Z}_i$ is Gaussian

$$P(\mathbf{x}_i | \mathbf{Z}_i = \mathbf{z}_i) = \prod_{j=1}^J N(\mathbf{x}_i | \mu_j, \Sigma_j)^{z_{i,j}} \quad (18)$$

- 2 The joint distribution of $(\mathbf{X}_i, \mathbf{Z}_i)$:

$$P(\mathbf{x}_i, \mathbf{z}_i | \theta) = \prod_{j=1}^J \pi_j^{z_{i,j}} N(\mathbf{x}_i | \mu_j, \Sigma_j)^{z_{i,j}} \quad (19)$$

- 3 The distribution of $\mathbf{X}_i$

$$P(\mathbf{x}_i | \theta) = \sum_{\mathbf{z}_i} P(\mathbf{x}_i, \mathbf{z}_i | \theta) = \sum_{j=1}^J \pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j) \quad [\text{Why?}] \quad (20)$$

- 4 The log likelihood of $\mathbf{X}_1, \dots, \mathbf{X}_n$:

$$\ell(\theta) = \sum_{i=1}^n \log \left(\sum_{j=1}^J \pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j) \right) \quad (21)$$

Mixtures of Gaussians

- Gaussian mixture models are widely used in statistical analysis, machine learning, pattern recognition, and data mining.
- In many applications, their parameters are determined by maximum likelihood, typically using the EM algorithm.
- The conditional probability of $\mathbf{Z}_i = (Z_{i,1}, \dots, Z_{i,J})^\top$ given $\mathbf{x}_i$:

$$p_{i,k} = P(Z_{i,k} = 1 | \mathbf{x}_i) = \frac{P(Z_{i,k} = 1)P(\mathbf{x}_i | Z_{i,k} = 1)}{\sum_{j=1}^J P(Z_{i,j} = 1)P(\mathbf{x}_i | Z_{i,j} = 1)} \quad (22)$$

$$= \frac{\pi_k N(\mathbf{x}_i | \mu_k, \Sigma_k)}{\sum_{j=1}^J \pi_j N(\mathbf{x}_i | \mu_j, \Sigma_j)} \quad (23)$$

- $p_{i,k}$ can also be viewed as the responsibility that component k takes for ‘explaining’ the observation $\mathbf{x}_i$.

EM for Gaussian mixtures

- 1 Initialize the parameters and evaluate the initial values of the log likelihood.
- 2 **E step.** Evaluate the responsibilities $p_{i,j}, j = 1, \dots, J$, using the current parameter values
- 3 **M step.** Re-estimate the parameters using the current responsibilities

$$\pi_j^{\text{new}} = \frac{1}{n} \sum_{i=1}^n p_{i,j} \quad (24)$$

$$\mu_j^{\text{new}} = \frac{1}{\sum_{i=1}^n p_{i,j}} \sum_{i=1}^n p_{i,j} \mathbf{x}_i \quad (25)$$

$$\Sigma_j^{\text{new}} = \frac{1}{\sum_{i=1}^n p_{i,j}} \sum_{i=1}^n p_{i,j} (\mathbf{x}_i - \mu_j^{\text{new}})(\mathbf{x}_i - \mu_j^{\text{new}})^\top \quad (26)$$

for $j = 1, \dots, J$.

- 4 Evaluate the log likelihood and check for convergence of either the parameters or the log likelihood. If the convergence criterion is not satisfied, return to Step 2.

EM for Gaussian Mixtures: M-step

- Using responsibilities $p_{i,j} = P(Z_{i,j} = 1 \mid \mathbf{x}_i, \theta^{old})$, we obtain

$$Q(\theta) = \sum_{i=1}^n \sum_{j=1}^J p_{i,j} \log [\pi_j N(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)],$$

expected complete-data log-likelihood

M-step: Updating π_j

- Maximize

$$\sum_{i=1}^n \sum_{j=1}^J p_{i,j} \log \pi_j \quad \text{subject to} \quad \sum_{j=1}^J \pi_j = 1.$$

- Using Lagrange multipliers:

$$\pi_j^{new} = \frac{1}{n} \sum_{i=1}^n p_{i,j}.$$

M-step

Updating μ_j :

- Maximize

$$\sum_{i=1}^n p_{i,j} \log N(\mathbf{x}_i \mid \mu_j, \Sigma_j).$$

- Result:

$$\mu_j^{new} = \frac{\sum_{i=1}^n p_{i,j} \mathbf{x}_i}{\sum_{i=1}^n p_{i,j}}.$$

Updating Σ_j :

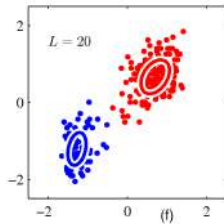
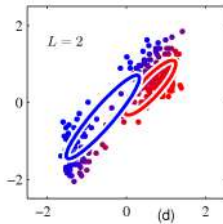
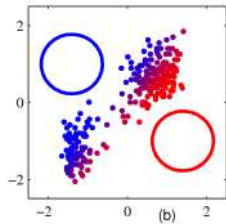
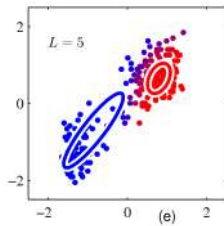
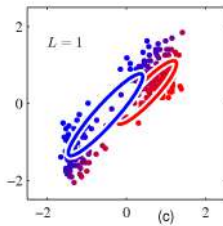
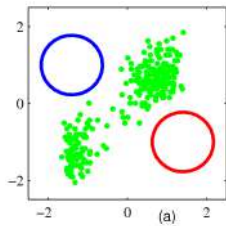
- Maximize

$$\sum_{i=1}^n p_{i,j} \log N(\mathbf{x}_i \mid \mu_j, \Sigma_j).$$

- Result:

$$\Sigma_j^{new} = \frac{\sum_{i=1}^n p_{i,j} (\mathbf{x}_i - \mu_j^{new})(\mathbf{x}_i - \mu_j^{new})^\top}{\sum_{i=1}^n p_{i,j}}.$$

Application to 'Old Faithful' Data



EM Algorithm

Bishop Chapter 9, Givens&Hoeting Chapter 4

Fangfang Wang

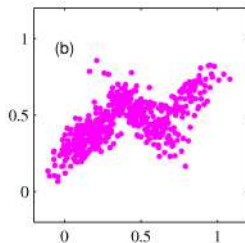
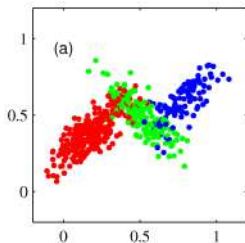
The General EM Algorithm

- Let $\mathbf{X}$ denote the observable data and θ the parameter to be estimated.
- The EM algorithm is particularly suited for problems in which there is a notion of “missing data”.
 - missing data: actual data that is missing, or some “imaginary” data that exists only in our minds (latent!).
- Let $\mathbf{Y}$ denote the complete data, composed of observable data $\mathbf{X}$ and missing data $\mathbf{Z}$; i.e., $\mathbf{Y} = (\mathbf{X}, \mathbf{Z})$.
 - This is the notation used in Givens & Hoeting which, as they admit, is not standard in the EM literature.
- The basic idea behind the EM algorithm is to iteratively impute the missing data.

The General EM Algorithm

500 points drawn from the mixture of 3 Gaussians in a 2-d space:

- The data set in Fig (a) is said to be complete, whereas that in Fig (b) is incomplete.
- In Fig (a), samples from the joint distribution $P(\mathbf{z})P(\mathbf{x}|\mathbf{z})$, in which the three states of $\mathbf{z}$ are depicted in red, green, and blue.
- In Fig (b), the corresponding samples from the marginal distribution $P(\mathbf{x})$



More notation

- $L_Y(\theta)$: complete-data likelihood, the joint distribution of $\mathbf{Y} = (\mathbf{X}, \mathbf{Z})$, $p(\mathbf{X}, \mathbf{Z}|\theta)$.
- $L_X(\theta)$: observed likelihood, obtained by marginalizing the joint distribution of $(\mathbf{X}, \mathbf{Z})$, $p(\mathbf{X}|\theta)$.
- $P(\mathbf{z}|\mathbf{X}, \theta)$: conditional distribution of $\mathbf{Z}$ given $\mathbf{X}$, an essential piece.
 - In practice, we are not given the complete data set, but only the incomplete data. Our state of knowledge of the values of $\mathbf{Z}$ is given only by $P(\mathbf{z}|\mathbf{X}, \theta)$.
- We shall suppose that direct optimization of $L_X(\theta)$ is difficult, but that optimization of $L_Y(\theta)$ is significantly easier.

The General EM Algorithm

- Because we cannot use $\log L_{\mathbf{Y}}(\theta)$, we consider its expected value under the posterior distribution of $\mathbf{Z}$.
- Define the conditional expectation of the complete-data log likelihood at θ , given $\mathbf{X}$ and θ' :

$$Q(\theta|\theta') = \mathbb{E}(\log L_{\mathbf{Y}}(\theta)|\mathbf{X}, \theta') = \int \log L_{\mathbf{Y}}(\theta) P(d\mathbf{z}|\mathbf{X}, \theta') \quad (1)$$

- The only “random” part of $\mathbf{Y}$ is $\mathbf{Z}$.
- The expectation is wrt $\mathbf{Z}$, given $\mathbf{X}$.
- E step: use the current parameter values θ^{old} to find $P(\mathbf{z}|\mathbf{X}, \theta^{old})$, then use this posterior distribution to find $Q(\theta|\theta^{old})$.
- In the subsequent M step, we maximize $Q(\theta|\theta^{old})$ to get a revised estimate θ^{new} .

The General EM Algorithm

The EM algorithm iterates computing $Q(\theta|\theta')$, which involves an expectation and then maximizing it.

- 1 Choose an initial setting for the parameters θ_0 .
- 2 At iteration $k \geq 1$ do:

E step Evaluate

$$Q(\theta|\theta_{k-1}) = \text{E}(\log L_{\mathbf{Y}}(\theta) | \mathbf{X}, \theta_{k-1}) \quad (2)$$

M step Update

$$\theta_k = \arg \max_{\theta} Q(\theta|\theta_{k-1}) \quad (3)$$

- 3 Repeat these steps until convergence is reached.

Example 1: Gaussian mixtures revisited

1 Gaussian mixture distribution

$$P(\mathbf{x}) = \sum_{j=1}^J \pi_j N(\mathbf{x}|\mu_j, \Sigma_j), \quad \mathbf{x} \in \mathbb{R}^d \quad (4)$$

with $\sum_{j=1}^J \pi_j = 1$.

2 Recall: the joint distribution of $(\mathbf{X}_i, \mathbf{Z}_i)$:

$$P(\mathbf{x}_i, \mathbf{z}_i|\theta) = \prod_{j=1}^J \pi_j^{z_{i,j}} N(\mathbf{x}_i|\mu_j, \Sigma_j)^{z_{i,j}}$$

3 $\log L_Y(\theta) = \sum_{i=1}^n \sum_{j=1}^J z_{i,j} \{\log \pi_j + \log N(\mathbf{x}_i|\mu_j, \Sigma_j)\}$

4 $P(\mathbf{z}|\mathbf{X}, \theta) \propto \prod_{i=1}^n \prod_{j=1}^J \pi_j^{z_{i,j}} N(\mathbf{x}_i|\mu_j, \Sigma_j)^{z_{i,j}}$

Example 1: Gaussian mixtures revisited

$$Q(\theta|\theta') = E(\log L_{\mathbf{Y}}(\theta) | \mathbf{X}, \theta') \quad (5)$$

$$= \sum_{i=1}^n \sum_{j=1}^J E(z_{i,j} \{\log \pi_j + \log N(\mathbf{x}_i | \mu_j, \Sigma_j)\} | \mathbf{X}, \theta') \quad (6)$$

$$= \sum_{i=1}^n \sum_{j=1}^J E(z_{i,j} | \mathbf{X}, \theta') \{\log \pi_j + \log N(\mathbf{x}_i | \mu_j, \Sigma_j)\} \quad (7)$$

$$= \sum_{i=1}^n \sum_{j=1}^J p_{i,j} \{\log \pi_j + \log N(\mathbf{x}_i | \mu_j, \Sigma_j)\} \quad (8)$$

where

$$p_{i,j} = E(Z_{i,j} | \mathbf{X}, \theta') = P(Z_{i,j} = 1 | \mathbf{x}_i, \theta') \quad (9)$$

$$= \frac{P(Z_{i,j} = 1)P(\mathbf{x}_i | Z_{i,j} = 1, \theta')}{\sum_{k=1}^J P(Z_{i,k} = 1)P(\mathbf{x}_i | Z_{i,k} = 1, \theta')} \quad (10)$$

$$= \frac{\pi_j N(\mathbf{x}_i | \mu'_j, \Sigma'_j)}{\sum_{k=1}^J \pi_k N(\mathbf{x}_i | \mu'_k, \Sigma'_k)}. \quad (11)$$

Example 2: Mixtures of Bernoulli distribution

- 1 Consider D independent binary variables X_i , $i = 1, 2, \dots, D$:
 $X_i \sim \text{Ber}(\mu_i)$, i.e.,

$$P(X_i = x_i) = \mu_i^{x_i} (1 - \mu_i)^{1-x_i} \quad (12)$$

- 2 Let $\mathbf{X} = (X_1, X_2, \dots, X_D)^\top$, $\mathbf{x} = (x_1, x_2, \dots, x_D)^\top$, and
 $\vec{\mu} = (\mu_1, \mu_2, \dots, \mu_D)^\top$.

- 3 Multivariate Bernoulli dist: the distribution of $\mathbf{X}$ is

$$p(\mathbf{x}|\vec{\mu}) = \prod_{i=1}^D \mu_i^{x_i} (1 - \mu_i)^{1-x_i}, \quad \mathbf{x} \in \mathbb{R}^D \quad (13)$$

- 4 $E(\mathbf{X}) = \vec{\mu}$ and $\text{Cov}(\mathbf{X}) = \text{diag}(\mu_1(1 - \mu_1), \dots, \mu_D(1 - \mu_D))$.

Example 2: Mixtures of Bernoulli distribution

- 1 Consider a finite mixture of multivariate Bernoulli distributions:

$$p(\mathbf{x}|\vec{\mu}, \pi) = \sum_{k=1}^K \pi_k p(\mathbf{x}|\vec{\mu}_k), \quad \mathbf{x} \in \mathbb{R}^D, \quad (14)$$

where $\vec{\mu} = \{\vec{\mu}_1, \dots, \vec{\mu}_K\}$, $\pi = \{\pi_1, \dots, \pi_K\}$, and

$$p(\mathbf{x}|\vec{\mu}_k) = \prod_{i=1}^D \mu_{k,i}^{x_i} (1 - \mu_{k,i})^{1-x_i} \quad (15)$$

- 2 $E(\mathbf{X}) = \sum_{k=1}^K \pi_k \vec{\mu}_k$
- 3 $\text{Cov}(\mathbf{X}) = \sum_{k=1}^K \pi_k (\Sigma_k + \vec{\mu}_k \vec{\mu}_k^\top) - E(\mathbf{X})E(\mathbf{X})^\top$, where $\Sigma_k = \text{diag}(\mu_{k,1}(1 - \mu_{k,1}), \dots, \mu_{k,D}(1 - \mu_{k,D}))$.
- 4 The mixture dist. can capture correlations between variables, because cov matrix is no longer diagonal.

Example 2: Mixtures of Bernoulli distribution

1 Given the data $\{\mathbf{x}_1, \dots, \mathbf{x}_N\} \in \mathbb{R}^D$, the log likelihood function is

$$\ell(\vec{\mu}, \pi | \mathbf{x}) = \log p(\mathbf{x}_1, \dots, \mathbf{x}_N | \vec{\mu}, \pi) \quad (16)$$

$$= \sum_{n=1}^N \log p(\mathbf{x}_n | \vec{\mu}, \pi) \quad (17)$$

$$= \sum_{n=1}^N \log \left\{ \sum_{k=1}^K \pi_k p(\mathbf{x}_n | \vec{\mu}_k) \right\} \quad (18)$$

2 MLE does not have closed form.

3 Use EM algorithm

Example 2: Mixtures of Bernoulli distribution

- 1 Recall that the EM algorithm iterates computing the conditional expectation of the complete-data log likelihood given X :

$$Q(\theta|\theta') = E_{\mathbf{Z}} (\log L_Y(\theta) | \mathbf{X}, \theta')$$

- 2 The complete-data likelihood $L_Y(\theta)$
- 3 The conditional distribution of $\mathbf{Z}$, given X (posterior distribution of the latent variables)
- 4 latent variables: $\mathbf{Z}$?

Example 2: Mixtures of Bernoulli distribution

- For each data point $\mathbf{x}_n \in \mathbb{R}^D$, introduce a random vector $\mathbf{Z}_n = (Z_{n,1}, \dots, Z_{n,K})^\top$ describing which of the K clusters $\mathbf{x}_n$ is assigned to:
 - If $\mathbf{x}_n$ is assigned to cluster k , then $Z_{n,k} = 1$ and $Z_{n,j} = 0$ for $j \neq k$.
 - $Z_{n,j} \in \{0, 1\}$ and $\sum_{j=1}^K Z_{n,j} = 1$.
 - $P(Z_{n,j} = 1) = \pi_j$, $\sum_{j=1}^K \pi_j = 1$.
- The marginal distribution of $\mathbf{Z}_n$ is

$$P(\mathbf{Z}_n = \mathbf{z}_n) = P(Z_{n,1} = z_{n,1}, \dots, Z_{n,K} = z_{n,K}) = \prod_{k=1}^K \pi_k^{z_{n,k}} \quad (19)$$

- The conditional distribution of $\mathbf{x}_n$ given a particular value for $\mathbf{Z}_n$ is

$$P(\mathbf{x}_n | \mathbf{Z}_n = \mathbf{z}_n, \vec{\mu}) = \prod_{k=1}^K p(\mathbf{x}_n | \vec{\mu}_k)^{z_{n,k}} \quad (20)$$

Example 2: Mixtures of Bernoulli distribution

The complete-data log likelihood

$$\ell(\vec{\mu}, \pi | \mathbf{x}, \mathbf{z}) = \sum_{n=1}^N \sum_{k=1}^K z_{n,k} \left\{ \log \pi_k + \sum_{i=1}^D [x_{ni} \log \mu_{ki} + (1 - x_{ni}) \log(1 - \mu_{ki})] \right\} \quad (21)$$

where $\mathbf{x} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \in \mathbb{R}^D$, $\mathbf{z} = \{\mathbf{z}_1, \dots, \mathbf{z}_N\} \in \mathbb{R}^K$.

Take expectation of $\ell(\vec{\mu}, \pi | \mathbf{x}, \mathbf{z})$ wrt the posterior dist of $\mathbf{Z} = \{\mathbf{Z}_1, \dots, \mathbf{Z}_N\}$ at θ^{old} :

$$\begin{aligned} & \mathbb{E}_{\mathbf{Z}} \left(\ell(\vec{\mu}, \pi | \mathbf{x}, \mathbf{z}) | \mathbf{x}, \theta^{old} \right) \\ &= \sum_{n=1}^N \sum_{k=1}^K \mathbb{E}(z_{n,k} | \mathbf{x}_n, \theta^{old}) \left\{ \log \pi_k + \sum_{i=1}^D [x_{ni} \log \mu_{ki} + (1 - x_{ni}) \log(1 - \mu_{ki})] \right\} \end{aligned} \quad (22)$$

where $\mathbb{E}(z_{n,k} | \mathbf{x}_n, \theta^{old})$ is the posterior prob, or responsibility, of component k given data point $\mathbf{x}_n$ and θ^{old} :

$$\mathbb{E}(z_{n,k} | \mathbf{x}_n, \theta^{old}) = P(z_{n,k} = 1 | \mathbf{x}_n, \theta^{old}) = \frac{\pi_k^{old} p(\mathbf{x}_n | \vec{\mu}_k^{old})}{\sum_{j=1}^K \pi_j^{old} p(\mathbf{x}_n | \vec{\mu}_j^{old})} \quad (23)$$

Example 2: Mixtures of Bernoulli distribution

- 1 In the E step: compute $E(z_{n,k} | \mathbf{x}_n, \theta^{old})$, $k = 1, \dots, K$, $n = 1, \dots, N$

$$E(z_{n,k} | \mathbf{x}_n, \theta^{old}) = \frac{\pi_k^{old} p(\mathbf{x}_n | \vec{\mu}_k^{old})}{\sum_{j=1}^K \pi_j^{old} p(\mathbf{x}_n | \vec{\mu}_j^{old})} \quad (24)$$

- 2 In the M step: $\max_{\vec{\mu}_1, \dots, \vec{\mu}_K, \pi_1, \dots, \pi_K} E_Z (\ell(\vec{\mu}, \pi | \mathbf{x}, \mathbf{z}) | \mathbf{x}, \theta^{old})$

- $\vec{\mu}_k^{new} = \frac{1}{N_k} \sum_{n=1}^N E(z_{n,k} | \mathbf{x}_n, \theta^{old}) \mathbf{x}_n$
- $\pi_k^{new} = \frac{N_k}{N}$

where $N_k = \sum_{n=1}^N E(z_{n,k} | \mathbf{x}_n, \theta^{old})$ (effective number of data points associated with component k).

Example 2: Mixtures of Bernoulli distribution

- 1 Model handwritten digits (from MNIST data set)
- 2 A dataset of $N = 600$ digits, consisting of '2', '3', and '4':



- 3 Convert each digit image (28×28 pixel) into a binary vector by setting all elements whose values exceed .5 to 1 and setting the remaining elements to 0.
- 4 Each pixel of a binary image has a Bernoulli dist.



All $D = 784$ pixels $X = (X_1, \dots, X_D)^\top$ together define a multivariate Bernoulli dist.

Example 2: Mixtures of Bernoulli distribution

- 1 Fit the dataset with a mixture of $K = 3$ multivariate Bernoulli distributions.
- 2 Initialization: $\pi_1 = \pi_2 = \pi_3 = 1/3$, and $\mu_{k,j} \sim U(0.25, 0.75)$ with $\sum_j \mu_{k,j} = 1$
- 3 After 10 iterations of EM algorithm, estimated $\mu_{k,j}$ for each of the 3 components in the mixture model:



Reshape the pixels into a 28x28 matrix

- 4 Using a single multivariate Bernoulli and maximum likelihood:



This amounts to averaging counts in each pixel.

Example 3: Multinomial Model

- Bernoulli dist
- Binomial dist
- Categorical dist: $X \in \{1, 2, \dots, k\}$, $P(X = i) = p_i$,
 $\sum_{i=1}^k p_i = 1$.
- Multinomial dist:
 - Suppose that there are n independent trials and each trial has k mutually exclusive outcomes with corresponding probabilities $p_1, \dots, p_k$. Let X_i = number of times outcome i is observed over the n trials, $i = 1, \dots, k$. Then the vector $X = (X_1, \dots, X_k)$ follows a multinomial dist with parameters n and $\pi = (p_1, \dots, p_k)$.
 - The probability mass function is:

$$P(X_1 = x_1, \dots, X_k = x_k | n, \pi) = \frac{n!}{x_1! \dots x_k!} p_1^{x_1} \dots p_k^{x_k},$$

when $\sum_{i=1}^k x_i = n$.

Example 3: Multinomial Model

- 1 Suppose that 197 animals are distributed multinomially into 4 categories with cell-probabilities

$$\pi_{\theta} = (1/2 + \theta/4, (1 - \theta)/4, (1 - \theta)/4, \theta/4)$$

for some unknown $\theta \in [0, 1]$. The observed number in each cell was $\mathbf{x} = (125, 18, 20, 34)$.

- 2 Log likelihood function is

$$\ell(\theta|\mathbf{x}) = C + x_1 \log(2 + \theta) + (x_2 + x_3) \log(1 - \theta) + x_4 \log(\theta) \quad (25)$$

- 3 Use EM algorithm:

- Assume the complete data is given by $\mathbf{Y} = (Y_{11}, Y_{12}, Y_2, Y_3, Y_4)$ and $\mathbf{Y} \sim \text{Multi}(n, \pi_{\theta}^*)$, where

$$\pi_{\theta}^* = (1/2, \theta/4, (1 - \theta)/4, (1 - \theta)/4, \theta/4)$$

- $\mathbf{X} = (Y_{11} + Y_{12}, Y_2, Y_3, Y_4)$ and $\mathbf{Z} = Y_{12}$.
- Log likelihood of $\mathbf{Y}$ is

$$\ell(\theta|\mathbf{x}, \mathbf{z}) = C + (y_{12} + y_4) \log(\theta) + (y_2 + y_3) \log(1 - \theta) \quad (26)$$

Example 3: Multinomial Model

- 1 Posterior dist of Y_{12} given $\mathbf{X}$:

$$P_{\theta}(Y_{12} = y_{12} | \mathbf{X} = \mathbf{x}) = \frac{P(Y_{12}=y_{12}, \mathbf{X}=\mathbf{x})}{P(\mathbf{X}=\mathbf{x})} = \frac{P(\mathbf{Y}=\mathbf{y})}{P(\mathbf{X}=\mathbf{x})} = C \frac{(1/2)^{y_{11}} (\theta/4)^{y_{12}}}{(1/2 + \theta/4)^{y_{11} + y_{12}}} =$$

$$C \left(\frac{2}{2+\theta} \right)^{x_1 - y_{12}} \left(\frac{\theta}{2+\theta} \right)^{y_{12}} \sim \text{Bin}(x_1, \frac{\theta}{2+\theta})$$

- 2 E-step:

$$\begin{aligned} Q(\theta | \theta^{old}) &= \mathbb{E}_{\mathbf{Z}} \left(\ell(\theta | \mathbf{x}, \mathbf{z}) | \mathbf{X}, \theta^{old} \right) \\ &= C + \mathbb{E}_{\mathbf{Z}} \left(Y_{12} | \mathbf{X}, \theta^{old} \right) \log(\theta) + (y_2 + y_3) \log(1 - \theta) + y_4 \log(\theta) \\ &= C + x_1 p^{old} \log(\theta) + (x_2 + x_3) \log(1 - \theta) + x_4 \log(\theta) \end{aligned}$$

where $p^{old} = \frac{\theta^{old}}{2 + \theta^{old}}$

- 3 M-step: maximize $Q(\theta | \theta^{old})$ to obtain θ^{new} :

$$\theta^{new} = \frac{p^{old} x_1 + x_4}{p^{old} x_1 + x_2 + x_3 + x_4}$$

Example 3: Multinomial Model

```
## EM Algorithm
EMmultinomial = function(oldtheta, X)
{
  x1 = X[1]; x2 = X[2]; x3 = X[3]; x4 = X[4]
  oldp = oldtheta/(2+oldtheta)
  theta = (oldp*x1+x4)/(oldp*x1+x2+x3+x4)
  return(theta)
}

#EM Iterations
EMiterMultinomial=function(theta, X, tol=1e-8, maxit=100)
{
  tol.met = FALSE; iter = 0
  while(!tol.met){
    iter = iter+1
    oldtheta = theta
    theta = EMmultinomial(oldtheta, X)
    if(abs(theta-oldtheta)<tol|iter>maxit) { tol.met=TRUE }
  }
  return( list(theta=theta,iter=iter))
}
```

Example 3: Multinomial Model

```
results = EmitterMultinomial(0.25, c(125, 18, 20, 34))
results$theta
results$iter
```

Recall that the Log likelihood function of $\mathbf{x}$ is

$$\ell(\theta|\mathbf{x}) = C + x_1 \log(2 + \theta) + (x_2 + x_3) \log(1 - \theta) + x_4 \log(\theta) \quad (27)$$

```
# Golden section search
llk = function(theta){
  X = c(125, 18, 20, 34) # data
  x1 = X[1]; x2 = X[2]; x3 = X[3]; x4 = X[4]
  llk = x1*log(2+theta)+(x2+x3)*log(1-theta)+x4*log(theta)
  return(llk)
}

thetas = seq(0.01, 0.99, 0.01)
plot(thetas, llk(thetas), type='l')
source('gsection.r')
gsection(llk, 0.4, 0.9, 0.7)
```

EM algorithm for Exponential Family Models

- When Y are modeled to have a distribution in the exponential family, the density of Y can be written as

$$f(\mathbf{y}|\theta) = C(\mathbf{y})a(\theta) \exp\{\theta^\top s(\mathbf{y})\} \quad (28)$$

where θ is a vector of natural parameters and $s(\mathbf{y})$ is sufficient statistic.

- The log-likelihood of Y is

$$\log L_Y(\theta) = C + \log a(\theta) + \theta^\top s(\mathbf{y}) \quad (29)$$

- For problems where Y is modeled as an exponential family, EM algorithm takes a relatively simple form.

EM algorithm for Exponential Family Models

- For exponential families, Q function becomes:

$$\begin{aligned} Q(\theta|\theta^{old}) &= \mathbb{E}_{\mathbf{Z}} \left(\log L_{\mathbf{Y}}(\theta) | \mathbf{X}, \theta^{old} \right) \\ &= C + \log a(\theta) + \theta^{\top} \mathbb{E}_{\mathbf{Z}} \left(s(\mathbf{Y}) | \mathbf{X}, \theta^{old} \right). \end{aligned}$$

- To maximize this, take derivative wrt θ and set to zero:

$$-\frac{a'(\theta)}{a(\theta)} = \mathbb{E}_{\mathbf{Z}} \left(s(\mathbf{Y}) | \mathbf{X}, \theta^{old} \right) \quad (30)$$

- Note that $-a'(\theta) = a(\theta)\mathbb{E}(s(\mathbf{Y})|\theta)$.
- Let s^{old} be the RHS.
- M step: obtain θ^{new} by solving

$$\mathbb{E}(s(\mathbf{Y})|\theta) = s^{old} \quad (31)$$

EM algorithm for Exponential Family Models

- E step: compute

$$s^{old} = E_Z (s(\mathbf{Y})|\mathbf{X}, \theta^{old}) \quad (32)$$

- M step: obtain θ^{new} by solving

$$E(s(\mathbf{Y})|\theta) = s^{old} \quad (33)$$

Example 4: Censored Exponential Model

- Complete data $Y_1, \dots, Y_N \sim \text{iid } \text{Exp}(\theta)$.
- Complete data log-likelihood

$$\log L_Y(\theta) = N \log \theta - \theta \sum_{i=1}^N Y_i \quad (34)$$

where $s(\mathbf{Y}) = \sum_{i=1}^N Y_i$.

- Suppose some observations are right-censored.
 - 1 Right censoring occurs when a subject leaves the study before an event occurs, or the study ends before the event has occurred.
 - 2 Example: consider patients in a clinical trial to study the effect of treatments on stroke occurrence. The study ends after 5 years. Those patients who have had no strokes by the end of the 5th year are censored. If the patient leaves the study at time c , then the event occurs in (c, ∞) .

Example 4: Censored Exponential Model

- Write observed data as pairs $X_i = (X_i^*, \delta_i)$, $i = 1, 2, \dots, N$, where

$$X_i^* = \min(Y_i, c_i), \quad \delta_i = \begin{cases} 1 & \text{if } Y_i \leq c_i \\ 0 & \text{if } Y_i > c_i \end{cases} \quad (35)$$

and c_i are the censoring levels.

- Example: consider the clinical trial to study the effect of treatments on stroke occurrence.
Each patient has their own zero time, the time at which the patient entered the study (accrual time). For each patient, record time to event or censoring time, whichever is the smaller, and $\delta_i = 1$ if the event occurs and $\delta_i = 0$ if the patient is censored.
- Missing data Z consists of the actual event times for the censored observations.

Example 4: Censored Exponential Model

E step requires

$$s^{old} = \mathbb{E}_{\mathbf{Z}} \left(s(\mathbf{Y}) | \mathbf{X}, \theta^{old} \right) = \sum_{i=1}^N \mathbb{E}_{\mathbf{Z}_i} (Y_i | \mathbf{X}_i, \theta^{old}) \quad (36)$$

$$= \sum_{i=1}^N \left[\delta_i Y_i + (1 - \delta_i) \mathbb{E}_{\mathbf{Z}_i} (Y_i | Y_i > c_i, \theta^{old}) \right] \quad (37)$$

$$= \sum_{i=1}^N \left[\delta_i Y_i + (1 - \delta_i) (c_i + 1/\theta^{old}) \right] \quad (38)$$

$$= \sum_{i=1}^N (\delta_i Y_i + (1 - \delta_i) c_i) + \frac{1}{\theta^{old}} \sum_{i=1}^N (1 - \delta_i) \quad (39)$$

$$= N\bar{X}^* + \frac{1}{\theta^{old}} \sum_{i=1}^N (1 - \delta_i) \quad (40)$$

Example 4: Censored Exponential Model

Recall:

- E step: compute s^{old}
- M step: obtain θ^{new} by solving $E(s(\mathbf{Y})|\theta) = s^{old}$, and
- $E(s(\mathbf{Y})|\theta) = N/\theta$.

The EM update in this case is

$$\theta^{new} = \frac{N}{s^{old}} \quad (41)$$

$$= \left\{ \bar{X}^* + \frac{1}{\theta^{old}N} \sum_{i=1}^N (1 - \delta_i) \right\}^{-1} \quad (42)$$

i.e.,

$$\theta_{k+1} = \left\{ \bar{X}^* + \frac{1}{\theta_k N} \sum_{i=1}^N (1 - \delta_i) \right\}^{-1}. \quad (43)$$

Bayesian Inference and Computation

Rizzo Chapter 9

Fangfang Wang

Outline

- 1 Basics of Bayesian Inference
- 2 MCMC Integration
- 3 Metropolis-Hastings Algorithm

Outline

- 1 Basics of Bayesian Inference
- 2 MCMC Integration
- 3 Metropolis-Hastings Algorithm

Basic probability laws

- *Joint probability* of two events, A and B :

$$p(A \cap B) = p(A)p(B|A) = p(A|B)p(B)$$

- *Total probability*: Let $\bar{B}$ denote the complement of B . Then

$$\begin{aligned} p(A) &= p(A \cap B) + p(A \cap \bar{B}) \\ &= p(A|B)p(B) + p(A|\bar{B})p(\bar{B}) \end{aligned}$$

- *Bayes rule*:

$$p(B|A) = \frac{p(A \cap B)}{p(A)} = \frac{p(A|B)p(B)}{p(A|B)p(B) + p(A|\bar{B})p(\bar{B})}$$

Bayesian paradigm

■ Basic notation

y = data (observable)

θ = parameter (not observable)

$p(y|\theta)$ = data distribution

$p(\theta)$ = prior distribution

$p(\theta|y)$ = posterior distribution

■ Joint distribution

$$p(y, \theta) = p(y|\theta)p(\theta) = p(y)p(\theta|y)$$

Bayesian paradigm

- Inference about θ based on posterior distribution

$$p(\theta|y) = \frac{p(y, \theta)}{p(y)} = \frac{p(y|\theta)p(\theta)}{p(y)}$$

- $p(y)$ is the marginal distribution of the data.
- After observing y it is the normalizing constant for the posterior

$$p(\theta|y) \propto p(y|\theta)p(\theta)$$

Classical (frequentist) paradigm

- No prior distribution
- θ is a fixed unknown parameter
- $p(y|\theta)$ is called the **likelihood** when regarded as a function of θ
- Estimate θ by the value $\hat{\theta}$ that maximizes the likelihood
- Inference about θ based on repeated sample properties of $\hat{\theta}$

Example 1: single binomial

- Suppose Y is the number of successes in n Bernoulli trials, each with success probability θ , i.e., $Y \sim \text{Bin}(n, \theta)$

$$p(y|\theta) = p(Y = y|\theta) = \binom{n}{y} \theta^y (1 - \theta)^{n-y} \quad y = 0, 1, \dots, n$$

- Beta prior distribution: $\theta \sim \text{Beta}(\alpha, \beta)$

$$p(\theta) = \frac{1}{B(\alpha, \beta)} \theta^{\alpha-1} (1 - \theta)^{\beta-1} \quad 0 < \theta < 1$$

where $B(\alpha, \beta)$ is the normalizing constant for the beta distribution

- α, β : **hyperparameters**, i.e., the parameters of the prior distribution

Example 1: single binomial

- **Conjugate prior:** posterior is in the same family as the prior

$$\theta|Y = y \sim \text{Beta}(y + \alpha, n - y + \beta)$$

- If posterior dist is in the same prob dist family as prior prob dist, prior and posterior are then called conjugate dists, and prior is called a conjugate prior for likelihood function.
- Mean of $\text{Beta}(a, b)$ is $a/(a + b)$
- **Bayes estimate:** Posterior mean of θ

$$E(\theta|y) = \frac{y + \alpha}{n + \alpha + \beta} \quad (1)$$

$$= \frac{n}{n + \alpha + \beta} \left(\frac{y}{n} \right) + \frac{\alpha + \beta}{n + \alpha + \beta} \left(\frac{\alpha}{\alpha + \beta} \right) \quad (2)$$

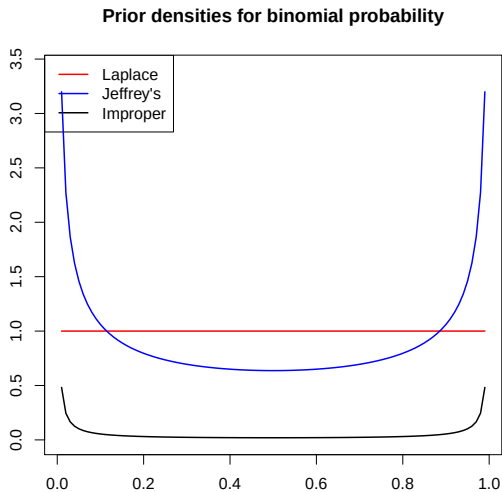
- MLE: $\hat{\theta} = y/n$, the sample proportion of successes.
- Weighted average of the usual frequentist estimator (sample mean) and the prior mean.

Example 1: single binomial

Choice of hyperparameters:

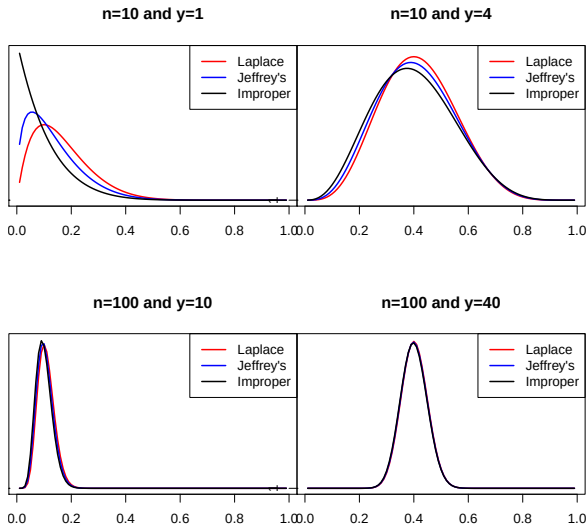
- **Laplace prior** ($\alpha = \beta = 1$): $p(\theta)$ is the uniform distribution on $(0,1)$.
- **Jeffreys prior** ($\alpha = \beta = 1/2$): $p(\theta) \propto \sqrt{I(\theta)}$.
- **Improper prior** ($\alpha = \beta = 0$): $p(\theta) \propto \theta^{-1}(1 - \theta)^{-1}$, which is improper.

Example 1: single binomial



Example 1: single binomial

Posterior distributions:



Posterior inference

- The **Bayes estimate** of the parameter θ is the posterior mean, $E(\theta|y)$. Alternatively use the posterior median or mode.
- Uncertainty about the parameter is measured using a **posterior (credible) interval**
e.g. an interval that contains 95% of the posterior density.
- Typically this is done by reporting the 0.025 and 0.975 quantiles of the posterior density.
- Alternatively, one could report the 95% **highest posterior density (HPD) interval**.

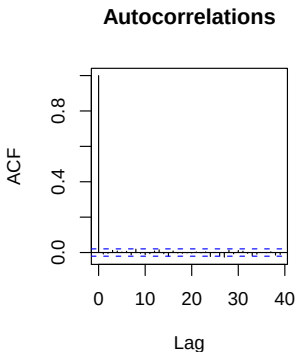
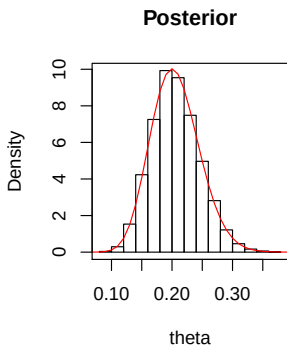
Sampling from the posterior

- Suppose it is possible to simulate from the posterior distribution.
- Let $\theta_1, \dots, \theta_N$ denote a random sample from the posterior distribution, $p(\theta|y)$.
- Let $\theta_{(i)}$ denote the i th order statistic.
- If $N = 1000$, approximate 95% posterior credible interval is

$$(\theta_{(25)}, \theta_{(975)})$$

Example 1: Independent sampling from the posterior

Posterior for binomial probability with Laplace prior ($\alpha = \beta = 1$)
after observing $y = 20$ successes in $n = 100$ trials



Markov chain Monte Carlo (MCMC)

- Usually it isn't possible to draw independent samples from the posterior
- Typically Bayesian inference is based on simulating a **Markov chain** of parameter values whose limiting marginal distribution is equal to $p(\theta|y)$. Denote by

$$\theta_1, \theta_2, \dots, \theta_N, \dots$$

Outline

- 1 Basics of Bayesian Inference
- 2 MCMC Integration
- 3 Metropolis-Hastings Algorithm

Why Do We Need MCMC?

- In many statistical problems, we want to compute expectations of the form

$$E[g(X)] = \int g(x)f(x) dx.$$

- In Bayesian inference, we often need

$$E[g(\theta) \mid x] = \int g(\theta)f(\theta \mid x) d\theta.$$

- These integrals are often:
 - high-dimensional,
 - analytically intractable,
 - difficult for direct numerical integration.
- Monte Carlo idea: approximate the integral using simulated samples.

Monte Carlo Integration Review

If $X_1, \dots, X_m \stackrel{iid}{\sim} f$, then

$$\bar{g} = \frac{1}{m} \sum_{i=1}^m g(X_i)$$

is an estimator of

$$E[g(X)] = \int g(x)f(x) dx.$$

Problem:

- What if we *cannot* sample directly from f ?

Solution:

- Construct a **Markov chain** whose stationary distribution is f .
- Then use the simulated chain values to approximate the expectation.

Key Idea of MCMC

MCMC Principle

Construct a Markov chain

$$X_0, X_1, X_2, \dots$$

such that its **stationary distribution** is the target density $\pi(x)$.

Then, under suitable conditions,

$$\frac{1}{m} \sum_{t=1}^m g(X_t) \xrightarrow{a.s.} E_{\pi}[g(X)].$$

Important difference from ordinary Monte Carlo:

- Samples are **dependent**, not iid.
- The dependence comes from the Markov chain structure.

Why MCMC Is Especially Useful in Bayesian Inference

Suppose

- observed data: $x = (x_1, \dots, x_n)$
- parameter: θ
- prior density: $f_\theta(\theta)$
- likelihood: $f(x \mid \theta)$

Then the posterior is

$$f(\theta \mid x) = \frac{f(x \mid \theta)f_\theta(\theta)}{\int f(x \mid \theta)f_\theta(\theta) d\theta}.$$

We often want

$$E[g(\theta) \mid x] = \int g(\theta)f(\theta \mid x) d\theta.$$

Challenge: the normalizing constant in the denominator is usually hard to compute.

Posterior Expectation in a More General Form

Many Bayesian quantities reduce to

$$E[g(Y)] = \frac{\int g(t)\pi(t) dt}{\int \pi(t) dt},$$

where $\pi(t)$ is known only up to a proportionality constant.

Key advantage of MCMC:

- We often do *not* need the normalizing constant.
- We only need a function proportional to the target density.

This is one of the main reasons MCMC is so powerful in Bayesian computation.

MCMC Estimator

Suppose $\{X_t\}$ is a Markov chain with stationary distribution π . Then the MCMC estimator of $E_\pi[g(X)]$ is

$$\bar{g}_m = \frac{1}{m} \sum_{t=1}^m g(X_t).$$

Under irreducibility, aperiodicity, and ergodicity,

$$\bar{g}_m \xrightarrow{a.s.} E_\pi[g(X)].$$

Interpretation:

- A long run of the chain behaves like a sample from π .

Ordinary Monte Carlo vs MCMC

	Ordinary MC	MCMC
Sample type	iid	dependent
Need direct sampler?	Yes	No
Target density known up to constant?	Usually no	Yes
Main difficulty	simulation from f	chain design/convergence

Tradeoff:

- MCMC is more flexible,
- but dependence introduces convergence and efficiency issues.

Outline

- 1 Basics of Bayesian Inference
- 2 MCMC Integration
- 3 Metropolis-Hastings Algorithm**

Metropolis-Hastings algorithm

- The Metropolis-Hastings algorithm is a class of MCMC methods.
- Special cases: the Metropolis sampler, the Gibbs sampler, the independence sampler, and the random walk.
- Main idea: generate a Markov Chain $\{x_t, t = 0, 1, 2, \dots\}$ such that its stationary distribution is the target distribution.
- The algorithm specifies, for a given state x_t , how to generate the next state x_{t+1} .
- A candidate point y is generated from a proposal distribution $g(\cdot|x_t)$.
 - 1 If y is **accepted**, the chain moves to state y at time $t + 1$ and $x_{t+1} = y$;
 - 2 Otherwise, the chain stays in state x_t and $x_{t+1} = x_t$.

Metropolis-Hastings algorithm

Goal: sample from a target density $\pi(x)$.

- Proposal or candidate density: g
- The Metropolis-Hastings sampler generates the Markov chain $\{x_0, x_1, \dots\}$ as follows:
 - 1 Generate x_0 from g .
 - 2 At iteration t , given current state x_t
 - 1 Draw $y \sim g(\cdot | x_t)$
 - 2 Draw $u \sim U(0, 1)$
 - 3 Calculate MH-ratio

$$R(x_t, y) = \frac{\pi(y)/\pi(x_t)}{g(y | x_t)/g(x_t | y)}$$

- 4 If $u \leq R(x_t, y)$, accept y and set $x_{t+1} = y$; otherwise set $x_{t+1} = x_t$.

Acceptance Probability

The Metropolis–Hastings acceptance probability is

$$\alpha(x_t, y) = \min \left(1, \frac{\pi(y)g(x_t | y)}{\pi(x_t)g(y | x_t)} \right).$$

Then:

- With probability $\alpha(x_t, y)$, accept and set

$$x_{t+1} = y.$$

- Otherwise reject and keep

$$x_{t+1} = x_t.$$

Important: only the ratio of $\pi(\cdot)$ is needed; π is the stationary distribution of the chain. (See Rizzo, Page 248)

Interpretation of the Acceptance Rule

Suppose we move from x to y .

- If

$$\frac{\pi(y)g(x | y)}{\pi(x)g(y | x)} > 1,$$

then accept with probability 1.

- Otherwise, accept with probability equal to the ratio.

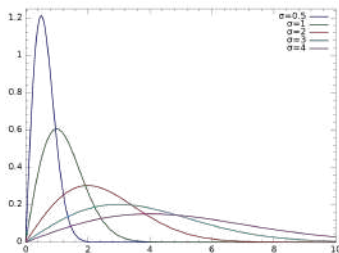
Interpretation:

- The algorithm favors moves toward higher-probability regions.
- But it still occasionally accepts lower-probability moves.
- This prevents the chain from getting stuck too easily.

Example 2: Generate Rayleigh random variables

The Rayleigh density is

$$f(x) = \frac{x}{\sigma^2} e^{-x^2/(2\sigma^2)}, \quad x \geq 0, \sigma > 0. \quad (3)$$



■ $E(X) = \sigma\sqrt{\pi/2}, \text{Var}(X) = \sigma^2(4 - \pi)/2, \text{mode} = \sigma$

Example 2: Generate Rayleigh random variables

- 1 Set $g(\cdot|x)$ to the density of $\chi^2(x)$.
- 2 Generate x_0 from $\chi^2(1)$.
- 3 Repeat for $t = 0, \dots, N$,
 - 1 Draw $y \sim \chi^2(x_t)$
 - 2 Draw $u \sim U(0, 1)$
 - 3 Calculate MH-ratio

$$R(x_t, y) = \frac{f(y)/f(x_t)}{g(y|x_t)/g(x_t|y)}$$

where $g(y|x_t)$ is the $\chi^2(x_t)$ density evaluated at y , and $g(x_t|y)$ is the $\chi^2(y)$ density evaluated at x_t .

- 4 If $u \leq R(x_t, y)$, accept y and set $x_{t+1} = y$; otherwise set $x_{t+1} = x_t$.
- 5 Increment t .

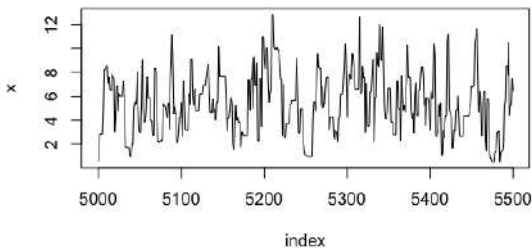
Example 2: Rayleigh distribution for $\sigma = 4$

```
# Evaluate the Rayleigh density.
f <- function(x, sigma) {
  if (any(x < 0)) return (0)
  stopifnot(sigma > 0)
  return((x/sigma^2)*exp(-x^2/(2*sigma^2)))
}

sigma = 4; N = 10000;
x = numeric(N)
x[1] = rchisq(1, df=1); u = runif(N)
k = 0
for (i in 2:N) {
  xt = x[i-1]
  y = rchisq(1, df = xt)
  num = f(y, sigma)/f(xt, sigma)
  den = dchisq(y, df = xt)/dchisq(xt, df = y)
  if (u[i] <= num/den) x[i] = y else {
    x[i] = xt
    k = k+1 #y is rejected
  }
}
```

Example 2: Rayleigh distribution for $\sigma = 4$

```
> print(k)
[1] 4136
> index <- 5000:5500
> # Display a partial plot starting at time index 5000
> y1 = x[index]
> plot(index, y1, type="l", main="", ylab="x")
```



Interpreting the Output

Typical observations:

- The chain may reject a substantial number of proposals.
- Repeated values in the chain correspond to rejected proposals.

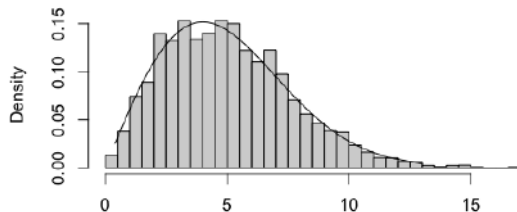
Visual diagnostics:

- Trace plot
- Histogram with target density overlaid
- QQ plot

These help assess whether the chain appears to be sampling from the desired target distribution.

Example 2: Rayleigh distribution for $\sigma = 4$

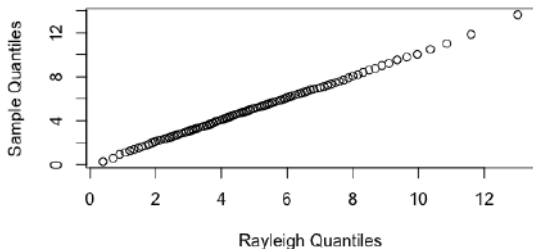
The histogram of the generated sample with the Rayleigh density superimposed



Example 2: Rayleigh distribution for $\sigma = 4$

- Quantile-quantile plot: compares the quantiles of the target Rayleigh distribution with the quantiles of the generated chain
- The quantiles of the Rayleigh distribution are given by

$$x_q = F^{-1}(q) = \sigma \{-2 \log(1 - q)\}^{1/2}, \quad 0 < q < 1.$$



- Use other method to generate Rayleigh random variable?

Example 2: Rayleigh distribution for $\sigma = 4$

```
q = ppoints(100) # seq. of prob. points
xq = sigma * sqrt(-2 * log(1 - q)) # Rayleigh quantiles
Q = quantile(x, q) # Sample quantiles

b = 2001 #discard the burnin sample
y = x[b:N]

hist(y, breaks="scott", main="", xlab="", freq=FALSE)
lines(xq, f(xq, 4))

qqplot(xq, Q, main="",
        xlab="Rayleigh Quantiles", ylab="Sample Quantiles")
```

Example 3: Beta-binomial model

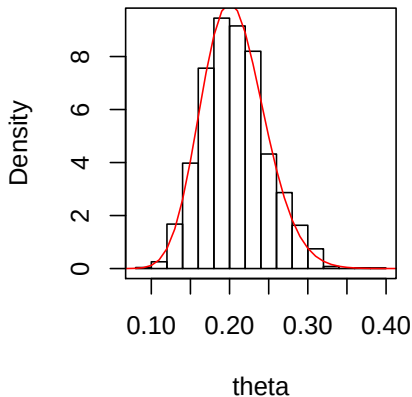
- Posterior: $\theta|Y = y \sim \text{Beta}(A, B)$ with $A = y + \alpha$ and $B = n - y + \beta$
- Try a uniform candidate density: $g(x) = 1, 0 < x < 1$
- Starting value: $\theta_0 = 0.5$.
- $N = 10,000$, burn-in = 1000
- Repeat for $t = 0, \dots, N$,
 - 1 Draw $x \sim U(0, 1), u \sim U(0, 1)$
 - 2 Calculate MH-ratio

$$R(\theta_t, x) = \frac{f(x)/f(\theta_t)}{g(x|\theta_t)/g(\theta_t|x)} = \frac{x^{A-1}(1-x)^{B-1}}{\theta_t^{A-1}(1-\theta_t)^{B-1}} \quad (4)$$

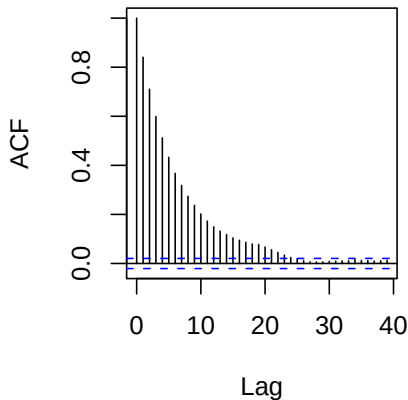
- 3 If $u \leq R(\theta_t, x)$, set $\theta_{t+1} = x$; otherwise set $\theta_{t+1} = \theta_t$.
- 4 Increment t .

Example 3: MCMC sampling from the posterior

Posterior



Autocorrelations



Markov Chain Monte Carlo Methods

Rizzo Chapter 9

Fangfang Wang

Outline

- 1 Random Walk Metropolis
- 2 The Independence Sampler
- 3 Gibbs Sampler
- 4 Application: change point analysis

Outline

- 1 Random Walk Metropolis
- 2 The Independence Sampler
- 3 Gibbs Sampler
- 4 Application: change point analysis

Metropolis Sampler

A special case occurs when the proposal density is **symmetric**:

$$g(y | x) = g(x | y).$$

Then the proposal terms cancel, and the acceptance probability simplifies to

$$\alpha(x, y) = \min \left(1, \frac{\pi(y)}{\pi(x)} \right).$$

This simpler version is called the **Metropolis sampler**.

Random Walk Metropolis

A very common choice is

$$Y = X_t + Z,$$

where Z is a random increment from a symmetric distribution.

Example:

$$Y \mid X_t \sim N(X_t, \sigma^2).$$

Since this proposal is symmetric,

$$\alpha(X_t, Y) = \min \left(1, \frac{\pi(Y)}{\pi(X_t)} \right).$$

This is called the **Random Walk Metropolis** algorithm. This is an example of a Metropolis sampler.

Why Proposal Scale Matters

In Random Walk Metropolis, the proposal variance controls performance.

If proposal variance is too small:

- almost all proposals are accepted,
- but the chain moves only in tiny steps,
- mixing is very slow.

If proposal variance is too large:

- many proposals are rejected,
- the chain gets stuck often.

We need a **reasonable balance**.

Rule of Thumb for Acceptance Rate

For one-dimensional random walk Metropolis:

- an acceptance rate roughly in the range

0.15 to 0.50

is often considered reasonable.

Interpretation:

- Too high $\Rightarrow$ moves are too small.
- Too low $\Rightarrow$ moves are too ambitious.

This is not a strict rule, but a useful practical guideline.

Example 1: Student t Target Distribution

Suppose the target is Student t_ν with density

$$f(x) \propto \left(1 + \frac{x^2}{\nu}\right)^{-(\nu+1)/2}.$$

Use a random walk proposal:

$$Y \mid X_t \sim N(X_t, \sigma^2).$$

Then the acceptance ratio is

$$R(x_t, y) = \frac{f(y)}{f(x_t)} = \frac{(1 + y^2/\nu)^{-(\nu+1)/2}}{(1 + x_t^2/\nu)^{-(\nu+1)/2}}.$$

Example 1

```
rw.Metropolis <- function(n, sigma, x0, N) {  
  # n: degrees of freedom  
  # N: length of the chain  
  x <- numeric(N)  
  x[1] <- x0  
  u <- runif(N)  
  k <- 0  
  
  for (i in 2:N) {  
    y <- rnorm(1, x[i-1], sigma)  
  
    if (u[i] <= (dt(y, n) / dt(x[i-1], n)))  
      x[i] <- y  
    else {  
      x[i] <- x[i-1]  
      k <- k + 1  
    }  
  }  
  
  return(list(x=x, k=k))  
}
```

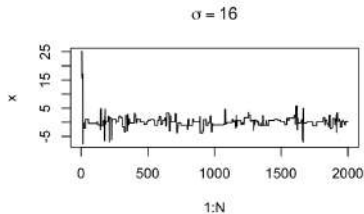
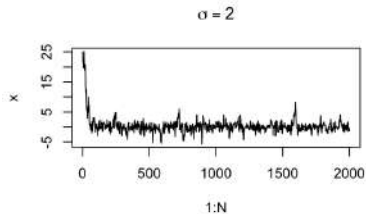
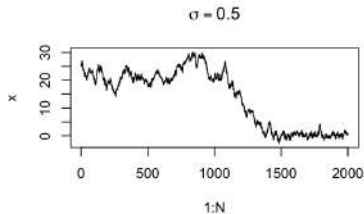
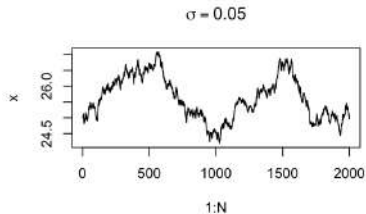
Example 1: Different Proposal Scales

```
n <- 4
N <- 2000
sigma <- c(.05, .5, 2, 16)
x0 <- 25

rw1 <- rw.Metropolis(n, sigma[1], x0, N)
rw2 <- rw.Metropolis(n, sigma[2], x0, N)
rw3 <- rw.Metropolis(n, sigma[3], x0, N)
rw4 <- rw.Metropolis(n, sigma[4], x0, N)

> # Number of candidate points rejected:
> print(c(rw1$k, rw2$k, rw3$k, rw4$k))
[1] 10 161 898 1800
> # Rejection rate:
> print(c(rw1$k, rw2$k, rw3$k, rw4$k)/N)
[1] 0.0050 0.0805 0.4490 0.9000
```

Example 1



Interpretation of Proposal Scale Experiment

Case 1: very small proposal variance

- acceptance rate is very high,
- chain moves too slowly,
- behaves almost like a random walk.

Case 2: very large proposal variance

- many rejections,
- chain often stays in the same place.

Best case: moderate proposal variance

- good balance between movement and acceptance.

Takeaway from Random Walk Metropolis

MCMC performance depends strongly on **tuning**.

In particular:

- A correct algorithm can still be very inefficient.
- Convergence in theory does not imply good practical performance.
- Tuning the proposal is essential.

Outline

- 1 Random Walk Metropolis
- 2 The Independence Sampler
- 3 Gibbs Sampler
- 4 Application: change point analysis

Independence Sampler

Another special case of Metropolis–Hastings is the independence sampler.

The proposal density does *not* depend on the current state:

$$g(Y \mid X_t) = g(Y).$$

Hence

$$\alpha(X_t, Y) = \min \left(1, \frac{\pi(Y)g(X_t)}{\pi(X_t)g(Y)} \right).$$

Key point:

- easy to implement;
- works well when g is close to the target density;
- can mix poorly when the proposal is badly matched to the target.

Example 3: Estimating a Mixture Weight

Observed data come from a two-component normal mixture

$$pN(\mu_1, \sigma_1^2) + (1 - p)N(\mu_2, \sigma_2^2),$$

where $\mu_1, \mu_2, \sigma_1^2, \sigma_2^2$ are known and the unknown parameter is $p \in (0, 1)$. The mixture density is

$$f^*(z) = pf_1(z) + (1 - p)f_2(z).$$

The proposal distribution should be supported on the set of valid probabilities p , so use a Beta proposal

$$Y \sim \text{Beta}(a, b).$$

With proposal density g , the acceptance rule is

$$\alpha(X_t, Y) = \min \left(1, \frac{\pi(Y)g(X_t)}{\pi(X_t)g(Y)} \right).$$

Example 3

```
m = 5000 #length of chain
xt = numeric(m)
a = 1; b = 1 #parameter of Beta(a,b) proposal dist
n = 30 #sample size
p = 0.2; mu = c(0, 5); sigma = c(1, 1)

# observed sample from the mixture
i = sample(1:2, size=n, replace=TRUE, prob=c(p, 1-p))
x = rnorm(n, mu[i], sigma[i])
```

```
# generate the independence sampler chain
u = runif(m)
y = rbeta(m, a, b) #proposal distribution
xt[1] = 0.5

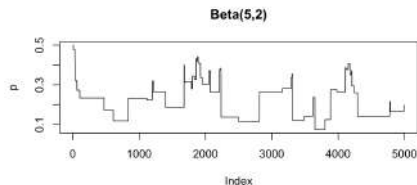
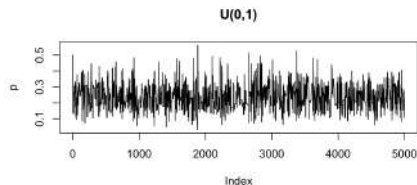
for (i in 2:m) {
  fy = y[i]*dnorm(x, mu[1], sigma[1]) +
    (1-y[i])*dnorm(x, mu[2], sigma[2])
  fx = xt[i-1]*dnorm(x, mu[1], sigma[1]) +
    (1-xt[i-1])*dnorm(x, mu[2], sigma[2])

  r = prod(fy/fx) *
    (xt[i-1]^(a-1) * (1-xt[i-1])^(b-1)) / (y[i]^(a-1) * (1-y[i])^(b-1))

  if (u[i] <= r) xt[i] = y[i] else
    xt[i] = xt[i-1]
}

plot(xt, type="l", ylab="p")
hist(xt[101:m], main="", xlab="p", prob=TRUE)
print(mean(xt[101:m]))
```

Uniform Proposal vs Beta(5,2) Proposal



Two proposals:

- $U(0,1)$ (or $\text{Beta}(1,1)$)
- $\text{Beta}(5,2)$

Main conclusion:

- Uniform mixes well and converges quickly
- $\text{Beta}(5,2)$ produces a much less efficient chain
- Proposal shape matters: a poor global proposal can slow the chain even when the algorithm is theoretically correct

Outline

- 1 Random Walk Metropolis
- 2 The Independence Sampler
- 3 Gibbs Sampler**
- 4 Application: change point analysis

Gibbs Sampler

- The Gibbs sampler is another special case of Metropolis–Hastings.
- Gibbs sampler is often applied when the target is a multivariate distribution.
- Suppose that all the univariate conditional densities are fully specified and it is reasonably easy to sample from them.
- The chain is generated by sampling from the marginal distributions of the target distribution, and every candidate point is accepted.

Gibbs Sampler

- Let $\mathbf{X} = (X_1, \dots, X_d)'$ be a random vector in $\mathbb{R}^d$, with density $f(\mathbf{x}) = f(x_1, \dots, x_d)$ (Target density)
- Define the $d - 1$ dimensional random vectors:

$$\mathbf{X}_{-j} = (X_1, \dots, X_{j-1}, X_{j+1}, \dots, X_d)'$$

- Define the univariate conditional density of X_j given $\mathbf{X}_{-j} = \mathbf{x}_{-j}$:

$$f(x_j | \mathbf{x}_{-j})$$

- The Gibbs sampler generates the chain by sampling from each of the d conditional densities $f(x_j | \mathbf{x}_{-j})$.

Gibbs Sampler

- 1 Initialize $\mathbf{X}(0)$ at time $t = 0$.
- 2 For each iteration, indexed $t = 1, 2, \dots$, repeat:
 - 1 Set $\mathbf{x} = \mathbf{X}(t - 1)$.
 - 2 For each coordinate $j = 1, \dots, d$,
 - 1 Generate $X_j^*(t) \sim f(\cdot | \mathbf{x}_{-j})$
 - 2 Update $x_j = X_j^*(t)$
 - 3 Set $\mathbf{X}(t) = (X_1^*(t), \dots, X_d^*(t))'$ (every candidate is accepted)
 - 4 Increment t .

Example 4: Generate bivariate Normal

- Suppose $\mathbf{X} = (X_1, X_2)' \sim N(\mu, \Sigma)$ where

$$\mu = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix} \quad \text{and} \quad \Sigma = \begin{pmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{21} & \sigma_2^2 \end{pmatrix}$$

- The conditional for X_1 given $X_2 = x_2$ is $N(\mu_{1|2}, \sigma_{1|2}^2)$, where

$$\mu_{1|2} = \mu_1 + \frac{\sigma_{12}}{\sigma_2^2}(x_2 - \mu_2) \tag{1}$$

$$\sigma_{1|2}^2 = \sigma_1^2 - \frac{\sigma_{12}^2}{\sigma_2^2} = \sigma_1^2(1 - \rho^2) \tag{2}$$

- Similar formulas for X_2 given $X_1 = x_1$

Example 4: Generate bivariate Normal

- The chain is generated by sampling from

$$f(x_1|x_2) \sim N(\mu_1 + \frac{\sigma_{12}}{\sigma_2^2}(x_2 - \mu_2), \sigma_1^2(1 - \rho^2)) \quad (3)$$

$$f(x_2|x_1) \sim N(\mu_2 + \frac{\sigma_{21}}{\sigma_1^2}(x_1 - \mu_1), \sigma_2^2(1 - \rho^2)) \quad (4)$$

- At each iteration, the Gibbs sampler

- 1 set $(x_1, x_2)' = X(t - 1)$
- 2 generate $x_1^*(t) \sim f(x_1|x_2)$
- 3 update $x_1 = x_1^*(t)$
- 4 generate $x_2^*(t) \sim f(x_2|x_1)$
- 5 Set $X(t) = (x_1^*(t), x_2^*(t))'$

Example 4: Generate bivariate Normal

```
# Initialize parameters
rho = 0.75      #correlation
mu1 = 0; mu2 = 2
sigma1 = 1; sigma2 = 1.5
s1 = sqrt(1-rho^2)*sigma1
s2 = sqrt(1-rho^2)*sigma2

N = 5000      #length of chain
X = matrix(0, N, 2) #the chain, a bivariate sample

##### Generate the chain #####
X[1,] = c(5, 5)
for (i in 2:N) {
  x2 = X[i-1, 2]
  m1 = mu1 + rho*(x2 - mu2)*sigma1/sigma2
  X[i, 1] = rnorm(1, m1, s1)
  x1 = X[i, 1]
  m2 = mu2 + rho*(x1 - mu1)*sigma2/sigma1
  X[i, 2] = rnorm(1, m2, s2)
}
```

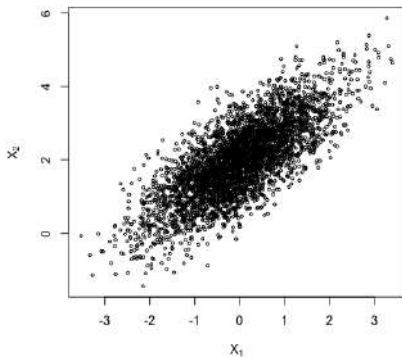
Example 4: Generate bivariate Normal

```
burn = 1000    #burn-in length
b = burn + 1
x = X[b:N,]

> # compare sample statistics to parameters
> colMeans(x)
[1] 0.02069764 2.04219816
> cov(x)
      [,1]      [,2]
[1,] 1.036245 1.169929
[2,] 1.169929 2.343502
> cor(x)
      [,1]      [,2]
[1,] 1.0000000 0.7507507
[2,] 0.7507507 1.0000000
```

Example 4: Generate bivariate Normal

```
plot(x, main="", cex=.5, xlab=bquote(X[1]),  
     ylab=bquote(X[2]), ylim=range(x[,2]))
```

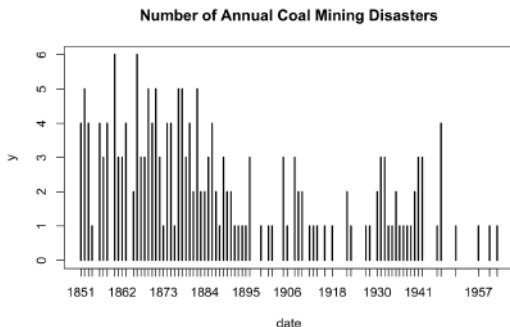


Outline

- 1 Random Walk Metropolis
- 2 The Independence Sampler
- 3 Gibbs Sampler
- 4 Application: change point analysis

Coal mining disasters data

Number of annual coal mining disasters from March 15, 1851 until March 22, 1962:



The annual counts show many more disasters in earlier years and noticeably fewer in later years, making a change-point model plausible.

Observed data and piecewise Poisson model

- Use Poisson distribution to model the frequency of rare events.
- Let Y_i be the number of disasters in year i , where year 1851 corresponds to $i = 1$.
- Assume the change point occurs at year k .

Assume

$$Y_i \sim \text{Poisson}(\mu), \quad i = 1, \dots, k,$$

$$Y_i \sim \text{Poisson}(\lambda), \quad i = k + 1, \dots, n.$$

For the coal data:

- $n = 112$ yearly observations,
- final year is 1962,
- goal: estimate (μ, λ, k) .

Prior specification

Consider independent priors:

$$k \sim \text{Uniform}\{1, 2, \dots, n\}, \quad \mu \sim \text{Gamma}(0.5, b_1), \quad \lambda \sim \text{Gamma}(0.5, b_2).$$

Additional hyperparameters are introduced through

$$b_1 \mid \mu \sim \text{Gamma}(0.5, \mu + 1), \quad b_2 \mid \lambda \sim \text{Gamma}(0.5, \lambda + 1).$$

This hierarchy keeps the model flexible while still allowing convenient Gibbs updates.

Full conditional distributions

Let

$$S_k = \sum_{i=1}^k Y_i, \quad S'_k = \sum_{i=k+1}^n Y_i = S_n - S_k.$$

Then the fully specified conditionals are

$$\mu \mid y, \lambda, b_1, b_2, k \sim \text{Gamma}(0.5 + S_k, k + b_1),$$

$$\lambda \mid y, \mu, b_1, b_2, k \sim \text{Gamma}(0.5 + S'_k, n - k + b_2),$$

$$b_1 \mid y, \mu, \lambda, b_2, k \sim \text{Gamma}(0.5, \mu + 1),$$

$$b_2 \mid y, \mu, \lambda, b_1, k \sim \text{Gamma}(0.5, \lambda + 1).$$

Conditional posterior for the change point

The conditional posterior of k is discrete:

$$f(k \mid Y, \mu, \lambda, b_1, b_2) = \frac{L(Y; k, \mu, \lambda)}{\sum_{j=1}^n L(Y; j, \mu, \lambda)},$$

for $k = 1, 2, \dots, n$, where

$$L(Y; k, \mu, \lambda) = e^{k(\lambda - \mu)} \left(\frac{\mu}{\lambda} \right)^{S_k}.$$

So each Gibbs iteration updates k by sampling from a multinomial distribution on $\{1, \dots, n\}$ with probabilities proportional to $L(Y; k, \mu, \lambda)$.

Gibbs sampling algorithm

At iteration t ,

- 1 Sample $\mu^{(t)}$ from

$$\text{Gamma}(0.5 + S_{k^{(t-1)}}, k^{(t-1)} + b_1^{(t-1)}).$$

- 2 Sample $\lambda^{(t)}$ from

$$\text{Gamma}(0.5 + S'_{k^{(t-1)}}, n - k^{(t-1)} + b_2^{(t-1)}).$$

- 3 Sample $b_1^{(t)}$ and $b_2^{(t)}$ from their gamma conditionals.
- 4 Update $k^{(t)}$ from the discrete distribution proportional to $L(Y; k, \mu^{(t)}, \lambda^{(t)})$.

Every proposed update is drawn directly from its full conditional, so every draw is accepted.

R skeleton: initialization

```
# Gibbs sampler for the coal mining change point
n <- length(y)
m <- 1000
mu <- lambda <- k <- numeric(m)
L <- numeric(n)

k[1] <- sample(1:n, 1)
mu[1] <- 1
lambda[1] <- 1
b1 <- 1
b2 <- 1
```

The chain stores the sampled values of μ , λ , and k across iterations.

R skeleton: one Gibbs iteration

```
for (i in 2:m) {  
  kt = k[i-1]  
  r = .5 + sum(y[1:kt])  
  mu[i] = rgamma(1, shape = r, rate = kt+b1)  
  
  r = if (kt + 1 > n) .5 else .5 + sum(y[(kt+1):n])  
  lambda[i] = rgamma(1, shape = r, rate = n-kt+b2)  
  
  b1 = rgamma(1, shape = .5, rate = mu[i]+1)  
  b2 = rgamma(1, shape = .5, rate = lambda[i]+1)  
}
```

R skeleton: updating the change point

```
for (j in 1:n) {  
  L[j] = exp((lambda[i] - mu[i])*j) *  
          (mu[i]/lambda[i])^sum(y[1:j])  
}  
L = L/sum(L)  
k[i] = sample(1:n, prob = L, size = 1)
```

This is the key discrete update: compute the normalized posterior weights for all candidate change points, then sample one value of k .

Posterior summaries after burn-in

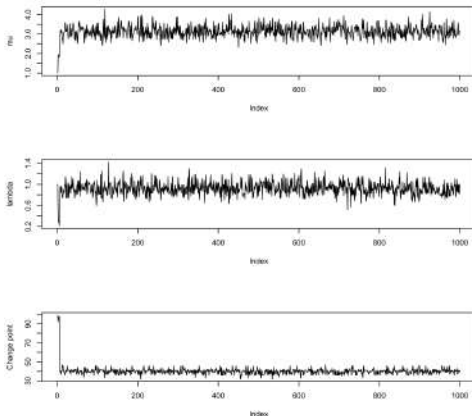
After discarding the first 200 iterations, the approximate posterior means

$$\hat{k} \approx 39.85, \quad \hat{\mu} \approx 3.133, \quad \hat{\lambda} \approx 0.9221.$$

Interpretation:

- The estimated change point is around year $k = 40$.
- Since 1851 is year 1, this corresponds to about 1890.
- The estimated disaster rate drops substantially after the change point.

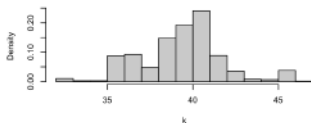
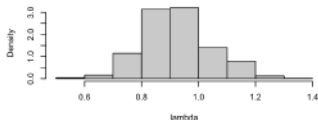
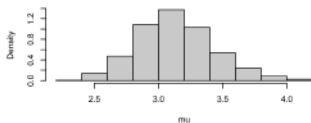
Trace plots of the Gibbs output



The chains for μ , λ , and k stabilize quickly.

The k -chain fluctuates around values near 40, reinforcing the evidence for a late-19th-century change point.

Marginal posterior histograms



The posterior distributions are concentrated around

- $\mu \approx 3.1$,
- $\lambda \approx 0.9$,
- $k \approx 40$.

The drop from about 3.1 to about 0.9 suggests a substantial decline in the mean number of disasters after the inferred change point.

R code

```
library(boot) #for coal data
data(coal)
year <- floor(coal)
y <- table(year)
plot(y, main="Number of Annual Coal Mining Disasters")

y <- floor(coal[[1]])
y <- tabulate(y)
y <- y[1851:length(y)]

# Gibbs sampler for the coal mining change point

# initialization
n <- length(y) #length of the data
m <- 1000 #length of the chain
mu <- lambda <- k <- numeric(m)
L <- numeric(n)
k[1] <- sample(1:n, 1)
mu[1] <- 1; lambda[1] <- 1
b1 <- 1; b2 <- 1
```

```

# run the Gibbs sampler
for (i in 2:m) {
  kt <- k[i-1]
  #generate mu
  r <- .5 + sum(y[1:kt])
  mu[i] <- rgamma(1, shape = r, rate = kt + b1)

  #generate lambda
  if (kt + 1 > n) r <- .5 + sum(y) else
  r <- .5 + sum(y[(kt+1):n])
  lambda[i] <- rgamma(1, shape = r, rate = n - kt + b2)
  #generate b1 and b2
  b1 <- rgamma(1, shape = .5, rate = mu[i]+1)
  b2 <- rgamma(1, shape = .5, rate = lambda[i]+1)

  for (j in 1:n) {
    L[j] <- exp((lambda[i] - mu[i]) * j) *
      (mu[i] / lambda[i])^sum(y[1:j])
  }
  L <- L / sum(L)
  #generate k from discrete distribution L on 1:n
  k[i] <- sample(1:n, prob=L, size=1)
}

```

```
b <- 201
j <- k[b:m]
print(mean(k[b:m]))
print(mean(mu[b:m]))
print(mean(lambda[b:m]))

par(mfrow=c(3,1))
plot(mu, type="l", ylab="mu")
plot(lambda, type="l", ylab="lambda")
plot(k, type="l", ylab="Change point")

par(mfrow=c(3,1))
hist(mu[b:m], main="", xlab="mu", prob=TRUE)
hist(lambda[b:m], main="", xlab="lambda", prob=TRUE)
hist(k[b:m], main="", xlab="k", prob=TRUE)
```

Nonparametric Density Estimation

Rizzo Chapter 10

Fangfang Wang

Motivation

- Goal: estimate an unknown density $f(x)$ from data $X_1, \dots, X_n$
- Parametric approach:
 - Assume a specific distribution (e.g., Normal)
- Nonparametric approach:
 - No parametric assumption on f
 - Let the data determine the shape
- Key idea: smooth the data to recover the underlying distribution

Why nonparametric density estimation?

- Goal: estimate an unknown density directly from a sample without specifying a parametric family.
- Density estimation is useful for visualization, exploratory analysis, modality assessment, skewness, and tail behavior.
- The core tradeoff is always the same: **too little smoothing** gives a rough estimate, while **too much smoothing** hides important features.
- Histogram $\rightarrow$ averaged shifted histogram (ASH) $\rightarrow$ kernel density estimator.

Outline

1 Histogram density estimation

2 Averaged Shifted Histogram

3 Kernel Density Estimation

- Boundary effects

Outline

1 Histogram density estimation

2 Averaged Shifted Histogram

3 Kernel Density Estimation

- Boundary effects

Histogram density estimator

Suppose the real line is partitioned into equal-width bins

$$[t_k, t_{k+1}), \quad h = t_{k+1} - t_k,$$

and let v_k be the number of observations in the k th bin. Then the histogram density estimator is

$$\hat{f}(x) = \frac{v_k}{n} \cdot \frac{1}{h}, \quad t_k \leq x < t_{k+1}. \quad (1)$$

- v_k/n is the relative frequency in the bin
- Dividing by h turns relative frequency into density height
- Piecewise constant, and discontinuous

Interpretation of Histogram

- Counts number of observations in each bin
- Normalizes by:
 - sample size n
 - bin width h
- Integrates to 1
- Acts as a simple density estimator

Key issue: choice of bin width h

- Small h : noisy estimate
- Large h : oversmoothed estimate

Mean Integrated Squared Error

Mean Integrated Squared Error (MISE) measures the overall accuracy of a density estimator:

$$\text{MISE} = \mathbb{E} \left[\int (\hat{f}(x) - f(x))^2 dx \right]$$

Interpretation:

- It averages the squared error over all x and over randomness of the sample
- It is the global performance criterion for density estimation

Bias–Variance Decomposition:

$$\text{MISE} = \int \text{Bias}^2(x) dx + \int \text{Var}(\hat{f}(x)) dx$$

Bias–Variance Analysis for Histogram

For the histogram estimator:

$$\text{Bias} = O(h), \quad \text{Var}(\hat{f}(x)) = O\left(\frac{1}{nh}\right).$$

Interpretation:

- Small h : many bins, low bias, high variability.
- Large h : few bins, smoother picture, but higher bias.

Hence

$$\text{MISE} \approx C_1 h^2 + C_2 \frac{1}{nh}$$

The mean integrated squared error suggests an optimal order

$$h \asymp n^{-1/3}.$$

Recall the histogram estimator:

$$\hat{f}(x) = \frac{N_B}{nh}, \quad N_B = \sum_{i=1}^n I(X_i \in B)$$

where B is the bin of width h containing x .

Bias:

- $\mathbb{E}[\hat{f}(x)] = \frac{1}{h}P(X \in B) = \frac{1}{h} \int_B f(u) du$
- Taylor expansion: $f(u) = f(x) + f'(x)(u - x) + O(h^2)$
- Since bin is not centered at x , first-order term remains

$$\Rightarrow \text{Bias}(x) = \mathbb{E}[\hat{f}(x)] - f(x) = O(h)$$

Variance:

- $N_B \sim \text{Binomial}(n, p_B)$, where $p_B \approx f(x)h$
- $\text{Var}(N_B) = np_B(1 - p_B) \approx nf(x)h$

$$\text{Var}(\hat{f}(x)) = \frac{1}{n^2 h^2} \text{Var}(N_B) \approx \frac{f(x)}{nh}$$

Sturges' Rule

Choose number of bins:

$$k = \lceil 1 + \log_2 n \rceil$$

Then bin width:

$$h = \frac{\text{range}}{k}$$

Comments:

- Works well for small n
- Assumes near-normal data

Scott's (Normal Reference) Rule

Scott (1979)¹:

$$\text{MISE} = h^2 \frac{1}{12} \int f'(x)^2 dx + \frac{1}{nh} + O\left(\frac{1}{n} + h^3\right) \quad (2)$$

The optimal choice of bin width:

$$h^* = \left(\frac{6n}{\int f'(x)^2 dx} \right)^{1/3} \quad (3)$$

Optimal bin width (normal reference, i.e. f is Normal with variance σ^2):

$$h = 3.5 \hat{\sigma} n^{-1/3}$$

Features:

- Derived from minimizing MISE
- Sensitive to outliers (uses $\hat{\sigma}$)

¹Scott (1979): On optimal and data-based algorithm, *Biometrika*.

Freedman–Diaconis Rule

Robust alternative:

$$h = 2 * \text{IQR} * n^{-1/3}$$

Advantages:

- Uses IQR instead of standard deviation
- Less sensitive to outliers

Comparison of Bin Width Rules

Rule	Formula
Sturges	$h = \frac{Range}{\lceil 1 + \log_2 n \rceil}$
Scott	$h = 3.5 \hat{\sigma} n^{-1/3}$
Freedman–Diaconis	$h = 2 IQR n^{-1/3}$

Summary:

- Sturges: simple, small samples; but oversmooths
- Scott: optimal under normality
- FD: robust to outliers

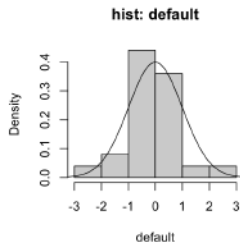
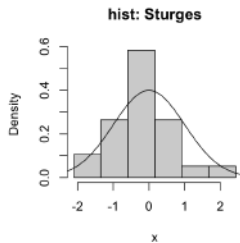
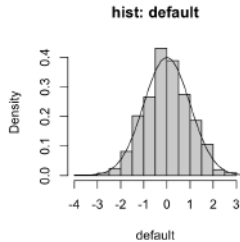
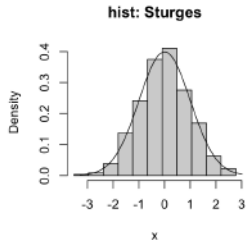
R code: explicit Sturges breaks

```
n = 25
x = rnorm(n)

nclass = ceiling(1 + log2(n))
cwidth = diff(range(x)) / nclass
breaks = min(x) + cwidth * 0:nclass
h.sturges = hist(x, breaks = breaks, freq = FALSE,
                 main = "hist: Sturges")

z = qnorm(ppoints(1000))
lines(z, dnorm(z))

h.default = hist(x, freq = FALSE, xlab = "default",
                 main = "hist: default")
lines(z, dnorm(z))
```

$n = 25 :$  $n = 1000 :$ 

Limitations of Histogram

- Discontinuous estimator
- Depends on:
 - bin width
 - bin origin
- Sensitive to arbitrary choices

Motivation for improvement:

- smoother estimators
- reduced dependence on bin alignment

Outline

1 Histogram density estimation

2 Averaged Shifted Histogram

3 Kernel Density Estimation

- Boundary effects

Averaged shifted histogram (ASH)

ASH averages several histograms with the same width but shifted origins:

$$\hat{f}_{ASH}(x) = \frac{1}{m} \sum_{j=1}^m \hat{f}_j(x).$$

where $\hat{f}_j(x)$ is the histogram estimator constructed using bins shifted by $\frac{j}{m}h$. More precisely,

$$\hat{f}_j(x) = \frac{1}{nh} \sum_{i=1}^n \mathbf{1}(X_i \in B_j(x)), \quad (4)$$

where $B_j(x)$ is the bin (under the j th shift) that contains x .

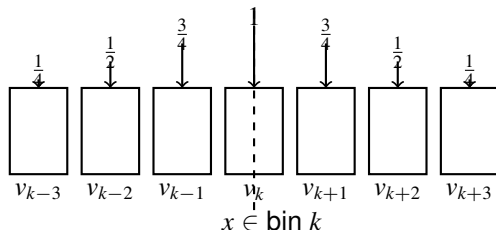
ASH as Smoothing Nearby Bin Counts

- Divide the real line into fine bins of width: $\delta = \frac{h}{m}$
- Let v_k = number of observations in bin k
- Suppose x lies in bin k

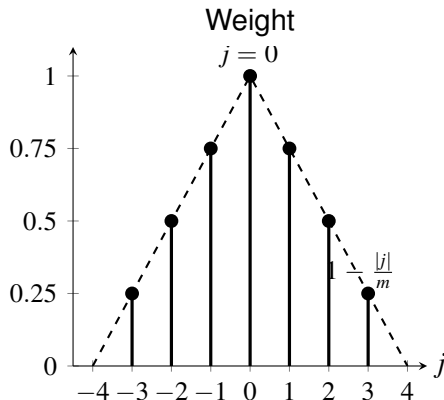
ASH representation:

$$\hat{f}_{ASH}(x) = \frac{1}{nh} \sum_{j=1-m}^{m-1} \left(1 - \frac{|j|}{m}\right) v_{k+j} \quad (5)$$

Triangular weighting of nearby counts



Visualizing the ASH Weights



$$\hat{f}_{ASH}(x) = \frac{1}{nh} \sum_{j=1-m}^{m-1} \left(1 - \frac{|j|}{m}\right) v_{k+j}$$

- The center bin v_k gets the largest weight.
- Nearby bins get smaller weights.
- The weights decrease linearly as $|j|$ increases.
- Bins farther away than distance $m - 1$ are ignored.

ASH as Weighted Average of Bin Counts

ASH representation:

$$\hat{f}_{ASH}(x) = \frac{1}{nh} \sum_{j=1-m}^{m-1} \left(1 - \frac{|j|}{m}\right) v_{k+j} \quad (6)$$

- Weighted average of nearby bin counts
- Highest weight at center bin ($j = 0$)
- Linearly decreasing weights as distance increases

Key insight:

- Produces smoothing similar to kernel methods
- Equivalent to using a triangular kernel

Properties of ASH

- Reduces discontinuity of histogram
- Approximates kernel density estimator
- Still depends on bin width h

Insight:

- ASH bridges histograms and kernel methods

From Histogram to Kernel Methods

- Histogram:
 - counts observations in bins
- ASH:
 - averages shifted histograms
- Kernel density estimator:
 - places a smooth function at each data point

Transition:

- Replace bins with smooth kernels

Outline

1 Histogram density estimation

2 Averaged Shifted Histogram

3 Kernel Density Estimation

- Boundary effects

From histograms to kernels

Starting from a window of width $2h$, the naive estimator is

$$\hat{f}(x) = \frac{k}{2nh},$$

where k is the number of observations in $(x - h, x + h)$.

This can be written as

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h} w\left(\frac{x - X_i}{h}\right),$$

where

$$w(t) = \frac{1}{2} I(|t| < 1),$$

- $w(t)$ is the **rectangular kernel**.
- Each observation contributes a rectangle of area $1/n$ centered at X_i .

General kernel density estimator

Kernel density estimation replaces the rectangular weight function by a general kernel K satisfying

$$\int_{-\infty}^{\infty} K(t) dt = 1.$$

The estimator is

$$\hat{f}_K(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h} K\left(\frac{x - X_i}{h}\right). \quad (10.12)$$

Interpretation

- K controls the **shape** of each local contribution.
- h is the **bandwidth** (or smoothing parameter, window width).
- Small h : rough estimate with more local detail.
- Large h : smoother estimate with more bias and less variance.

Properties inherited from the kernel

From (10.12), smoothness of K transfers to $\hat{f}_K$:

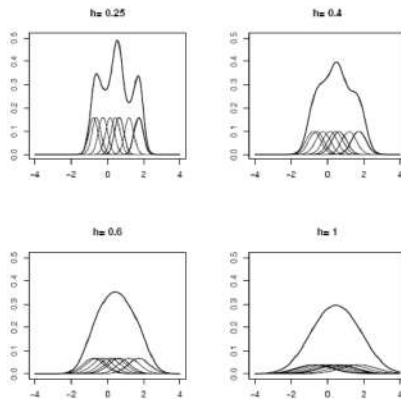
- If K is continuous at x , then $\hat{f}_K(x)$ is continuous.
- If $K^{(r)}$ exists, then $\hat{f}_K$ has an r th derivative.
- For the Gaussian kernel, $\hat{f}_K$ is infinitely differentiable.

Thus kernel estimators are typically much smoother than histogram estimators.

Key idea

The histogram corresponds to a rectangular kernel. Kernel density estimation generalizes the histogram by replacing discontinuous boxes with smoother local bumps.

Effect of bandwidth: Gaussian kernel example



- Same $n = 10$ sample points, different bandwidths $h = 0.25, 0.4, 0.6, 1$.
- As h increases, the estimate becomes smoother.

Bandwidth as the main tuning parameter

- In practice, the choice of **bandwidth** matters much more than the choice of kernel.
- Small h reveals local features but can introduce artificial bumps.
- Large h reduces variability but may oversmooth multimodality or skewness.

R terminology

In `density()`, the smoothing parameter is `bw`. The default rule corresponds to Silverman's rule-of-thumb in many common settings.

Examples of kernels

Common kernel choices include:

$$\text{Gaussian: } K(t) = \frac{1}{\sqrt{2\pi}} e^{-t^2/2},$$

$$\text{Epanechnikov: } K(t) = \frac{3}{4}(1 - t^2), \quad |t| < 1,$$

$$\text{Rectangular: } K(t) = \frac{1}{2}, \quad |t| < 1,$$

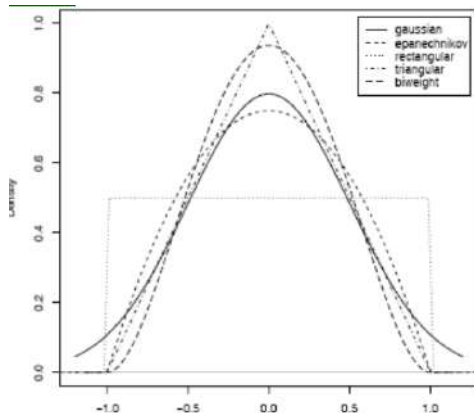
$$\text{Triangular: } K(t) = 1 - |t|, \quad |t| < 1.$$

$$\text{Biweight (or quartic): } K(t) = \frac{15}{16}(1 - t^2)^2, \quad |t| < 1.$$

Practical message

The choice of kernel is usually less important than the choice of bandwidth h .

Common kernels



- Most are symmetric densities centered at 0.
- Different kernels look different, but their overall efficiencies are quite close.

Kernel comparison table

Kernel	$K(t)$	Support	σ_K^2	Efficiency
Gaussian	$\frac{1}{\sqrt{2\pi}} \exp(-\frac{1}{2}t^2)$	$\mathbb{R}$	1	1.0513
Epanechnikov	$\frac{3}{4}(1-t^2)$	$ t < 1$	1/5	1
Rectangular	$\frac{1}{2}$	$ t < 1$	1/3	1.0758
Triangular	$1- t $	$ t < 1$	1/6	1.0143
Biweight	$\frac{15}{16}(1-t^2)^2$	$ t < 1$	1/7	1.0061
Cosine	$\frac{\pi}{4} \cos \frac{\pi}{2} t$	$\mathbb{R}$	$1-8/\pi^2$	1.0005

- The Epanechnikov kernel has efficiency 1 under the IMSE criterion.
- The efficiencies of other kernels are very close to 1.
- Hence bandwidth selection is usually more important than kernel selection.

Bandwidth for Gaussian kernel

For a Gaussian kernel, the IMSE-optimal normal-reference bandwidth is

$$h = \left(\frac{4}{3}\right)^{1/5} \sigma n^{-1/5} \approx 1.06 \sigma n^{-1/5}.$$

A common robust rule replaces σ by

$$\hat{\sigma} = \min \left(S, \frac{IQR}{1.34} \right),$$

leading to Silverman's rule of thumb:

$$h = 0.9 \hat{\sigma} n^{-1/5}.$$

- This is the rule behind `bw.nrd0()` in R.
- It is a good default for many unimodal and moderately skewed distributions.
- The $n^{-1/5}$ rate matches the nonparametric bias–variance tradeoff for smooth estimators.

Why $n^{-1/5}$?

For kernel density estimation (under smoothness assumptions):

$$\text{Bias}(x) = O(h^2), \quad \text{Var}(\hat{f}(x)) = O\left(\frac{1}{nh}\right).$$

MISE approximation:

$$\text{MISE}(h) \approx C_1 h^4 + \frac{C_2}{nh}.$$

Optimal bandwidth: balance the two terms

$$h^4 \sim \frac{1}{nh} \implies h^5 \sim \frac{1}{n} \implies h \asymp n^{-1/5}.$$

- The $n^{-1/5}$ rate is the optimal smoothing level for *smooth* estimators.
- It arises from the generic tradeoff:

$$\text{Bias}^2 \sim h^4 \quad \text{vs} \quad \text{Variance} \sim \frac{1}{nh}.$$

Why $\text{Bias}(x) = O(h^2)$ for KDE

Recall that Kernel density estimator is

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h} K\left(\frac{x - X_i}{h}\right).$$

Take expectation:

$$\mathbb{E}[\hat{f}(x)] = \int \frac{1}{h} K\left(\frac{x - u}{h}\right) f(u) du. \quad (7)$$

Let $t = \frac{x-u}{h}$:

$$\mathbb{E}[\hat{f}(x)] = \int K(t) f(x - ht) dt. \quad (8)$$

Taylor expansion of $f(x - ht)$:

$$f(x - ht) = f(x) - htf'(x) + \frac{h^2 t^2}{2} f''(x) + O(h^3). \quad (9)$$

Then:

$$\mathbb{E}[\hat{f}(x)] = f(x) \int K(t)dt - hf'(x) \int tK(t)dt + \frac{h^2 f''(x)}{2} \int t^2 K(t)dt + O(h^3). \quad (10)$$

Use kernel properties:

$$\int K(t)dt = 1, \quad \int tK(t)dt = 0.$$

$$\Rightarrow \mathbb{E}[\hat{f}(x)] = f(x) + \frac{h^2}{2} f''(x) \mu_2(K) + O(h^3), \quad (11)$$

where $\mu_2(K) = \int t^2 K(t)dt$.

Conclusion:

$$\text{Bias}(x) = \mathbb{E}[\hat{f}(x)] - f(x) = O(h^2).$$

Why $\text{Var}(\hat{f}(x)) = O(1/(nh))$ for KDE

KDE:

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h} K\left(\frac{x - X_i}{h}\right)$$

Let

$$Z_i = \frac{1}{h} K\left(\frac{x - X_i}{h}\right) \Rightarrow \hat{f}(x) = \frac{1}{n} \sum Z_i$$

Thus

$$\text{Var}(\hat{f}(x)) = \frac{1}{n} \text{Var}(Z_1)$$

Key idea:

- Z_i is nonzero only if $|X_i - x| \lesssim h$
- Probability $\approx h$, magnitude $\approx 1/h$

$$\Rightarrow \text{Var}(Z_1) \approx \left(\frac{1}{h}\right)^2 \cdot h = \frac{1}{h}$$

Formal calculation:

$$\mathbb{E}[Z_1^2] = \frac{1}{h} \int K^2(t) f(x - ht) dt \approx \frac{f(x)}{h} R(K)$$

$$\Rightarrow \text{Var}(Z_1) = O\left(\frac{1}{h}\right)$$

Thus,

$$\text{Var}(\hat{f}(x)) = O\left(\frac{1}{nh}\right)$$

Interpretation:

- About nh observations contribute locally
- Averaging $\Rightarrow$ variance $\sim 1/(nh)$

Equivalent kernel rescaling in R

If two kernels have standard deviations σ_{K_1} and σ_{K_2} , then approximately equivalent smoothing can be obtained by

$$h_2 \approx \frac{\sigma_{K_1}}{\sigma_{K_2}} h_1.$$

- `density()` in R rescales kernels so that `bw` is the standard deviation of the kernel.
- The option `give.Rkern = TRUE` returns the **canonical bandwidth factor**.
- Available kernels include `gaussian`, `epanechnikov`, `rectangular`, `triangular`, `biweight`, `cosine`, and `optcosine`.

Example 10.7: Old Faithful waiting time

For `geyser$waiting` in MASS:

$$h_1 = 1.06 \operatorname{sd}(\text{waiting}) n^{-1/5} = 4.7085,$$

$$h_2 = 0.9 \min\left(\frac{\operatorname{IQR}(\text{waiting})}{1.34}, \operatorname{sd}(\text{waiting})\right) n^{-1/5} = 3.9978.$$

The default `density(waiting)` output reports

$$\text{Bandwidth 'bw'} = 3.998.$$

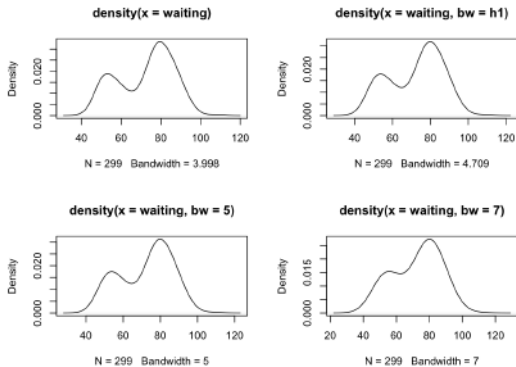
So the default is essentially using Silverman's rule-of-thumb.

Example 10.7: Old Faithful waiting time

```
library(MASS)
waiting <- geyser$waiting
n <- length(waiting)
h1 <- 1.06*sd(waiting)*n^(-1/5) # Optimal
h2 <- .9*min(c(IQR(waiting)/1.34, sd(waiting)))*n^(-1/5)
print(c(h1, h2))

par(mfrow=c(2,2))
plot(density(waiting))
plot(density(waiting, bw = h1)) #Gaussian
plot(density(waiting, bw = 5)) #Gaussian
plot(density(waiting, bw = 7)) #Gaussian
```

Old Faithful: effect of changing the bandwidth



- Smaller bandwidths reveal stronger multimodality.
- Larger bandwidths blur the valley between the two main modes.
- For this data, the default bandwidth provides a reasonable compromise.

Example 10.8: precipitation data

The dataset `precip` records average precipitation amounts for 70 U.S. cities and Puerto Rico.

$$\blacksquare h_0 = \text{bw.nrd0}(\text{precip}) = 3.8479$$

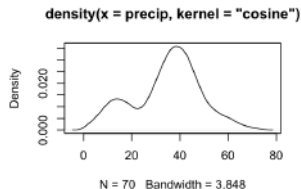
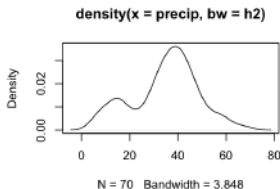
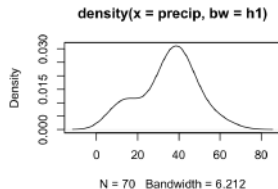
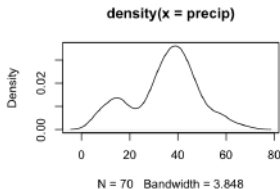
$$\blacksquare h_1 = 1.06 \text{sd}(\text{precip}) n^{-1/5} = 6.2118$$

$$\blacksquare h_2 = 0.9 \min(\text{IQR}/1.34, \text{sd}) n^{-1/5} = 3.8479$$

```
n <- length(precip)
h1 <- 1.06 * sd(precip) * n^(-1/5)
h2 <- .9 * min(c(IQR(precip)/1.34, sd(precip))) * n^(-1/5)
h0 <- bw.nrd0(precip)
```

```
par(mfrow = c(2, 2))
plot(density(precip)) #default Gaussian (h0)
plot(density(precip, bw = h1)) #Gaussian, bandwidth h1
plot(density(precip, bw = h2)) #Gaussian, bandwidth h2
plot(density(precip, kernel = "cosine"))
```

Precipitation example: density plots



- Different bandwidths produce different smoothness levels.
- The cosine kernel with comparable bandwidth is visually similar to the Gaussian result.
- However, the support of precipitation is nonnegative, so boundary effects matter.

Estimating $\hat{f}(x)$ at arbitrary points

The object returned by `density()` contains vectors `x` and `y`. To estimate the density at new points, use linear interpolation:

$$d \leftarrow \text{density}(\text{precip}), \quad x_{\text{new}} = 0, 10, 20, \dots, 70.$$

Then

```
approx(d$x, d$y, xout = xnew)
```

returns interpolated density estimates.

For repeated evaluation, define a function with

```
fhat <- approxfun(d$x, d$y).
```

This is a convenient computational device when a closed-form formula is unavailable.

Example 10.8: precipitation data

```
> d <- density(precip)
> xnew <- seq(0, 70, 10)
> approx(d$x, d$y, xout = xnew)
$x
[1] 0 10 20 30 40 50 60 70

$y
[1] 0.0009487629 0.0109615916 0.0100237284 0.0210798860 0.0357491244
[6] 0.0144029982 0.0054739734 0.0011710969

> fhat <- approxfun(d$x, d$y)
> fhat(xnew)
[1] 0.0009487629 0.0109615916 0.0100237284 0.0210798860 0.0357491244
[6] 0.0144029982 0.0054739734 0.0011710969
```

Boundary kernels and boundary bias

Kernel Density Estimator:

$$\hat{f}_K(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h} K\left(\frac{x - X_i}{h}\right). \quad (10.12)$$

Near the boundary of the support, kernel estimators can incur larger errors because symmetric kernels place positive weight outside the support.

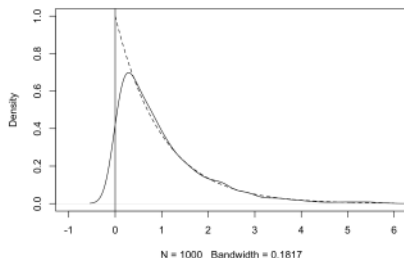
For nonnegative data such as precipitation:

- ordinary kernel estimates may suggest impossible negative values,
- the bias near 0 can be substantial,
- special corrections or boundary kernels are needed.

Diagnostic sign

If the estimated density assigns visible mass to negative values for a nonnegative variable, boundary bias is likely present.

Example 10.10: exponential density and boundary problem



- Data are simulated from an $\text{Exponential}(1)$.
- The dashed line is the true density; the solid line is the Gaussian kernel estimate.
- The smooth kernel estimate does not respect the discontinuity at $x = 0$.

Reflection boundary technique

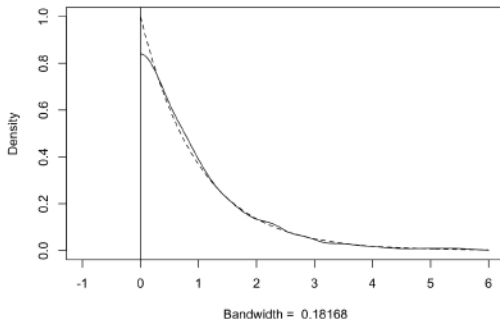
A simple fix when the support is $[0, \infty)$ is the **reflection method**.

- 1 Reflect the sample: append $-X_1, \dots, -X_n$ to the data.
- 2 Compute a density estimate g using the $2n$ reflected observations.
- 3 Use the original sample size n to determine the smoothing parameter.
- 4 On $x \geq 0$, define

$$\hat{f}(x) = 2\hat{g}(x).$$

- This restores mass near the boundary.
- It is especially useful when the support endpoint is known.

Example 10.11: reflected estimate



- The reflected estimate tracks the true exponential density more closely near $x = 0$.
- Reflection is easy to implement and often works well for support boundaries at 0.

Example 10.11: reflected estimate

```
x <- rexp(1000, 1)
plot(density(x), xlim = c(-1, 6), ylim = c(0, 1), main="")
abline(v = 0)
# add the true density to compare
y <- seq(.001, 6, .01)
lines(y, dexp(y, 1), lty = 2)

## Reflection boundary technique
xx <- c(x, -x)
g <- density(xx, bw = bw.nrd0(x))
a <- seq(0, 6, .01)
ghat <- approx(g$x, g$y, xout = a)
fhat <- 2*ghat$y # density estimate along a
bw <- paste("Bandwidth = ", round(g$bw, 5))
plot(a, fhat, type="l", xlim=c(-1, 6), ylim=c(0, 1),
      main = "", xlab = bw, ylab = "Density")
abline(v = 0)
# add the true density to compare
y <- seq(.001, 6, .01)
lines(y, dexp(y, 1), lty = 2)
```

Takeaways

- Kernel density estimation smooths the empirical distribution by averaging local kernel contributions.
- The estimator is

$$\hat{f}_K(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h} K\left(\frac{x - X_i}{h}\right).$$

- The **bandwidth** is the key tuning parameter; the precise kernel matters less.
- Silverman's rule-of-thumb provides a practical default in many situations.
- Boundary bias can be substantial for nonnegative data; reflection is a simple correction.

Summary

- Histogram estimators are simple and interpretable, but depend strongly on bin width and bin origin.
- ASH smoothes the histogram while retaining a binning interpretation.
- Kernel density estimation replaces hard bins with smooth kernels.
- In practice, bandwidth choice matters much more than kernel choice.
- Boundary bias is real and often visible; reflection is a simple and useful correction.

Statistical Text Mining

Fangfang Wang

Why Text Mining?

- Text is unstructured, high-dimensional data
- Applications:
 - Sentiment analysis (finance, marketing)
 - Topic discovery
 - Document classification
 - Information retrieval
- Goal: classify documents into categories
- Two fundamental approaches:
 - Generative model: Naive Bayes
 - Discriminative model: Logistic Regression

Bag-of-Words Representation

- Represent document as word counts
- Vocabulary: $V = \{w_1, \dots, w_p\}$ where p = vocabulary size.
- Document vector

$$\mathbf{x} = (x_1, \dots, x_p)$$

x_j = count of word w_j

- Ignores word order

Outline

1 Case Study: Email Spam Detection

2 Naive Bayes Model

3 Logistic Regression Model

Outline

1 Case Study: Email Spam Detection

2 Naive Bayes Model

3 Logistic Regression Model

Problem Setup

Goal: Classify emails as Spam or Ham

- Input: Email text d_i
- Output: $y_i \in \{0, 1\}$
- Task: Learn $P(y \mid x)$

Examples:

- “Win a free iPhone now” $\rightarrow$ Spam
- “Meeting at 2pm tomorrow” $\rightarrow$ Ham
- “Limited time offer, click here” $\rightarrow$ Spam
- “Project report attached” $\rightarrow$ Ham

R Implementation

```
> library(tm)
> df <- read.csv("emails.csv")
> head(df)
```

	text	label
1	Win a free iPhone now	1
2	Limited time offer click here	1
3	Congratulations you won a prize	1
4	Claim your free reward today	1
5	Exclusive deal just for you	1
6	Get cash fast now	1

```
>
```

Create a Text Corpus:

```
corpus <- VCorpus(VectorSource(df$text))
```

- Uses the tm package
- Converts raw text into a corpus object
- Think of this as a list of documents with metadata

Text Preprocessing

```
corpus <- tm_map(corpus, content_transformer(tolower))  
corpus <- tm_map(corpus, removePunctuation)  
corpus <- tm_map(corpus, removeWords, stopwords("english"))
```

- 1 Lowercasing, e.g., "Free" → "free"; Avoids treating same word as different tokens
- 2 Remove punctuation, e.g., "win!!!" → "win"
- 3 Remove stopwords.
 - Removes common words like: "the", "and", "is";
 - These words usually carry little predictive power

Build Document-Term Matrix (DTM)

```
> dtm <- DocumentTermMatrix(corpus)
> dim(dtm)
[1] 50 122
```

- Converts text into a matrix representation (feature matrix)

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots \\ x_{21} & x_{22} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}_{n \times p}$$

- x_{ij} = number of times word j appears in email i
- Rows = number of documents; $n = 50$ emails (documents)
- Columns = number of unique words (vocabulary size); $p = 122$ unique words (terms)
- Each email x_i is a point in 122-dimensional space; a bag-of-words vector

Example:

Email	free	win	meeting
Email 1	1	1	0
Email 2	0	0	1

```

> Terms (dtm)
[1] "2pm"           "act"           "agenda"        "appointment"   "approaching"
[7] "attached"      "availability"  "available"      "big"           "bonus"
[13] "buy"          "call"         "can"           "cash"          "cheap"
[19] "click"        "confirm"      "confirmed"     "congratulations" "deadline"
[25] "discounted"   "discussion"   "document"      "double"        "draft"
[31] "easily"       "email"       "exclusive"     "extra"         "fast"
[37] "file"         "final"       "follow"        "forward"       "free"
[43] "get"          "guaranteed"  "help"          "home"          "hot"
[49] "invoice"      "iphone"      "join"          "just"          "know"
[55] "launched"    "lecture"     "let"           "limited"        "looking"
[61] "lunch"       "meds"        "meeting"      "monday"        "money"
[67] "next"        "noon"        "notes"        "now"           "offer"
[73] "outing"      "paper"       "planned"      "please"        "presentation"
[79] "price"       "prize"       "prizes"       "products"      "project"
[85] "quarter"     "ready"       "receive"      "reminder"      "report"
[91] "research"    "response"    "review"       "reward"        "schedule"
[97] "selected"    "seminar"     "slides"       "soon"          "special"
[103] "submit"     "sync"        "team"         "thanks"        "thoughts"
[109] "today"      "todays"      "tomorrow"     "upcoming"      "updated"
[115] "urgent"     "vacation"    "week"         "weekly"        "win"
[121] "working"    "works"

```

Inspect part of DTM

```
> Terms(dtm)[1:10]
[1] "2pm"          "act"          "agenda"       "appointment"  "approaching"  "assignment"
[8] "availability" "available"    "big"
>
> inspect(dtm[1:5, 1:10])
<<DocumentTermMatrix (documents: 5, terms: 10)>>
Non-/sparse entries: 0/50
Sparsity           : 100%
Maximal term length: 12
Weighting           : term frequency (tf)
Sample             :
      Terms
Docs 2pm act agenda appointment approaching assignment attached availability available big
1    0  0    0          0          0          0          0          0          0  0
2    0  0    0          0          0          0          0          0          0  0
3    0  0    0          0          0          0          0          0          0  0
4    0  0    0          0          0          0          0          0          0  0
5    0  0    0          0          0          0          0          0          0  0
>
> # Count nonzeros:
> sum(as.matrix(dtm) != 0)
[1] 172
```

Multinomial model for one document

Naive Bayes assumes:

- Given class $y_i = k$, the words in document i follow a multinomial distribution:

$$P(\mathbf{x}_i \mid y_i = k) = \frac{(\sum_j x_{ij})!}{\prod_j x_{ij}!} \prod_{j=1}^p \theta_{jk}^{x_{ij}}$$

where:

- $\theta_{jk} = P(\text{word } j \mid \text{class } k)$
- $\sum_j \theta_{jk} = 1$

Suppose the email is “free offer now”. Then:

$$x_{i,\text{free}} = 1, \quad x_{i,\text{offer}} = 1, \quad x_{i,\text{now}} = 1, \quad \text{all other } x_{ij} = 0.$$

The likelihood becomes:

$$P(\mathbf{x}_i \mid y_i = k) \propto \theta_{\text{free},k} \cdot \theta_{\text{offer},k} \cdot \theta_{\text{now},k}$$

Only nonzero entries in the DTM matter!

Full likelihood for the dataset

Over all documents:

$$\mathcal{L} = \prod_{i=1}^n P(y_i) P(\mathbf{x}_i \mid y_i)$$

Log-likelihood:

$$\ell = \sum_{i=1}^n \log P(y_i) + \sum_{i=1}^n \sum_{j=1}^p x_{ij} \log \theta_{j,y_i}$$

- The DTM entries x_{ij} appear as weights in the likelihood.
- If a word appears many times $\rightarrow$ larger contribution
- If a word never appears $\rightarrow$ no contribution
- Sparsity matters!

Because most $x_{ij} = 0$: $\sum_{j=1}^p x_{ij} \log \theta_{j,k}$ only sums over words that appear in the document, making computation efficient!

MLE

Define the total count of word j in class k :

$$N_{jk} = \sum_{i:y_i=k} x_{ij}$$

```
N_jk <- colSums(dtm[df$label == k, ])
```

Then the log-likelihood simplifies to:

$$\ell = \sum_k \sum_j N_{jk} \log \theta_{jk} + \text{const}$$

Maximizing gives:

$$\hat{\theta}_{jk} = \frac{N_{jk}}{\sum_j N_{jk}}$$

With Laplace smoothing:

$$\hat{\theta}_{jk}^L = \frac{N_{jk} + \alpha}{\sum_{j'} N_{j'k} + \alpha p}$$

R Implementation

```
dtm_matrix <- as.matrix(dtm)

# class labels
y <- df$label

# word counts by class
N_spam <- colSums(dtm_matrix[y == 1, ])
N_ham  <- colSums(dtm_matrix[y == 0, ])

# MLE
theta_spam_mle <- N_spam / sum(N_spam)
theta_ham_mle  <- N_ham  / sum(N_ham)

# Laplace estimator
alpha <- 1
p <- ncol(dtm_matrix)

theta_spam_smooth <- (N_spam+alpha) / (sum(N_spam)+alpha*p)
theta_ham_smooth  <- (N_ham+alpha) / (sum(N_ham)+alpha*p)
```

R Implementation

```
# Put results in a table
theta_table <- data.frame(
  word = colnames(dtm_matrix),
  N_spam = N_spam,
  theta_spam_mle = theta_spam_mle,
  theta_spam_smooth = theta_spam_smooth,
  N_ham = N_ham,
  theta_ham_mle = theta_ham_mle,
  theta_ham_smooth = theta_ham_smooth
)

> head(theta_table)
```

	word	N_spam	theta_spam_mle	theta_spam_smooth	N_ham	theta_ham_mle
2pm	2pm	0	0.00000000	0.005	1	0.0106383
act	act	1	0.01282051	0.010	0	0.0000000
agenda	agenda	0	0.00000000	0.005	1	0.0106383
appointment	appointment	0	0.00000000	0.005	1	0.0106383
approaching	approaching	0	0.00000000	0.005	1	0.0106383
assignment	assignment	0	0.00000000	0.005	1	0.0106383

```

  theta_ham_smooth
2pm      0.009259259
act      0.004629630
agenda   0.009259259
appointment 0.009259259
approaching 0.009259259
assignment 0.009259259
>
> # To inspect the word "free":
> theta_table[theta_table$word == "free", ]
```

	word	N_spam	theta_spam_mle	theta_spam_smooth	N_ham	theta_ham_mle	theta_ham_smooth
free	free	3	0.03846154	0.02	0	0	0.00462963

Prediction

New email:

`"Free offer just for you"`

- High probability words under spam:
 - free, offer
- Compute scores for each class
- Assign label with highest score

Prediction Rule

- 1 Convert the new email into a (sparse) vector $\mathbf{x} = (x_1, \dots, x_p)$
- 2 Use the multinomial NB scoring rule:

$$\log P(y = k) + \sum_{j=1}^p x_j \log \theta_{jk}$$

For each class $k \in \{0, 1\}$:

$$\text{score}(k) = \log \frac{n_k}{n} + \sum_{j=1}^p x_j \log \hat{\theta}_{jk}$$

where n_k = number of emails in class k , n = total number of emails. Because most $x_j = 0$, this reduces to

$$\text{score}(k) = \log \frac{n_k}{n} + \sum_{\text{words in email}} \log \hat{\theta}_{jk}$$

- 3 Decision rule: Choose the class with the larger score

$$\hat{y} = \arg \max_k \text{score}(k)$$

Prediction

Compute scores (conceptual example)

Word	$\hat{\theta}_{j,1}$ (spam)	$\hat{\theta}_{j,0}$ (ham)
free	0.05	0.001
offer	0.04	0.002
just	0.02	0.03
you	0.03	0.04

Spam score:

$$\log P(y = 1) + \log(0.05) + \log(0.04) + \log(0.02) + \log(0.03)$$

Ham score:

$$\log P(y = 0) + \log(0.001) + \log(0.002) + \log(0.03) + \log(0.04)$$

R Implementation

```
# new email
new_text <- "Free offer just for you"

# preprocess SAME as training
new_corpus <- VCorpus(VectorSource(new_text))
new_corpus <- tm_map(new_corpus, content_transformer(tolower))
new_corpus <- tm_map(new_corpus, removePunctuation)
new_corpus <- tm_map(new_corpus, removeWords, stopwords("english"))

# convert to DTM using training vocabulary
new_dtm <- DocumentTermMatrix(new_corpus,
                               control = list(dictionary = colnames(dtm)))

x_new <- as.matrix(new_dtm)

# class priors
p_spam <- mean(df$label == 1)
p_ham  <- mean(df$label == 0)

# scores
score_spam <- log(p_spam) + sum(x_new * log(theta_spam_smooth)) #-12.94004
score_ham  <- log(p_ham)  + sum(x_new * log(theta_ham_smooth)) #-16.63666

# prediction
if (score_spam > score_ham) {
  "spam"
} else {
  "ham"
}
```

Outline

1 Case Study: Email Spam Detection

2 Naive Bayes Model

3 Logistic Regression Model

Naive Bayes Model

We model:

$$P(y \mid \mathbf{x}) \propto P(y) P(\mathbf{x} \mid y)$$

Naive assumption:

$$P(\mathbf{x} \mid y) = \prod_{j=1}^p P(x_j \mid y)$$

Key idea:

- Words are conditionally independent given class

Multinomial Naive Bayes Model

- Class label $y_i \in \{1, \dots, K\}$
- Conditional model:

$$\mathbf{x}_i \mid y_i = k \sim \text{Multinomial}(N_i, \boldsymbol{\theta}_k)$$

- $\boldsymbol{\theta}_k = (\theta_{k1}, \dots, \theta_{kp})$
- Independence assumption:

$$P(\mathbf{x}_i \mid y_i = k) = \frac{N_i!}{\prod_j x_{ij}!} \prod_{j=1}^p \theta_{kj}^{x_{ij}}$$

Naive Bayes Likelihood

■ Joint likelihood:

$$L = \prod_{i=1}^n P(y_i)P(\mathbf{x}_i \mid y_i)$$

■ Log-likelihood:

$$\ell = \sum_{i=1}^n \log P(y_i) + \sum_{i=1}^n \sum_{j=1}^p x_{ij} \log \theta_{yij} + C$$

■ Parameters:

$$\pi_k = P(y = k), \quad \boldsymbol{\theta}_k$$

MLE for Naive Bayes

- Class prior:

$$\hat{\pi}_k = \frac{n_k}{n}$$

- Word probabilities:

$$\hat{\theta}_{kj} = \frac{\sum_{i:y_i=k} x_{ij}}{\sum_j \sum_{i:y_i=k} x_{ij}}$$

- With Laplace smoothing:

$$\hat{\theta}_{kj} = \frac{\sum_{i:y_i=k} x_{ij} + \alpha}{\sum_j \sum_{i:y_i=k} x_{ij} + \alpha p}$$

Naive Bayes Decision Rule

- Posterior:

$$P(y = k \mid \mathbf{x}) \propto \pi_k \prod_{j=1}^p p_{kj}^{x_j}$$

- Log posterior:

$$\log P(y = k \mid \mathbf{x}) = \log \pi_k + \sum_{j=1}^p x_j \log \theta_{kj}$$

Decision:

$$\hat{y} = \arg \max_k \left[\log P(y = k) + \sum_j x_j \log \theta_{jk} \right]$$

Other NB Models

Bernoulli NB:

- Features: binary indicators

$$x_j \in \{0, 1\}$$

(word present or not)

- Model:

$$P(\mathbf{x} \mid y = k) = \prod_j p_{jk}^{x_j} (1 - p_{jk})^{1-x_j}$$

- This depends on presence/absence, not counts

Gaussian NB Model:

- Features: numeric counts
- Model:

$$x_j \mid y = k \sim \mathcal{N}(\mu_{jk}, \sigma_{jk}^2)$$

Gaussian Naive Bayes Model

```
library(e1071)
model <- naiveBayes(as.matrix(dtm), as.factor(df$label))
model$tables

> model$tables[["free"]]
      free
as.factor(df$label) [,1]      [,2]
0 0.00 0.0000000
1 0.15 0.3663475
```

- Converts labels → factor (required for classification)
- Stores feature-wise distributions

R Code

```
# New email
new_text <- "Free offer just for you"

# Preprocess new email
new_corpus <- VCorpus(VectorSource(new_text))
new_corpus <- tm_map(new_corpus, content_transformer(tolower))
new_corpus <- tm_map(new_corpus, removePunctuation)
new_corpus <- tm_map(new_corpus, removeWords, stopwords("english"))

# Build DTM using the training vocabulary
new_dtm <- DocumentTermMatrix(
  new_corpus,
  control = list(dictionary = colnames(dtm))
)

# Predict class using Gaussian Naive Bayes
pred_class <- predict(model, as.matrix(new_dtm))
pred_class # 1
```


Outline

1 Case Study: Email Spam Detection

2 Naive Bayes Model

3 Logistic Regression Model

Logistic Regression Model

- Direct modeling of:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + \exp(-\beta_0 - \mathbf{x}^\top \boldsymbol{\beta})}$$

- Bernoulli likelihood:

$$P(y_i|\mathbf{x}_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

where

$$p_i = \frac{1}{1 + \exp(-\beta_0 - \mathbf{x}_i^\top \boldsymbol{\beta})}$$

- Logistic Regression Log-likelihood:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Gradient and Optimization

- Score function:

$$\nabla \ell = \sum_{i=1}^n (y_i - p_i) \mathbf{x}_i$$

- Solve via:
 - Newton-Raphson
 - Gradient descent
- Convex optimization problem

R Implementation

```
# Packages
install.packages(c("tm", "glmnet"))    # run once
library(tm)
library(glmnet)

# Load data
df <- read.csv("emails.csv", stringsAsFactors = FALSE)

# Text preprocessing
corpus <- VCorpus(VectorSource(df$text))
corpus <- tm_map(corpus, content_transformer(tolower))
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeWords, stopwords("english"))

# Document-term matrix
dtm <- DocumentTermMatrix(corpus)

x <- as.matrix(dtm)
y <- df$label    # 1 = spam, 0 = ham
```

R Implementation

```
# Train/test split
set.seed(123)
train_id <- sample(seq_len(nrow(x)), size = 0.7 * nrow(x))

x_train <- x[train_id, ]
x_test  <- x[-train_id, ]

y_train <- y[train_id]
y_test  <- y[-train_id]

# Fit regularized logistic regression
model_logit <- cv.glmnet(
  x_train,
  y_train,
  family = "binomial",
  alpha = 1
)
```

R Implementation

```
# Predicted probabilities
prob <- predict(
  model_logit,
  newx = x_test,
  s = "lambda.min",
  type = "response"
)

# Convert probabilities to class labels
pred <- ifelse(prob > 0.5, 1, 0)

# Accuracy
mean(pred == y_test)
```

Predict a new email

```
new_text <- "Free offer just for you"

new_corpus <- VCorpus(VectorSource(new_text))
new_corpus <- tm_map(new_corpus, content_transformer(tolower))
new_corpus <- tm_map(new_corpus, removePunctuation)
new_corpus <- tm_map(new_corpus, removeWords, stopwords("english"))

new_dtm <- DocumentTermMatrix(
  new_corpus,
  control = list(dictionary = colnames(dtm))
)

x_new <- as.matrix(new_dtm)

> predict(model_logit, newx = x_new, s = "lambda.min",
          type = "response")
lambda.min
1 0.9577995
```

Predicted probability that the new email is spam:

$$P(y = 1 \mid x_{\text{new}}) \approx 0.9577995$$

The prediction comes from the model with the best cross-validated λ .

Connection Between NB and Logistic

- Naive Bayes implies:

$$\log \frac{P(y = 1|\mathbf{x})}{P(y = 0|\mathbf{x})} = \log \frac{\pi_1}{\pi_0} + \sum_j x_j \log \frac{\theta_{1j}}{\theta_{0j}}$$

- This is linear in $\mathbf{x}$
- Hence:

$$\beta_j = \log \frac{\theta_{1j}}{\theta_{0j}}$$

Key Differences

- Naive Bayes:
 - Models $P(\mathbf{x}|y)$
 - Strong independence assumption
 - Closed-form solution
- Logistic Regression:
 - Models $P(y|\mathbf{x})$
 - No independence assumption
 - Requires numerical optimization

Bias-Variance Tradeoff

- Naive Bayes:
 - High bias
 - Low variance
- Logistic Regression:
 - Lower bias
 - Higher variance
- In high dimension ($p \gg n$):
 - NB often surprisingly competitive

High-Dimensional Considerations

- Sparse data matrix
- Regularization needed:

$$\ell(\beta) - \lambda \|\beta\|_1$$

- Feature selection:
 - Term Frequency–Inverse Document Frequency (TF-IDF)
 - Thresholding

Summary

- Naive Bayes:
 - Fast, interpretable
 - Works well in practice
- Logistic Regression:
 - More flexible
 - Better when independence fails
- Both lead to linear classifiers